

LABORATORIO DI ALGORITMI E STRUTTURE DATI – FERONE

UNIVERSITÀ DEGLI STUDI DI NAPOLI
“PARTHENOPE”



Appunti a cura di
FIORENTINO MICHELE

INTRODUZIONE

I seguenti appunti sono stati creati usando diverse **fonti**:

- “Introduzione agli algoritmi e strutture dati” – Cormen;
- “Automati, linguaggi e calcolabilità” – Hopcroft;
- slide presentate a lezione durante il corso, da parte di Camastra e Ferone;
- appunti presi personalmente durante le lezioni;
- vari siti web (GeeksForGeeks, StackOverflow, ...);
- vari canali youtube (Abdul Bari, fcamuso, ...);
- appunti da parte di Liccardo Giuseppe.

Ne consegue che questi appunti presentano una forte componente personale e sono il risultato dell’amalgamazione di molte fonti diverse. Ho cercato di essere quanto più non ambiguo e fedele alle lezioni dei prof, tuttavia è probabile che ci siano alcuni errori o imprecisioni. Anche l’implementazione degli algoritmi è spesso personale e non è detto che sia la migliore.

Per questo, tali appunti sono da considerare un “sostegno”, e NON hanno l’obiettivo di sostituire le lezioni.

INDICE

LEZ 1 A 5 – INTRODUZIONE AL C++	7
PUNTATORI E REFERENCE IN C++	7
AUTO & NULLPTR	8
OOP NEL C++	9
Classi ed oggetti del C++	9
Costruttori e distruttori	10
• Costruttore di Default	10
• Costruttore di Copia	11
Separare parte dichiarativa ed implementazione	11
Function overloading	12
Metodi Getter e Setter	12
Ereditarietà	13
Upcasting	14
Funzioni virtuale e Polimorfismo	14
Ambiguità	15
Polimorfismo	15
Function Overloading VS Function Override	16
Pure virtual function & abstract class	16
Classe friend	18
Using namespace std	18
Overload di operatori	19
TEMPLATE	20
Vincoli impliciti dei Template	21
Overloading vs Template	21
Overloading esplicito	23
Template come parametri di template	23
STRINGHE IN C++	24
Metodi per la manipolazione delle stringhe	24
stoi – stof	25
FILE IN C++	26
ESEMPI DI OPERAZIONE SU FILE	27
creazione dello stream	27
controllo della corretta apertura del file	27
scrittura/lettura di un file di testo	27
scrittura di un file binario	28
lettura di un file binario	28
Posizionamento in un file	29
stringstream	29
STANDARD TEMPLATE LIBRARY	30
Iteratori	30
Contenitori	30
Vector	30
Reverse Iterator	34
List	35
Contenitore associativi	36
Algoritmi in STL	36
Range based for loops	37
CONCORRENZA	38
Mutex	39

LEZ 6 – BUBBLE SORT, INSERTION SORT & DIVIDE ET IMPERA.....	40
BUBBLE SORT	40
INSERTION SORT	41
DIVIDE ET IMPERA	42
<i>Problema della ricerca binaria.....</i>	<i>42</i>
<i>Problema del massimo sottoarray.....</i>	<i>43</i>
LEZ 7 – MERGE SORT	45
MERGE SORT	45
ALGORITMO	46
• Merge sul posto con sentinelle	46
• Merge a due vie	48
LEZ 8 – QUICK SORT (E VARIANTI).....	50
QUICK SORT	50
RANDOMIZED QUICKSORT	51
MIGLIORAMENTI PER IL QUICKSORT	51
ALGORITMO	52
LEZ 9 – ALBERI & HEAP	55
ALBERI	55
ALBERI BINARI HEAP	56
<i>Max-Heapify</i>	<i>57</i>
<i>Build-Max-Heap</i>	<i>58</i>
ALGORITMI	59
LEZ 10 – HEAPSORT & CODE DI PRIORITÀ.....	60
HEAPSORT	60
CODE DI PRIORITÀ	61
LEZ 11 – ALGORITMI DI ORDINAMENTO LINEARI	64
COUNTING SORT	64
RADIX SORT	67
BUCKET SORT	70
LEZ 12 – ALBERI BINARI DI RICERCA (ABR)	72
OPERAZIONI SU ABR.....	73
TREE-SEARCH.....	73
TREE MINIMUM & TREE MAXIMUM	74
SUCCESSORE & PREDECESSORE	75
INSERT	76
DELETE (con Transplant)	77
ABR IN C++.....	83
Definizione Node e BST.....	84
Implementazione metodi privati di BST	85
Implementazione metodi pubblici di BST	88

LEZ 13/14 – ALBERI RED BLACK (ARB)	89
OPERAZIONI SU ALBERI RB	91
ROTAZIONE	92
INSERT	96
DELETION	101
Transplant per alberi RB	102
Cancellazione in RB	103
La procedura FIXUP	104
LEZ 15 – PD (FIBONACCI E PROBLEMA CATENA DI MONTAGGIO)	111
FIBONACCI IN C++	111
PROBLEMA DELLA CATENA DI MONTAGGIO	112
CARATTERIZZAZIONE DELLA SOTTOSTRUTTURA OTTIMA	113
PSEUDOCODICE	114
FASTEST_WAY – IMPLEMENTAZIONE IN C++	116
LEZ 16 – PD (Distanza di Editing e Zaino 01)	119
Distanza di Editing (di Levenshtein)	119
Codice in C++	120
Zaino 01	121
Pseudocodice	121
Codice in C++	122
LEZ 17 – Algoritmi Greedy (GAS e Codici di Huffman)	124
GREEDY ACTIVITY SELECTOR (GAS)	124
Codice in C++	125
CODICI DI HUFFMAN	126
Costruzione di un codice di Huffman	127
In C++	128
Decodifica di un codice di Huffman	130
LEZ 18 – Grafi (BFS, DFS, Ord. Topologico, Comp. Fort. Conn)	131
CLASSE NODE E CLASSE EDGE	131
CLASSI DEI GRAFI	132
Classe astratta Graph	132
Classe figlia Directed Graph	133
Classe figlia Undirected Graph	134
BFS	135
DFS	136
CLASSIFICAZIONE DEGLI ARCHI	137
ORDINAMENTO TOPOLOGICO (NUOVO)	138
COMPONENTI FORTEMENTE CONNESSE	140
In C++	143
LEZ 19 – Grafi (Insiemi Disgiunti e MST(Kruskal e Prim))	145
OPERAZIONI SU INSIEMI DISGIUNTI PER ALBERI	145
RAPPRESENTAZIONE DI INSIEMI DISGIUNTI MEDIANTE LISTE	146
In C++ (rappresentazione mediante alberi)	147
ALBERO DI COPERTURA DI MINIMO	148
KRUSKAL	149
PRIM	151

LEZ 20 – IL PROBLEMA DEI CAMMINI MINIMI (DIJKSTRA E BM-FORD)	152
IL PROBLEMA DEI CAMMINI MINIMI	152
<i>Archi con pesi negativi</i>	152
<i>Rappresentazione dei cammini minimi</i>	153
<i>Albero dei cammini minimi</i>	153
<i>Sottostruttura ottima dei cammini minimi</i>	153
<i>Corollario</i>	153
<i>Tecnica del Rilassamento</i>	154
PROPRIETÀ DEI CAMMINI MINIMI E DEL RILASSAMENTO	155
ALGORITMO DI DIJKSTRA	156
<i>Teorema della correttezza dell'algoritmo di Dijkstra</i>	158
ALGORITMO DI BELLMAN-FORD	159
<i>Correttezza dell'algoritmo di Bellman-Ford</i>	161
PRECISAZIONI SUI KRUSKAL, PRISM, DIJKSTRA E BELLMAN-FORD	163
LEZ 21 – BACKTRACKING	164
IL PROBLEMA DELLE 8 REGINE	164
<i>In C++</i>	165
EXTRA (PERCORSO HAMILTONIANO)	168
LEZ 22 – TABELLE HASH	169
FUNZIONI DI HASHING	169
<i>Hashing per divisione</i>	169
<i>Hashing per moltiplicazione</i>	170
<i>Hashing universale</i>	171
METODI DI RISOLUZIONE DELLE COLLISIONI	172
<i>Risoluzione delle collisioni per concatenazione</i>	172
<i>Risoluzione delle collisioni con indirizzamento aperto</i>	172

LEZ 1 a 5 – INTRODUZIONE AL C++

Il C++ è un linguaggio **multiparadigma**: è principalmente un linguaggio orientato ad oggetti, ma supporta anche la programmazione procedurale e quella basata sui tipi.

PUNTATORI E REFERENCE in C++

Il **linguaggio C++** mette a disposizione dello sviluppatore un metodo di indirizzamento della memoria alternativo all'uso dei puntatori: le variabili **reference**.

Un **riferimento** va considerato come un **secondo nome** per la variabile. Ogni volta che si accede ad un riferimento (o alias) si accede all'aria di memoria associata alla variabile.

Es.

```
int a = 1;
int& ref_a = a;

int b = 2;
ref_a = b; // Errore: non possiamo cambiare il riferimento dopo la dichiarazione!
int& r; // Errore: una variabile reference deve essere inizializzata!
```

Dal momento della sua definizione, ogni modifica del valore di *ref_a* si riflette automaticamente su *a* e non sarà più possibile cambiare il riferimento, puntando ad esempio ad un'altra variabile (per questo motivo, non si può dichiarare una reference non inizializzata). In pratica, definire una variabile reference corrisponde al fornire un nome alternativo per una variabile già esistente. Per questo motivo esse sono dette anche variabili *alias*.

Generalmente, i riferimenti vengono utilizzati nelle firme (signature) delle funzioni perché risolvono problemi di passaggio per riferimento del C.

Con il passaggio per riferimento all'interno della funzione si utilizzano direttamente i parametri in firma, senza usare, come nel C, l'operatore * (di dereferenziazione).

C++

vs.

C

```
void Swap(int &a,int &b)
{
    int t;
    t = a;
    a = b;
    b = t;
}
```

```
void Swap(int *a,int *b)
{
    int t;
    t = *a;
    *a = *b;
    *b = t;
}
```

Nel main avremo...

```
int main()
{   int x=5, y= 7;
    Swap (x, y);
    cout << x << endl
        << y << endl;
    return 0;
}
```

```
int main()
{   int x=5, y=7;
    Swap (&x, &y);
    cout << x << endl
        << y << endl;
    return 0 ;
}
```

AUTO & NULLPTR

È possibile dedurre il tipo di un oggetto dal suo iniziatore, con la parola chiave **auto**. Si consiglia di usarlo a meno che non ci sia un esplicito motivo contrario (es. la precisione float/double).

```
auto b = true;    // bool
auto ch = 'x';    // char
auto i = 123;     // int
auto d = 1.23;    // double
auto z = sqrt(y); // tipo restituito da sqrt
```

nullptr è un puntatore che non punta ad alcun oggetto, eliminando la potenziale confusione tra interi (0 o NULL) e puntatori. Infatti se assegnassimo nullptr ad una variabile di tipo intero, il compilatore ci riporterebbe un errore.

OOP NEL C++

Classi ed oggetti del C++

Uno dei più grandi vantaggi della OOP è l'ordine e la riutilizzabilità, e la sua importanza si fa sentire particolarmente in progetti medio/grandi.

In C++ la **classe** costituisce la base per la programmazione orientata ad oggetti (OOP). La **classe** è usata per definire la natura di un oggetto ed è l'unità base di **incapsulamento** del C++. Una classe ha un proprio **nome** e contiene due tipi di membri: **campi** e **metodi**.

Quindi, una **classe** definisce una collezione di *oggetti*, ovvero le possibili **istanze della classe**.

Con **capsula** intendiamo il codice compreso fra le due parentesi graffe della classe. Ad esempio " class Myclass { **sono il contenuto della capsula** }.

Una classe a prima vista sembra molto simile ad una struct (che ricordo permette l'aggregazione di più variabili anche se non dello stesso tipo), con la differenza che, per default, tutti i membri della **classe** sono *privati* mentre tutti i membri di una *struct* sono *pubblici*.

Pubblico, protetto e privato costituiscono gli **specificatori di accesso** del C++:

- **public**: tutti i membri definiti all'interno della classe saranno accessibili da qualsiasi punto del programma, e quindi anche da altre classi.
- **private**: i membri definiti all'interno della classe non saranno accessibili al di fuori della classe stessa.
- **protected**: i membri definiti all'interno della classe non saranno accessibili al di fuori della classe stessa, ad eccezione delle classi ereditate (figlie).

Di solito le variabili che descrivono lo stato sono locali e non sono visibili fuori dall'oggetto (**data hiding**). Il *comportamento* di una classe è controllato da funzioni che agiscono su queste variabili di stato.

I membri di una classe (dati e funzioni) sono accessibili tramite ". " o " -> ".

nb: sarebbe più corretto chiamare la funzione di una classe "metodo".

Un esempio di classe è:

```
class Dado {  
    int NumeroFacce;           //NumeroFacce e Valori sono private  
    int Valore;  
  
    public:                   //Da questo momento in poi, tutti i membri sono public  
    void lanciadado( ) { return rand()%6; } //lanciadado è un metodo della classe  
};                             //FA ATTENZIONE AL " ; " PRIMA DI " } ".
```

Nb: È anche possibile creare array di oggetti di una classe.

Costruttori e distruttori

In una classe un **costruttore** è un metodo con lo stesso nome della classe che viene chiamato ogni volta che viene istanziato un oggetto della classe.

Essenzialmente quello che fa un costruttore è eseguire una serie di operazioni ogni qualvolta viene creato un nuovo oggetto, e non è possibile evitare che queste operazioni vengano eseguite.

Ciò è particolarmente utile per evitare che il programmatore effettui queste operazioni ogni volta manualmente, col rischio che possa dimenticare la loro implementazione, automatizzandole, e quindi assicurarsi che l'oggetto venga inizializzato correttamente.

Il **distruttore** permette di deallocare la memoria allocata dal costruttore, che ha sempre lo stesso nome della classe ma preceduto dal carattere tilde "~".

Il distruttore non si può invocare da programma, è invocato dal compilatore quando viene deallocato lo spazio di memoria assegnato all'oggetto.

Il costruttore e il distruttore **non hanno valore di ritorno** (neanche **void**).

Quando si chiama il costruttore vengono prima inizializzate le variabili globali e poi quelle locali, mentre quando si chiama il distruttore vengono prima distrutte le variabili locali e poi quelle globali.

• **Costruttore di Default**

Se il programma non prevede alcun costruttore, il compilatore crea comunque un **costruttore di default**.

Ciò perché ci sono delle situazioni in cui è necessario chiamare in automatico il costruttore, e l'unico costruttore sicuro è quello senza parametri.

Il costruttore di default può tornarci anche utile quando vogliamo inizializzare in automatico un valore alla creazione di un certo oggetto.

Ad esempio, creare un oggetto "dado" senza specificare il numero di facce, perché nel costruttore di default abbiamo specificato che in questo caso assegneremo n facce.

Es.

Includiamo innanzitutto il costruttore di default nella capsula della classe, del tipo "nomeclasse();",

Poi attraverso il costruttore di default chiamiamo un altro costruttore, del tipo "dado::dado() : dado(6){ }", e quindi quando istanziamo un oggetto senza specificare il numero di facce, viene richiamato il costruttore di default che chiama il costruttore e che assegna al dado 6 facce.

• Costruttore di Copia

Il **costruttore di copia** crea un oggetto a partire da un altro oggetto della classe (i valori delle variabili membro vengono copiati).

Utile se dobbiamo lavorare sui dati di una classe.

Prototipo: `ClassName (const ClassName &old_obj);`

es.

```
Class Point
{
private:
    int x, y;
public:
    Point(int ascissa, int ordinata) { x = ascissa; y = ordinata; }

    // Copy constructor
    Point(const Point &p) {x = p.x; y = p.y; } //costante di tipo Point
};

int main()
{
    Point p(10, 15); // Normal constructor is called here
    Point pCopy = p1; // Copy constructor is called here, by assignment
    Point pCopy2(p1); // Copy constructor is called here, by constructor
}
```

Separare parte dichiarativa ed implementazione

Possiamo includere la dichiarazione e l'implementazione di un metodo direttamente nella classe stessa, ma è buona norma per migliorare l'ordine e la leggibilità del codice dividerli.

La dichiarazione del metodo rimane nella capsula della classe, mentre la sua implementazione avviene in un altro punto del codice (o in un altro file). In questo caso, prima del nome del metodo va specificato il nome della classe seguito dall'*operatore di risoluzione di scope* “ :: ” (Vale lo stesso per il costruttore).

La classe con la dichiarazione dei metodi andrà in un “file.h”, ed è buona norma assicurarci che il file non sia stato già incluso attraverso l'istruzione:

```
#ifndef FILE_H_INCLUDED           //if not defined file.h, ovvero se non è incluso
#define FILE_H_INCLUDED          //allora includi file.h

//La mia classe...

#endif                             //Fine dell'if
```

L'implementazione dei metodi avverrà in un file.cpp, dove ci assicureremo che siano incluse tutte le librerie necessarie (e namespace).

Function overloading

La **function overloading** indica la possibilità di usare lo stesso nome per due o più funzioni nella stessa classe. Ciò è possibile solo quando il compilatore è in grado di stabilire quale funzione chiamare e quindi le ridefinizioni di una funzione devono usare:

→ tipi differenti di parametri;

→ numero diverso di parametri.

Es.

```
float area(float lato);  
float area(float base, float altezza);
```

```
//Hanno diverso numero di parametri  
//Vengono trattate come 2 f. diverse
```

Metodi Getter e Setter

L'accesso delle variabili di tipo privato/protected di una classe deve essere mediato attraverso i metodi *getter* e *setter*.

Infatti, come ben sappiamo, sarebbe una pazzia dare libero accesso a tutte le variabili di una classe direttamente al programmatore.

Un metodo **getter** non è altro che un metodo che prende un valore e lo restituisce all'esterno.

Il tipo del metodo è di solito lo stesso della variabile da restituire.

Un metodo **setter** non è altro che un metodo con il quale è possibile settare un nuovo valore.

Il tipo del metodo è *void*.

Es.

```
int getValore ( ){ return valore; }
```

```
void setValore ( int NuovoValore ){ valore = NuovoValore; }
```

Accenno ai qualificatori Static

Una variabile statica è locale ad una particolare funzione. E' inizializzata una sola volta, la prima volta che tale funzione viene chiamata e il suo valore resta inalterato quando si esce dalla funzione, per cui quando si richiama nuovamente la funzione tale variabile ha ancora il valore assegnatogli precedentemente.

Per definire statica una variabile è sufficiente anteporre la parola *static* alla dichiarazione della variabile.

nb: Per richiamare metodi/var direttamente col nome della classe, devono essere *static*.

```
Es.  static int contatore;           //Nell'header.  
      int class::contatore=0;       //Nel file cpp. Inizializzato solo la prima volta, a 0.  
      contatore++;                  //Nel costruttore. Ogni nuovo oggetto incrementa cont.
```

Ereditarietà

L'**ereditarietà** è un meccanismo con cui si instaura una **gerarchia** tra la *classe base* (o superclasse) e la *classe derivata* (o sottoclasse), che aggiunge dettagli.

```
class nomeClasseDerivata : access nomeClasseBase {  
    // corpo della classe derivata  
};
```

tipo di ereditarietà	membri della classe base	membri ereditati in classe derivata
public	public protected private	public protected inaccessibile
protected	public protected private	protected protected inaccessibile
private	public protected private	private private inaccessibile

La classe derivata avrà accesso a tutti i metodi e membri della classe base (a meno che questi siano private).

Costruttori, distruttori, metodi virtuali non sono ereditati in una classe derivata. Se necessari, questi vanno creati.

Un esempio di costruttore della classe derivata che si “appoggia” alla classe base:

```
automobile(int NumRuote, int NumPasseggeri, tipo tipo_auto) :  
    veicolo(NumRuote, NumPasseggeri){  
    this->tipo_auto = tipo_auto;  
}
```

Dove i due punti “ : ” ci fanno capire che il costruttore di automobile (derivato) va a richiamare il costruttore di veicolo (base), il quale si occuperà lui di assegnare il NumRuote e NumPasseggeri all’oggetto.

“tipo_auto” è una caratteristica di automobile, e di questo se ne occuperà proprio il costruttore di automobile.

NB: con **this** facciamo riferimento alla variabile della classe e non del metodo. Si usa quando il parametro del metodo ha lo stesso nome della variabile della classe. Infatti:

this-> tipo_auto; //membro della classe automobile

tipo_auto; //parametro del metodo (In questo caso, il costruttore)

Upcasting

In C++ è possibile creare dei **puntatori** (o reference) del tipo della classe.

Con **upcasting** intendiamo l'utilizzo di un puntatore (o reference) della classe base per far riferimento ad un oggetto della classe derivata.

Ad esempio, supponendo che *Super* sia la classe base, e *Sub* quella derivata:

```
int main() {  
    Super* ptr;    // Super class pointer  
    Sub obj;  
    ptr = &obj;  
  
    Super &ref;    // Super class's reference  
    ref=obj;  
}
```

NOTA BENE:

- Puntando ad un oggetto della classe derivata si può accedere solo ai membri che sono stati ereditati dalla classe base.
- NON vale il viceversa: un puntatore ad una classe derivata non può puntare ad un oggetto della classe base.
- L'upcasting produce però l'**object slicing**, con perdita dei dati membro definiti a livello della classe derivata.
Ad esempio, un oggetto della classe point3D può essere convertito in un oggetto della classe Point, ma comportando la perdita dell'attributo z.

Funzioni virtuale e Polimorfismo

Una **funzione virtuale** è una funzione membro dichiarata in una classe base e ridefinita in una classe derivata.

Le funzioni virtuali sono lo strumento principale del **polimorfismo** ed implementano la filosofia "una sola interfaccia, più metodi".

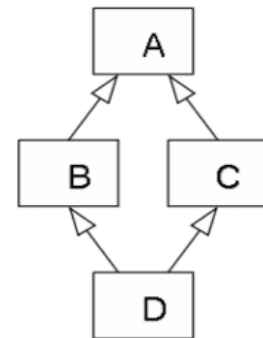
Ad esempio:

```
class base {  
    public: virtual void vfunc() { cout << "vfun() della classe base"; }  
}  
  
class derived : public base {  
    public: void vfunc() { cout << "vfunc() della classe derived"; }  
}
```

Ambiguità

Il meccanismo dell'ereditarietà multipla può portare a situazioni di incongruenza, infatti se due **classi base** hanno un **metodo pubblico** con lo **stesso nome**, e questo metodo viene invocato nella classe derivata, il compilatore non sa quale utilizzare (ovviamente ciò non può accadere con attributi e metodi privati).

Un tipico esempio di tale problema è il “problema del diamante”, che è un’ambiguità che si verifica quando due classi B e C sono derivate da una classe A, e la classe D eredita da B e C. Infatti, se B o C ridefiniscono un metodo di A (overriding) e D non lo ridefinisce a sua volta, non possiamo sapere se D erediterà il metodo da B o C.



Polimorfismo

I metodi pubblici di una classe costituiscono l'interfaccia della classe (cioè i messaggi che l'oggetto può interpretare).

La funzione è assegnata al messaggio in fase di codifica (early binding), tuttavia può essere necessario assegnare la funzione al messaggio run-time (late binding, o binding dinamico).

Con **polimorfismo** intendiamo la possibilità che le classi derivate implementino in modo differente i metodi e le proprietà dei propri antenati, rendendo possibile che gli **oggetti** appartenenti a delle **sottoclassi** di una stessa classe **rispondano diversamente** alle stesse istruzioni.

I **metodi** che vengono ridefiniti in una sottoclasse sono detti **polimorfi**.

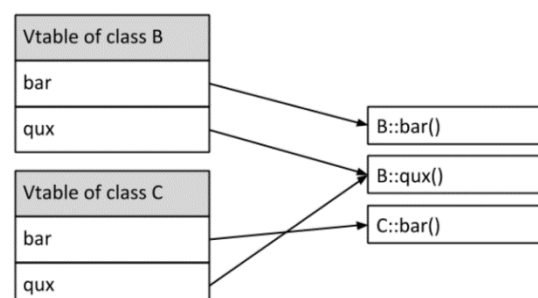
Il polimorfismo è particolarmente utile quando la versione del metodo da eseguire viene scelta **sulla base del tipo di oggetto** effettivamente **contenuto in una variabile** a runtime, invece che al momento della compilazione. Tale funzionalità è detta binding dinamico.

In C++ il polimorfismo e il binding dinamico si ottengono solo: dichiarando **virtual** i metodi polimorfi, e operando tramite puntatori e riferimenti.

Binding Dinamico in C++: esempio

```
#include<iostream>
class B{
public:
    virtual void bar(){};
    virtual void qux(){};
};

class C : public B{
public:
    void bar() override{};
};
```



Function Overloading VS Function Override

La **function overloading** (compile time polymorphism) indica la possibilità di usare lo stesso nome per due o più funzioni nella stessa classe. Ciò è possibile solo quando la "signature" (tipo e numero di parametri) delle funzioni è diversa.

La **function overriding** (o "virtuale", run-time polymorfism) indica la possibilità per una classe derivata di ridefinire una funzione della superclasse con la stessa "signature".

Nb: I costruttori possono essere "overloaded", ma non possono essere "overriding".

Pure virtual function & abstract class

Una **funzione virtuale pura** è una funzione virtuale che non ha una definizione nella classe base, la quale viene detta **classe base astratta**.

Una classe è **astratta** se contiene almeno una funzione virtuale pura.

La sintassi per dichiarare una funzione virtuale pura è:

virtual tipo nome_func(lista_parametri) = 0;

In tal caso ciascuna classe derivata deve prevedere la sua propria definizione altrimenti verrà considerata anch'essa **classe astratta**. Se la classe derivata non riesce a sovrascrivere la funzione virtuale pura, si verificherà un errore in fase di compilazione. Se ne deduce che non si può istanziare un oggetto di una classe astratta.

```
class number {
protected:
    int val;
public:
    void setval(int i) { val = i; }
    virtual void show() = 0;
};

class hextype : public number {
public:
    void show() {
        cout << "hex.: ";
        cout << hex << val << "\n";
    }
};

class dectype : public number {
public:
    void show() {
        cout << "dec.: ";
        cout << val << "\n";
    }
};

class octtype : public number {
public:
    void show() {
        cout << "oct.: ";
        cout << oct << val << "\n";
    }
};

int main()
{
    dectype d;
    hextype h;
    octtype o;
    d.setval(20);
    d.show();
    h.setval(20);
    h.show();
    o.setval(20);
    o.show();
    return 0;
}
```

dec.: 20
hex.: 14
oct.: 24

ESEMPIO DI VIRTUAL

Pensiamo a una Linked List di impiegati, dove distinguiamo due classi: Impiegato e Manager, quest'ultima che deriva da impiegato.

Supponiamo che voglia stampare le informazioni dell'Impiegato (o del Manager) e che quindi io abbia una funzione "getInfo()". Supponiamo che nella Linked List i nodi siano "puntatori ad impiegato".

Il problema è che essendo la LL una lista di puntatori ad Impiegato, quando andremo a chiamare getInfo() per un oggetto di classe Manager, mi chiamerà comunque il metodo di "Impiegato".

Quindi, noi dobbiamo dire al programma di non considerare il metodo di Impiegato ma quello di Manager, e possiamo farlo attraverso una virtual.

Facciamo quindi un override, così che, anche se si tratta di un puntatore ad Impiegato, mi chiama il "printlnfo()" di Manager.

Es:

```
class Impiegato{
    std::string nome;
    float salario;
public:
    Impiegato(std::string, float);
    void setNome(std::string);
    void setSalario(float);
    std::string getNome();
    float getSalario();

    virtual void printInfo();
};

class Manager : public Impiegato {
    float bonus;
public:
    Manager(std::string, float, float);
    void setBonus(float);
    float getBonus();

    void printInfo() override; /*
};

void ShowList(std::list<Impiegato*> LL){
    std::list<Impiegato*>::iterator it;
    for(it=LL.begin(); it!=LL.end(); ++it)
        (*it)->printInfo();
    std::cout<<"\n";
}
```

Classe friend

Una **classe friend** è una classe che può accedere ai membri protetti e privati di un'altra classe nella quale è dichiarata come *friend*.

Per esempio, una classe `LinkedList` può così aver accesso ai membri privati di un `Nodo`.

In modo simile, una **funzione friend** è una funzione alla quale viene data la possibilità di accedere ai membri privati e protetti di un'altra classe.

```
class Node {
private:
    int key;
    Node* next;
    /* Other members of Node Class */

    // Now class LinkedList can
    // access private members of Node
    friend class LinkedList;
};
```

```
class Node {
private:
    int key;
    Node* next;

    /* Other members of Node Class */
    friend int LinkedList::search();
    // Only search() of linkedList
    // can access internal members
};
```

Using namespace std

Attraverso lo **scope resolution operator** (`::`) è possibile implementare un metodo al di fuori della sua classe.

Esso informa il compilatore che un certo metodo appartiene ad una certa classe, e che quindi è nel suo *ambito di visibilità* (**scope**).

Es. `nome_classe :: nome_metodo`

Il **namespace** è una regione dichiarativa il cui obiettivo è “localizzare” i nomi degli identificatori evitando collisioni di nome.

È possibile creare un namespace come: `namespace nome_space { //codice... }`

Overload di operatori

Il C++ offre la possibilità di **ridefinire** le **operazioni standard** (uguaglianza, addizione, sottrazione, ecc.).

È particolarmente utile applicato alle classi, ad esempio quando vogliamo verificare se due oggetti sono uguali, o se vogliamo sommarli/sottrarli (si pensa alla somma/sottrazione di due vettori). Esso è definito come:

```
return_type class_name::operator# (arg_list)
```

Dove # indica l'operatore che si vuole ridefinire.

Esempio 1: somma di due vettori (V2 è l'operatore a destra del '+'):

```
Vettore2D Vettore2D::operator+ (Vettore2D V2){  
    Vettore2D tmp;  
    tmp.Vx = this -> Vx + V2.Vx;  
    tmp.Vy = this -> Vy + V2.Vy;  
    return tmp;  
}
```

W = U + V;

Esempio 2: Uguaglianza fra due date (restituisce True se tutti i param sono uguali):

```
bool Data::operator==(Data D2){  
    return giorno==D2.giorno && mese==D2.altra && anno==D2.anno;  
}
```

```
if ( D1 == D2)      cout << "date uguali";      else      cout << "date diverse";
```

Member initializer list

La **lista di inizializzazione di membri** è una caratteristica propria dei *costruttori di classe*; essa appare sempre tra la lista di argomenti del costruttore e il suo corpo ed è preceduta da ': '.

ESEMPLI:

```
class Complex {  
private:  
    double Re;  
    double Im;  
public:  
    Complex(double,double);  
    /* ... */  
};  
  
Complex::Complex(double a,double b) : Re(a), Im(b) { ... }
```

equivalente al costruttore con parametri

```
Complex::Complex(double a,double b)  
{  
    Re = a;  
    Im = b;  
    ...  
}
```

inizializzazione dei membri prima dell'esecuzione del corpo del costruttore

```
class Complex {  
private:  
    double Re;  
    double Im;  
public:  
    Complex(Complex &z);  
    /* ... */  
};  
  
Complex::Complex(Complex &z) : Re(z.Re), Im(z.Im) { }
```

equivalente al costruttore di copia

```
Complex::Complex(Complex &z)  
{  
    this->Re = z.Re;  
    this->Im = z.Im;  
}
```

TEMPLATE

L'idea dei **template** è quella di rendere una *funzione* o una *classe generica* **parametrica** al tipo del parametro (con parametrico si intende che dipende dal parametro).

Possiamo immaginare un template come una soluzione più compatta dell'utilizzo delle funzioni overload.

Ad esempio, se volessimo cambiare il segno di un dato numero, dovremmo inserire svariate funzioni override, una per ogni tipo (int, float, double), dovendo ripetere lo stesso codice più volte.

Con **template** invece basta inserire il codice una sola volta, sarà poi lui a capire di che tipo è la variabile e si adatterà di conseguenza.

Ad esempio:

```
template <typename T> //Dove T è il tipo che verrà
T cambiasegno( T numero ){    return -numero;    } //deciso in base alla var.
```

Allo stesso modo, i template vengono utilizzati per le **classi** per evitare di creare svariate classi simili.

Ad esempio:

```
template <class T> //Valore della classe numero ha un tipo
class Numero {    //generico (potrebbe essere int, float...)
    T valore;    //Esso varia in base al tipo della var.
public:
    numero( T n ) { valore = n; }
    T GetValore ( ) { return valore; }
};
```

NB: Non c'è alcuna ragione per preferire **class** a **typename** o viceversa, salvo alcuni casi particolari in cui è necessario usare class.

Una funzione generica viene anche chiamata **funzione template**. Quando il compilatore crea una versione specifica di una funzione generica, si dice che ha creato una **funzione generata**.

Una classe generica viene anche chiamata **classe template**, e può avere un numero qualsiasi di parametri predefiniti e/o template.

Per definire i **metodi non inline**, bisogna dichiarare per ogni membro che si tratta di codice relativo ad un template e specificare il parametro del template.

```
Es.  template <typename T>
      T nomeclasse<T>::Metodo(...){...}
```

```
NomeClasse <tipo> Oggetto;           //Istanza della classe
```

Vincoli impliciti dei Template

Nel caso stessimo usando gli operatori (<,>==...) con i template, quando un tipo **T** viene sostituito con un tipo non standard, ad esempio una classe definita dall'utente, il compilatore segnalerà un errore se non è stato **ridefinito** l'operatore utilizzato.

Infatti, possiamo pensare come abbia poco senso e sia ambiguo dire che un oggetto *cane* sia $<$ di un altro oggetto cane, a meno che non siamo stati proprio noi a ridefinire l'operatore $<$.

Overloading vs Template

Da notar bene che le funzioni template sono **limitate** rispetto al caso generale dell'overloading perché non si possono specificare comportamenti diversi al variare del tipo dell'argomento della funzione.

Stack generico

```
template <typename T>
class STACK
{
private:
    int Dim,    // massima occupazione della pila
        Top;   // ultimo elemento inserito
    T *Vet;    // pila di T
public:
    STACK ()
    {
        Vet = nullptr; // pila vuota
        Dim = 0;
        Top = -1;
    }
    void Push (T Elm);
    bool Pop  (T &Elm);
};
```

```

template <typename T> void STACK<T>::Push (T Elm)
{
    if (Top == Dim-1)    // se lo Stack è pieno
    {
        T *App;
        App = new T[Dim+10];    // trasloca lo stack
        for (int i=0; i<Dim; i++) App[i] = Vet[i];
        Dim += 10;
        delete [] Vet;
        Vet = App;                // allunga lo stack
    }
    Vet[ ++Top ] = Elm;
}

template <typename T> bool STACK<T>::Pop (T &Elm)
{
    bool Risp;
    if (Top >= 0) Risp = true, Elm = Vet[Top--];
    else        Risp = false;    // stack vuoto
    return Risp;
}

```



ASD Parte II – Prof. A. Ferone

```

#include <iostream>
using namespace std;

#include "STACK.h"
#include "STACK.cpp"

main()
{
    int    N;
    float  F;
    STACK <int> Sti;
    STACK <float> Stf;

    Sti.Push (25);
    Stf.Push (3.141592);

    if ( Sti.Pop(N) ) cout << N << endl;
    if ( Stf.Pop(F) ) cout << F << endl;
}

```

Overloading esplicito

È possibile eseguire l'**overloading esplicito** di una funzione generica.

La versione modificata tramite overloading, **nasconderà** la funzione generica relativa a quella specifica versione.

In generale, se è necessario impiegare versioni diverse di una funzione per molti tipi di dati, è meglio utilizzare funzioni modificate tramite overloading piuttosto che funzioni generiche.

Es:

```
template <typename X>
void Swap (X &A, Y &B)
{ ..... }

void Swap (int &A, int &B)
{ ..... }

main()
{ int i = 5, j = 10;
  char a = 'K', b = 'H';
  Swap (i, j);           // Versione modificata!
  Swap (a, b);           // Istanza di funzione generica
}
```

Template come parametri di template

L'argomento di un template può essere esso stesso un template:

```
template <class T1, template <class>
class C> class MiaClasse {...}
```

Questo permette la creazione e l'utilizzo di **metatemplates** (templates istanziati con templates) molto sofisticati.

La Standard Template Library fa uso di questa possibilità.

Stringhe in C++

Per poter utilizzare le stringhe in stile C++ bisogna includere il file di **libreria string**.
Con le stringhe in stile C++ la quantità di memoria allocata viene adattata automaticamente, al contrario di come avveniva in C.

Con le stringhe in stile C++ è possibile utilizzare l'**operatore di assegnazione** per assegnare ad esse un carattere, un'altra stringa in stile C++ o una stringa in stile C.

Le stringhe possono essere inizializzate come:

```
string s1="Sono una stringa in stile C++"; //Inizializzazione classica
```

```
string s2("Sono una stringa in stile C++"); //Inizializzazione in stile OOP tramite costruttore
```

Esempi di utilizzo di stringhe:

```
char s2[15]="con "; //sono una stringa in stile C
string s1, s3, s4, s5; //siamo stringhe in stile C++
s1="Operare "; //AMMESSO
s3=s2; //AMMESSO
s5='.'; //AMMESSO
s4=s1+s3+"le stringhe"+s5; //CONCATENAMENTO
```

Metodi per la manipolazione delle stringhe

Le stringhe in stile C++ sono **oggetti** dotati di funzioni membro (o **metodi**) che è possibile richiamare su una stringa tramite la classica notazione puntata *nomeOggetto.attributo* e *nomeOggetto.metodo*. (es. `my_string.length()`).

L'**accesso agli elementi** di una stringa può realizzarsi con il metodo `at(int i)` o tramite operatore `[ ]`. Il primo carattere è nella posizione zero.
(es. `string.at(1)` è la stessa cosa di `string[1]`).

L'**inserimento** di una stringa in un'altra può essere realizzato con il metodo `string& insert(int start, string s)`

- *start* è la posizione da cui inizia l'inserimento; le posizioni si iniziano a contare da zero;
- *s* è la stringa da inserire.

L'**estrazione** di una sotto-stringa viene fatta tramite il metodo `string& substr(int start,int n)`

- *start* è la posizione dalla quale inizia l'estrazione (le posizioni si iniziano a contare da zero);
- *n* è il numero di caratteri da estrarre; è opzionale: se non specificato estrae fino all'ultimo carattere;
- NOTA – la stringa sulla quale si richiama il metodo rimane invariata.

La **cancellazione** di una sotto-stringa viene fatta tramite il metodo `string& erase(int start, int n)`

- *start* è la posizione di inizio della cancellazione e si inizia a contare da zero;
- *n* è il numero di caratteri da cancellare; è opzionale: se non specificato cancella fino all'ultimo carattere;

Si può **cancellare e sostituire** una sotto-stringa all'interno di una stringa tramite il metodo `string& replace(int start, int n, string s)`

- *start* è la posizione di inizio della cancellazione e sostituzione (si inizia a contare da zero);
- *n* è il numero di caratteri da cancellare;
- *s* è la stringa da aggiungere.

Si può **ricercare** una sotto-stringa all'interno di una stringa tramite il metodo `int find(string s, int start)`

- *s* è la sotto-stringa da cercare;
- *start* è la posizione dalla quale iniziare la ricerca
- il metodo restituisce la posizione del primo carattere della sotto-stringa trovata, contata a partire da zero.

Si può **convertire** una stringa in stile C++ in una in stile C (vettore di char, *char**) con il metodo `char* c_char()`. Questo metodo è utile per rendere il codice C++ compatibile con le librerie che richiedono l'utilizzo di vettori di char e costanti stringa.

stoi – stof

Nel C++11 sono state introdotte funzioni che convertono una **string** in un valore numerico (stoi in int, stof in float).

```
int stoi(const string& Stringa, size_t* Quanti =  
nullptr, int Base = 10)  
int stof/stod(const string& Stringa, size_t* Quanti =  
nullptr)
```

- **Stringa** è un oggetto di tipo string o wstring;
- **Quanti** (se != nullptr) è un puntatore al numero di caratteri usati (per passaggio per indirizzo), ovvero è un indice sul primo carattere non numerico nella stringa;
- **Base** è la base del sistema di numerazione usato (tra 2 e 36, estremi compresi).

La stringa **deve** iniziare con un carattere numerico, viceversa verrà segnalato un errore.

```
string Str1 = "45ABC";
string Str2 = "3.14159.34";
string Str3 = "words";

int N1 = stoi(Str1); cout << N1 << endl;
float N2 = stof(Str2); cout << N2 << endl;
int N3 = stoi(Str3); cout << N3 << endl;
```

```
45
3.14159
terminate called after throwing an instance of
'std::invalid_argument' what(): stoi
This application has requested the Runtime to
terminate it in an unusual way.
```

File in C++

Prima di aprire un file bisogna creare uno **stream** (e aggiungere la libreria fstream). Esistono 3 tipi di stream:

Input:	ifstream in;	//Per leggere dati da file
Output:	ofstream out;	//Per scrivere dati su file
Input/Output:	fstream io;	//Per leggere/scrivere dati da/su file

Attraverso la funzione **open()**, la quale è un metodo delle classi precedenti, è possibile **aprire un file** collegandolo ad uno stream.

I suoi prototipi sono.

```
void ifstream::open( const char *filename, ios::openmode mode= ios::in);
void ofstream::open( const char *filename, ios::openmode mode= ios::out);
void fstream::open ( const char *filename, ios::openmode mode= ios::in | ios::out);
```

I vari openmode sono:

	ios::app;	//append
	ios::ate;	//seek end of file (At The End)
	ios::binary	//file binario
	ios::in	//per leggerlo
	ios::out	//per scrivere
	ios::trunc	//sovrascrive file esistente
dep.	ios::nocreate	//apre il file solo se esiste. Equivale a if(fp.fail()) cerr<<"errore!";
dep.	ios::noreplace	//apre il file solo se non esiste.

NB: Se non viene specificato un openmode, viene utilizzato quello di default. Possiamo aprire il file in più modalità usando l'operatore **| pipe**.

ESEMPI DI OPERAZIONE SU FILE

creazione dello stream

```
ofstream out("mystream.txt");           //crea lo stream di output e gli associa il file
ifstream in ("mystream.txt");           //crea lo stream di input e gli associa il file

ofstream out("mystream.bin", ios::out | ios::binary); //...per scrittura file binario
ifstream in("mystream.bin", ios::in | ios::binary);   //...per lettura file binario
```

controllo della corretta apertura del file

```
if ( !out ){                               if( !in ){
    cerr << "Errore in apertura file\n";    cerr<<"Errore in apertura file\n";
    exit(1);                                exit(1);
}
```

Per verificare che il file sia stato aperto correttamente si può anche utilizzare il metodo **fail()**:

```
if( fp.fail() ) {
    cout << "Errore in apertura file\n";
    exit(1);
}
```

Per chiudere -> out.close(); in.close();

scrittura/lettura di un file di testo

```
out<<"Nel mezzo del cammin di nostra vita\n";           //Scrive nel file riga per riga
out<<"mi ritrovai per una selva oscura\n";
out<<"che la diritta via era smarrita!\n";
```

```
string str;
while ( in ){
    getline( in, str );
    cout << str << endl;
}
//Legge il file riga per riga finché non è
//finito, ovvero finché "in" è vero.
//Quel che facciamo e ottenere una riga dal
//testo e copiarlo in una stringa di appoggio
//" str ".
```

Il metodo **getline** legge dalla posizione corrente al più **n** caratteri fermandosi al primo carattere EOF, newline o delimitatore *delim*.

scrittura di un file binario

Ci sono due modi in cui è possibile su un file binario.

Caso 1:

```
int n=10;           //È il modo più semplice, scrive il valore della variabile n
out << n;           //nel file di testo
```

Quello che accade è che all'oggetto associato a out (es. mystream.txt) esegue le seguenti operazioni:

- 1- Interpreta i 4 byte con cui l'intero n è rappresentato nella RAM;
- 2- Converte questo valore nei byte ASCII '1' e '0';
- 3- Inserisce questi due byte nel file di testo.

Caso 2 (A **blocchi di byte**):

```
int a[ ] = {0,1,2,3,4,5,6,7,8,9};           //Supponiamo di avere un array di dati da dover
                                              //inserire nel file

out.write((char*) a, sizeof(a);             //Copiamo ogni elemento di a nel file.
//È la stessa cosa di:                      for(k=0; k<10; k++)
                                              out.write((char*)&vec[k], sizeof(int);
```

Quello che accade è che il blocco dei 4 byte contenuti nella variabile n vengono scritti nel file così come sono, ossia secondo la **rappresentazione interna** della macchina.

lettura di un file binario

```
int dato;           //Legge il file byte per byte finché non è finito,
while ( !in.eof() ) { //ovvero finché non si raggiunge "in.eof", ovvero
    in.read(dato,sizeof(dato)); //il carattere di fine file. Copiamo ogni byte nella
    cout << dato << endl;      //variabile di appoggio "dato" e la stampiamo a
}                               //schermo.
```

Quindi, ricapitolando, usiamo:

1. il metodo **write()**, per SCRIVERE un blocco di byte in un file binario;
2. il metodo **read()**, per LEGGERE un blocco di byte da un file binario.

Posizionamento in un file

In un file, dopo un'operazione di lettura/scrittura, la successiva operazione di lettura/scrittura viene realizzata a partire dal primo byte successivo all'ultimo che è stato letto/scritto dal/nel file. Quindi le operazioni avvengono in maniera **sequenziale**.

È possibile spostarsi a piacere in un file *riposizionando* il puntatore a File.

- **seekg()**, per il riposizionamento del puntatore delle operazioni di LETTURA (la g finale sta per *get*); esso ha la seguente sintassi:

nomeOggetto.seekg(int offset, const start);

dove *offset* rappresenta il *numero di byte* di cui riposizionare il puntatore di lettura in avanti, nel caso di un valore positivo, o all'indietro, nel caso di un valore negativo, a partire dalla posizione espressa dalla costante *start* che può assumere uno dei seguenti valori:

1. **ios::beg**, inizio del file;
2. **ios::end**, fine del file;
3. **ios::cur**, posizione corrente del puntatore del file;

es. int pos=100;
fileBinario.seekg((pos-1)*sizeof(int), ios::beg); //si posiz. per leggere il 100° intero

- **seekp()**, per il riposizionamento del puntatore delle operazioni di SCRITTURA (la p finale sta per *put*). Per esso valgono esattamente le stesse cose dette per il metodo *seekg()*.

NB: Nella creazione dello stream "in" e "out" possono avere qualsiasi nome. Es. "FileBin"

StringStream

La classe stringstream associa ad un oggetto stringa uno stream, permettendoti di leggere dalla stringa come se fosse uno stream (come in cin). È simile a sprintf() e sscanf().

Questo ci permette di "stampare" utilizzando l'operatore << su una stringa.

```
#include <string>
#include <iostream>
#include <sstream>
using namespace std;
int main ()
{
    stringstream ss;
    double d = 2.4;
    ss <<"Il valore di "<<d<<" al quadrato è : "<< d*d;
    cout << ss.str() << endl;
    return 0;
}
```

Standard Template Library

La libreria standard STL è una libreria di classi di contenitori, algoritmi ed iteratori. È una libreria *generica*: tutti i suoi componenti sono parametrizzati mediante l' utilizzo dei template.

Iteratori

Gli iteratori sono dei **puntatori** agli elementi di un contenitore e ci permettono di muoverci all'interno di esso. In particolar modo, distinguiamo:

- **Iteratori monodirezionali**: permettono di accedere all'elemento successivo o al precedente.
- **Iteratori bidirezionali**: permettono di accedere sia all'elemento successivo che al precedente.
- **Iteratori ad accesso casuale**: permettono di accedere ad un qualunque elemento del contenitore.

Contenitori

Un **contenitore** è un oggetto capace di immagazzinare altri oggetti e che possiede metodi per accedere ai suoi elementi.

Ogni contenitore ha un **iteratore** associato che permette di *muoversi* tra gli elementi contenuti.

Una **sequenza** è un contenitore di lunghezza variabile i cui elementi sono organizzati **linearmente**. È possibile aggiungere e rimuovere elementi: **Vector** e **List**.

Un **contenitore associativo** è una sequenza che permette un efficiente accesso ai suoi elementi basato su una **chiave**: **Map**.

Vector

I **vector** sono contenitori sequenziali per rappresentare array che possono **cambiare size**. Le locazioni di memoria sono contigue. L'accesso è casuale, cioè vi è un veloce accesso agli elementi, tuttavia l'inserimento e l'estrazioni sono lenti.

È un template, per cui può essere utilizzata con più tipi di dato.

Es. `vector<int> v = { 7, 5, 16, 8 };`

Operazioni tramite vector (dove v rappresenta un qualsiasi vettore):

- `v.pop_back();` //elimina l'ultimo elemento
- `v.push_back(elem);` //Inserisce l'elemento in coda
- `v.size();` //restituisce la dimensione del vettore
- `v.resize( n );` //Modifica la dimensione del vettore
- `v.front();` //restituisce il primo elemento
- `v.back();` //restituisce l'ultimo elemento
- `v.begin();` //restituisce l'iteratore al primo elemento del vettore
- `v.end();` //restituisce l'iteratore oltre l'ultimo elemento del vettore
- `v.insert(v.begin() + integer, new value);` //aggiunge elemento PRIMA dell'indice specificato
- `v.erase(v.begin() + integer)` //elimina elemento dell'indice specificato
- `v[ i ];` //o anche `v.at( i );` restituisce l'elemento di posizione i
- `v.clear();` //svuota l'array, lasciando il contenitore vuoto
- `v.empty();` //Restituisce un valore booleano (1) se v è vuoto

NB: begin e end hanno bisogno anche della libreria <iterator>, per visualizzare l'elemento dobbiamo usare i puntatori. End va oltre l'ultimo elemento quindi per ottenere proprio l'ultimo dovremo sottrarre 1 all'indirizzo.

Per usare begin e end con gli iteratori dovremo usare una variabile di tipo `vector<tipo> iterator`, dichiarata come “ `vector<tipo>::iterator it;` “.

Es. visualizziamo primo ed ultimo elemento di un vector:

```
cout<<*v.begin();
cout<<*(v.end()-1);
```

Es. di visualizzazione con iterator:

```
vector<int>::iterator it;
```

```
for(it=v.begin(); it!=v.end(); ++it) //o anche it++, non sembra esserci differenza
cout<<*it<<" ";
```

OPZIONALE (COSE PRESE DA PROG II)

È possibile creare una **matrice mediante vector**, attraverso il comando:

- `vector<vector<int>>>A;` //Crea un ptr a vector di vector
- `vector<vector<int>>>A {{1,2,3},{4,5},{6}};` //Dichiarazione m. di vector explicit.
- `vector<vector<int>>>A (m, vector<int>(n));` //Inizializza la matrice di m righe e n colonne a 0. 2° parametro opzionale.
- `vector<vector<int>>>A (m, vector<int>(n,num));` //Inizializza un matrice di m righe e n colonne a *num*.

Se inizializziamo una matrice come nel primo caso, dovremo poi ricordarci di assegnare righe e colonne tramite `resize`.

`A.resize(n)` per le colonne, `A[ i ].resize(m)` per le righe.

Una vector di **stringhe** può essere inizializzato come:

```
vector<string> my_string;
```

È possibile cercare una stringa all'interno di un vector di stringhe con la funzione **find()**, della libreria `<algorithm>`.

Find() ritorna un indice al primo elemento dell'intervallo [primo,ultimo] che è uguale al valore. Se non viene trovato l'elemento, la funzione ritorna l'ultimo elemento dell'intervallo.

```
IteratorInput find(inizio, fine, valore);
```

es.

```
vector<string>::iterator it=find(my_string.begin(), my_string.end(), str);
```

Con **distance** calcoliamo il numero di elementi fra due iteratori.

Es.

```
distance(my_string.begin(), it);
```


Copia di file su vettori di stringhe

```
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;
int main()
{
    vector<string> v;
    ifstream in("Fillvector.cpp");
    string line;
    while(getline(in, line))
        v.push_back(line); // Add the line to the end
    // Add line numbers:
    for(int i = 0; i < v.size(); i++)
        cout << i << ": " << v[i] << endl;
    return 0;
}
```

Separazione delle parole

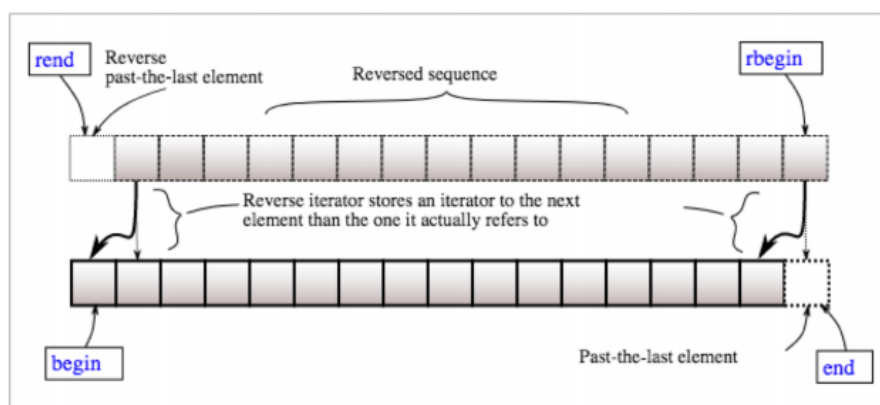
```
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;
int main()
{
    vector<string> words;
    ifstream in("GetWords.cpp");
    string word;
    while(in >> word)
        words.push_back(word);
    for(int i = 0; i < words.size(); i++)
        cout << words[i] << endl;
    return 0;
}
```

Esempio di iteratori

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
int main() {
    vector<string> SS;
    SS.push_back ("The number is 10");
    SS.push_back ("The number is 20");
    SS.push_back ("The number is 30");
    cout << "Loop by index:" << endl;
    for(int ii=0; ii < SS.size(); ii++) {    cout << SS[ii] << endl; }
    cout << endl << "Constant Iterator:" << endl;
    vector<string>::const_iterator cii;
    for(cii=SS.begin(); cii!=SS.end(); cii++) {    cout << *cii << endl; }
    cout << endl << "Reverse Iterator:" << endl;
    vector<string>::reverse_iterator rii;
    for (rii=SS.rbegin(); rii != SS.rend(); ++rii) { cout << *rii << endl; }
    cout << endl << "Sample Output:" << endl;
    cout << SS.size() << endl;
    cout << SS[2] << endl;
    swap(SS[0], SS[2]);
    cout << SS[2] << endl;
    return 0;
}
```

Reverse Iterator

Il `reverse_iterator` è una classe che inverte le direzioni di un dato iteratore. In altre parole, quando gli viene fornito un iteratore bidirezionale, `std::reverse_iterator` produce un nuovo iteratore che si muove dalla fine all'inizio della sequenza definita dagli iteratori bidirezionali sottostanti.

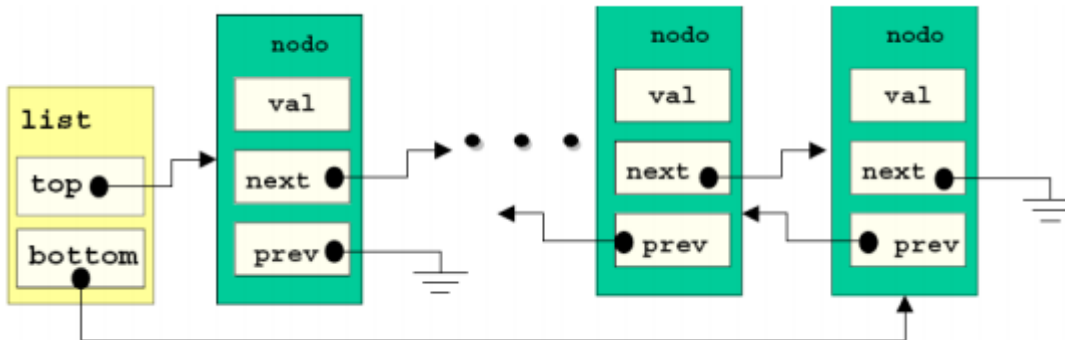


Es. `std::string s = "Hello, world";`
`std::reverse_iterator<std::string::iterator> r = s.rbegin();`

List

Una **list** è una sequenza che impiega un tempo costante per l'inserimento e la cancellazione di elementi in ogni punto della lista.

L'iteratore è bidirezionale. Le locazioni di memoria non sono contigue, inoltre nonostante l'inserimento e l'estrazione siano veloci, la ricerca è lenta.



Esempio:

```
#include <list>
#include <cstdlib>    // per rand()
#include <iostream>
using namespace std;
int main() {
    list<int> container;
    int val;
    for (int i=0; i<10; i++) {
        val = (int)((float)rand()/RAND_MAX*10);
        container.push_back(val);
    }
    list<int>::iterator it1;
    for ( it1=container.begin(); it1!= container.end(); it1++)
        cout << "elemento della lista: " << *it1 << endl;
    return 0;
}
```

Contenitore associativi

I **contenitori associativi** sono contenitori di coppie (key, value) e possiedono un iteratore bidirezionale. Un esempio di contenitore associativo è **map**:

In una map, gli elementi sono ordinati secondo la *chiave*, ed ogni chiave è univoca.

È possibile inserire un elemento tramite `mymap.insert(pair<type, type>(value1, value2));`; `make_pair` è simile a `pair`, solo che deduce il tipo (fa uso di template), e quindi ci basta scrivere: `mymap.insert(make_pair(value1, value2));`

Esempio:

```
#include <map>
#include <iostream>
#include <string>
using namespace std;
int main() {
    map<string,int> amap;
    amap["Primo"] = 1;
    amap["Secondo"] = 2;
    cout << "Size : " << amap.size() << endl;
    amap["Terzo"] = 3;
    amap["Quarto"] = 4;
    amap.insert(pair<string,int>(string("Quinto"),5));
    amap.insert(make_pair("Sesto",6));
    cout << "Size : " << amap.size() << endl;
    map<string,int>::iterator it;
    for ( it=amap.begin(); it!=amap.end(); it++)
        cout << "map : " << it->first << " " << it->second << endl;
    cout << amap.find("Terzo")->second << endl; //se Terzo non esiste?
    cout << amap.at("Quinto") << endl;
    amap.clear(); //rimuove gli elementi dalla mappa
    return 0;
}
```

Algoritmi in STL

Gli algoritmi in STL sono delle funzioni globali capaci di agire su contenitori differenti. Sono incluse operazioni di **ordinamento** (sort, merge, min, max...), di **ricerca** (find, count equal...), di **trasformazione** (transform, replace, fill, rotate, shuffle...) e **generiche operazioni numeriche** (accumulate, adjacent, difference...).

Range based for loops

Dalla versione **C++ 11** è possibile usare il **range-based for loop** (ciclo for su sequenze). Da un punto di vista semantico, questo nuovo costrutto è del tutto equivalente al tradizionale ciclo **for**. Tuttavia, soprattutto nel caso in cui il ciclo sia basato sull'uso di **iteratori**, la nuova sintassi risulta più leggibile e quasi auto esplicativa.

La sintassi del ciclo basato su intervalli si compone di 3 elementi principali:

- 1- *Range-declaration*: la dichiarazione di una variabile avente lo stesso tipo degli elementi dell'intervallo.
- 2- *Range-expression*: una espressione, anche un valore temporaneo, il cui valore sia una sequenza di elementi.
- 3- Il corpo del ciclo, in cui possono apparire le istruzioni **break** e **continue**.

Es.

```
for (int n : v) { std::cout << n << "\n"; }
```

Esempi:

```
vector<int> V;  
for (int i=1; i<= 100; i++) V.push_back(i*i);
```

Sono equivalenti le formulazioni:

```
for (int i=0; i<V.size(); i++)  
    cout << V[i] << " ";  
for (vector<int>::iterator It = V.begin(); It!= V.end(); It ++)  
    cout << *It << " ";  
for (auto It = V.begin(); It != V.end(); It ++ )  
    cout << *It << " ";  
for (auto P = begin(V); P != end(V); P ++ )  
    cout << *P << " ";  
for (auto X : V )  
    cout << X << " ";
```



Se vogliamo fare riferimento agli elementi per una **modifica**, occorre utilizzare il reference **&**.

```
int Seq[] = {0,1,2,3,4,5,6,7,8,9};  
...  
for (auto& X : Seq)  
    if (X%2 == 0) X ++;
```

Concorrenza

La **concorrenza** è una caratteristica dei sistemi di elaborazione nei quali può verificarsi che un insieme di processi o sottoprocessi (**thread**) computazionali sia in esecuzione nello stesso istante. Tale sistema viene appunto chiamato **sistema a concorrenza** (o sistema concorrente).

Un **task** è un calcolo potenzialmente eseguibile in concorrenza con altri; es. una funzione **f()**;

Un **thread** è la rappresentazione system-level di un task in un programma.

Per compilare programmi che utilizzano thread è necessario aggiungere l'opzione -lpthread del linker.

Possiamo generare altri thread, costruendo un oggetto **thread** con il task da eseguire come argomento.

```
#include <thread>
void Funz();           // procedura qualsiasi
main()
{  thread Th1 { Funz }; // Funz() è eseguito in un thread separato
   ....
   Th1.join();          // attende che Th1 termini
}
```

Per passare dati al task mandato in esecuzione:

Da notare come per ricevere un risultato in risposta da un task, usiamo un parametro passato **per indirizzo** (i thread sono sempre void, non ritornano valori).

```
void Funz (float *, int, float *);

main()
{  float Vett[100];
   int   N = ...;
   float Somma;
   .....
   thread Th1 { Funz, Vett, N, &Somma };
   ....
   Th1.join();
}
```

nb: se il compilatore dà problemi con la libreria <thread> (tipo dice che non è definito), provare installare la versione di MinGW-64 più recente.

Mutex

Se mandassimo in esecuzione un programma in cui chiamiamo due thread uno dopo l'altro, senza alcuna forma di sincronizzazione tra i thread, l'ordine di esecuzione delle istruzioni nei due task è indefinita. Proprio per questo motivo usiamo i **mutex**.

Un **mutex** è un oggetto usato per rappresentare il diritto di accedere in maniera esclusiva ad una risorsa.

Per esercitare questo diritto il thread acquisisce il mutex **lock()**, accede alla risorsa ed infine rilascia il mutex **unlock()**.

```
#include<mutex>
mutex mux;
void Funz1 ( ) {mux.lock(); cout << "Primo thread "; mux.unlock();}
void Funz2 ( ) {mux.lock(); cout << "Secondo thread "; mux.unlock();}

main()
{   thread Th1 { Funz1 };
    thread Th2 { Funz2 };
    ....
    Th1.join();
    Th2.join();
}
```

Possiamo effettivamente pensare al Mutex come un *semaforo*, con **lock()** il semaforo è rosso, con **unlock()** il semaforo è verde.

In questo caso, il primo thread che passa ha il semaforo verde, come impegna l'incrocio diventa rosso, e una volta passato diventa verde di nuovo.

Se non usassimo i mutex, i due thread potrebbero cercare di stampare contemporaneamente, risultando in un messaggio inconsistente.

(potremmo avere un mex tipo: "Primo thSecondo threadread " invece di "Primo thread Secondo thread" o "Secondo thread primo thread".

LEZ 6 – Bubble Sort, Insertion Sort & Divide et Impera

BUBBLE SORT

Il Bubble Sort è uno degli algoritmi di ordinamento più facili da implementare.

L'idea è di confrontare gli elementi una coppia per volta, e se questi non sono nell'ordine corretto invertirli. Dovranno essere fatti confronti fino a raggiungere la fine dell'array.

Ogni qualvolta si ripete il procedimento, si considera un sottoarray di size-1 rispetto a quello precedente.

Se si ripete il procedimento per il numero di elementi n (meno 1) l'array sarà ordinato.

L'ordinamento è detto "a bolla" perché ricorda il movimento delle bollicine in un bicchiere di champagne.

Il Bubble Sort effettua all'incirca $\frac{n^2}{2}$ confronti e $\frac{n^2}{2}$ scambi, sia in media che nel caso peggiore. Pertanto la sua **complessità computazionale è sempre $O(n^2)$** .

```
void bubblesort(int A[], int n){
    for(int i=0; i<n-1; i++)
        for(int j=0; j<n-1-i; j++) //last i elements are already sorted
            if(A[j]>A[j+1])
                swap(A[j],A[j+1]);
}
```

Esempio:

2	3	4	5	1	unsorted
2	3	4	5	1	2 < 3, ok
2	3	4	5	1	3 < 4, ok
2	3	4	5	1	4 < 5, ok
2	3	4	5	1	5 > 1, swap
2	3	4	1	5	2 < 3, ok
2	3	4	1	5	3 < 4, ok
2	3	4	1	5	4 > 1, swap
2	3	1	4	5	2 < 3, ok
2	3	1	4	5	3 > 1, swap
2	1	3	4	5	2 > 1, swap
1	2	3	4	5	sorted

INSERTION SORT

L'**Insertion Sort** è un algoritmo di ordinamento molto semplice, e che risulta anche essere efficiente quando si deve ordinare un piccolo numeri di elementi.

L'algoritmo riproduce il modo con cui normalmente un essere umano ordina delle carte in una mano. Si inizia con una mano vuota e si prende una carta alla volta dal tavolo, inserendola nella *giusta posizione*. Per trovare la giusta posizione, si confronta la carta con ogni carta presente nella mano (es. leggendo da destra verso sinistra) e quando troviamo una carta il cui valore è minore della carta pescata, inseriamo la carta pescata subito dopo quella carta e facciamo slittare in avanti tutte le altre.

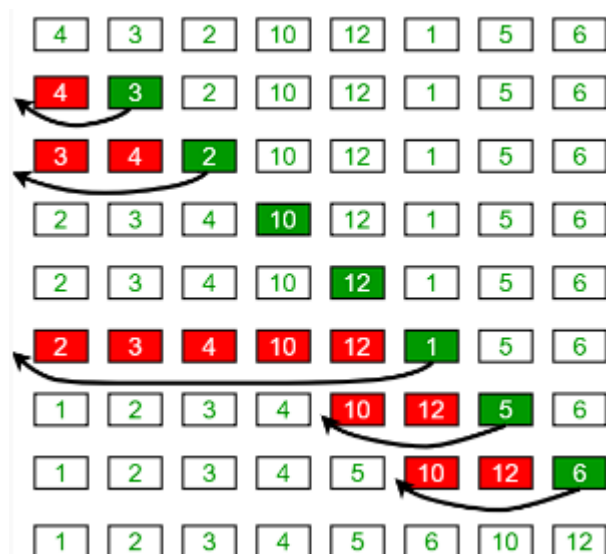
Come abbiamo già notato, la caratteristica di questo algoritmo è che l'insieme delle carte che avremo in mano in quel momento (sottoarray) sarà già ordinato, ed inoltre si tratta di un ordinamento *in loco* in quanto non vengono utilizzati array di appoggio.

(Implementazione dello pseudo-codice visto nella parte di Camastra.

nb: ho invertito gli indici *i* e *j*, e ho trasformato il while in un for).

```
void insertionsort(int A[], int n){
    int i,j,key;
    for(i=1; i<n; i++){
        key = A[i];
        for(j=i-1; j>=0 && A[j]>key; j--){
            A[j+1] = A[j];
        }
        A[j+1] = key;
    }
}
```

Esempio:



DIVIDE ET IMPERA

Il **Divide-et-Impera** è un approccio *top-down* che consiste nel suddividere un problema in sottoproblemi simili al problema originale, ma di dimensioni più piccole, che poi vengono risolti separatamente. Infine, le soluzioni dei sottoproblemi vengono combinate per ottenere la soluzione del problema originale.

Dunque, il **divide et impera** si compone di 3 passi:

- **Divide**: il problema viene suddiviso in vari sottoproblemi, simili al problema originale ma di dimensioni più piccole;
- **Impera**: i sottoproblemi vengono risolti in modo ricorsivo;
- **Combina**: Combinano le soluzioni dei sottoproblemi per costruire una soluzione del problema originale.

Questo approccio è utilizzato soprattutto da algoritmi con struttura ricorsiva (es. *MergeSort* e *QuickSort*). Quando i problemi sono abbastanza grandi da essere risolti ricorsivamente si ha il **caso ricorsivo**, mentre se sono sufficientemente piccoli da non richiedere ulteriori ricorsioni si è raggiunto il **caso base**.

Il principale vantaggio del Divide-et-Impera è che consente di avere tempi di esecuzione nel caso peggiore assai inferiori rispetto a quelli, ad esempio, degli *approcci incrementali* (es. Insertion Sort).

Problema della ricerca binaria

Dato un array **ordinato** di interi $A[1 \dots N]$ ed un numero X , determinare se $X \in A$.

Essendo che l'array è ordinato, possiamo partire dall'elemento centrale. Possiamo trovarci davanti a 2 casi base e 2 casi ricorsivi:

- 1 caso base: l'elemento è X , quindi ritorniamo *true*;
- 1 caso ricorsivo: l'elemento è più piccolo di X , questo significa che X deve trovarsi nella metà a *destra*;
- 2 caso ricorsivo: l'elemento è più grande di X , questo significa che X deve trovarsi nella metà a *sinistra*;
- 2 caso base: arrivati a un sottoarray di size 1, l'elemento è diverso da X . Questo significa che nell'array non è presente X , quindi ritorniamo *false*.

```
bool binarySearch(int A[], int x, int low, int high){
    int mid=(low+high)/2;
    if(A[mid] == x) return true;           //1 base
    else if(high<=low) return false;       //2 base
    else if(A[mid]>x) return binarySearch(A,x,low,mid-1); //1 ric
    else return binarySearch(A,x,mid+1,high); //2 ric
}
```

Problema del massimo sottoarray

Sia dato un vettore A di n elementi, determinare il sottoarray contiguo e non vuoto di A tale che la somma dei suoi elementi sia massima:

-2	-3	4	-1	-2	1	5	-3
0	1	2	3	4	5	6	7

$$4 + (-1) + (-2) + 1 + 5 = 7$$

Tuttavia, il problema non sussiste se A è composto:

- **solo da elementi positivi**: in questo caso il massimo sottoarray è l'intero vettore A.
- **solo da elementi negativi**: il problema del massimo sottoarray diventa il problema di determinare il valore massimo.

Dividiamo l'array in due parti **sx** e **dx**.

Possiamo trovare i massimi sottoarray del sottoarray di sx e di dx in modo ricorsivo, in quanto questi **due sottoproblemi** sono istanze più piccole del problema di trovare un massimo sottoarray.

Tuttavia, è da notar bene che la soluzione potrebbe trovarsi a cavallo tra i sottoarray dx e sx. Per questo motivo dovremo trovare anche un **massimo sottoarray che passa per il punto di mezzo m**.

Il massimo sottoarray che passa per il punto di mezzo (calcolato con maxCrossingSum) può essere:

- il **massimo sottoarray sinistro**, che contiene m;
- il **massimo sottoarray destro**, che contiene m;
- il **sottoarray**, che contiene m, che **contiene sia il massimo sottoarray sinistro che destro**. Questo sarà dato facendo la somma dei due sottoarray di prima, ma sottraendo una volta m (perché altrimenti lo conteremmo 2 volte).

Fra i tre sottoarray, prendere quello con somma maggiore.

```

int maxSubArraySum(int arr[], int l, int h)
{
    //Invalid Range: low is greater than high
    if (l > h)
        return INT_MIN;
    // Base Case: Only one element
    if (l == h)
        return arr[l];

    // Find middle point
    int m = (l + h) / 2;

    /* Return maximum of following three possible cases
       a) Maximum subarray sum in left half
       b) Maximum subarray sum in right half
       c) Maximum subarray sum such that the subarray
          crosses the midpoint */
    return max(maxSubArraySum(arr, l, m - 1),
               maxSubArraySum(arr, m + 1, h),
               maxCrossingSum(arr, l, m, h));
}

```

```

// Find the maximum possible sum in arr[] such that arr[m]
// is part of it
int maxCrossingSum(int arr[], int l, int m, int h)
{
    // Include elements on left of mid.
    int sum = 0;
    int left_sum = INT_MIN;
    for (int i = m; i >= l; i--) {
        sum = sum + arr[i];
        if (sum > left_sum)
            left_sum = sum;
    }

    // Include elements on right of mid
    sum = 0;
    int right_sum = INT_MIN;
    for (int i = m; i <= h; i++) {
        sum = sum + arr[i];
        if (sum > right_sum)
            right_sum = sum;
    }

    // return max
    return max(left_sum + right_sum - arr[m], left_sum, right_sum);
}

```

LEZ 7 – Merge Sort

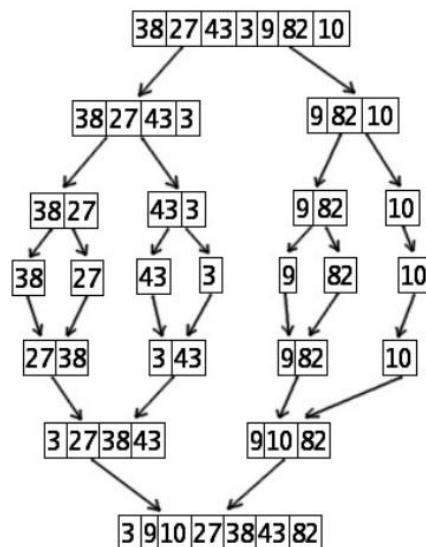
MERGE SORT

(Ripetizione Parte Camastra)

Il Merge-Sort è un algoritmo di ordinamento di complessità **$O(n\log(n))$** , **stabile** (cioè, mantiene l'ordine iniziale (di input) di elementi uguali nell'array ordinato output), e che adotta l'approccio **divide-et-impera**. Inoltre, è un esempio di **comparison sort**.

Possiamo dividere il funzionamento del Merge-Sort in tre passi:

- **divide**: l'array in input viene diviso in due sottoliste di metà lunghezza. Nel caso l'array fosse dispari, la prima sottolista avrà un elemento in più rispetto alla seconda.
- **impera**: viene ordinato, mediante chiamate ricorsive alla funzione stessa, ciascuno dei due sottoarray.
- **combina**: viene eseguito il Merge dei due sottoarray ordinati.



Dunque l'**idea** è chiara: l'algoritmo suddivide l'array originario in porzioni di dimensioni via via più piccole fino ad arrivare a porzioni di dimensioni 1. Al primo passo verrà applicato il *merge* su queste porzioni così da ottenere porzioni **ordinate** di dimensioni 2. Al passo successivo verrà applicato il merge a ciascuna coppia di porzioni di dimensioni 2 così da ottenere porzioni ordinate di dimensioni 4, e così via finché l'intero array non è ordinato. Dunque, il merge verrà applicato su sottosequenze di dimensioni via via crescenti.

L'**algoritmo di merging** può essere realizzato in due modi: *merge astratto sul posto* o *merge a due vie*.

Il Cormen propone un Merge sul posto con *sentinelle*.

ALGORITMO

Il **MergeSort** calcola prima l'indice mediano in modo tale da suddividere la porzione di array considerata in due parti di dimensioni $n/2$, quindi richiama ricorsivamente se stesso prima sulla porzione sinistra, poi sulla porzione destra, infine richiama il MergeSort per combinare i risultati ottenuti.

p rappresenta l'indice di inizio, r rappresenta l'indice di fine, q rappresenta l'indice mediano.

Il MergeSort verrà eseguito solo se l'indice di inizio è minore dell'indice di fine ($p < r$), questo perché se tale condizione non è rispettata ci troviamo di fronte ad un array (o sottoarray) di dimensione 1, che non può essere ulteriormente suddiviso e dunque ci troviamo nel caso *base*.

• Merge sul posto con sentinelle

Il **Merge** calcola prima le dimensioni delle due porzioni di array da fondere: n_1 è la dimensione della porzione di sinistra, mentre n_2 è la dimensione della porzione di destra. Quindi crea i due array L (porzione sinistra) e R (porzione destra), i quali conterranno una determinata porzione degli elementi di A .

Dopodiché, grazie a un ciclo For che va dall'indice di inizio p all'indice di fine r , controlla ad ogni passo il minore tra l' i -esimo elemento di L e il j -esimo elemento di R . Il minore dei due viene inserito nell'array di output A ed, inoltre, viene incrementato l'indice dell'array che ha "fornito" l'elemento ad A .

Da notare come gli array L ed R in realtà siano sovradimensionati di 1 elemento, che conterrà il valore $+\infty$, che corrispondono alle *sentinelle*. Il motivo per cui abbiamo bisogno di queste sentinelle è perché l'ultimo elemento rimasto dei due array non ancora inserito, per com'è fatto l'algoritmo, dovrà necessariamente fare il confronto con qualcosa, ma visto che è l'ultimo elemento non ha nessuno con cui fare confronti (in realtà dovrebbe essere inserito e basta). Per una questione di semplicità gli facciamo fare direttamente il confronto con $+\infty$ (la sentinella), da cui risulterà necessariamente più piccolo e verrà inserito nell'array A .

Es.

Supponiamo di avere l'array $[7|5]$, da cui otteniamo i sottoarray $[7]$ e $[5]$.

Procediamo alla fase di *merge*: entrambi gli array hanno size $n = 1$, ma abbiamo anche detto che dobbiamo inserire uno spazio per ∞ , dunque al momento della creazione il loro vero size sarà $n+1$. I due sottoarray saranno dunque $[7|\infty]$ e $[5|\infty]$.

A questo punto dobbiamo stabilire il loro ordine nell'array A . Usiamo un ciclo for, dove k deve andare da p a r , supponiamo che $p=1$ e $r=2$, dunque effettueremo 2 iterazioni.

Alla prima iterazione avremo che 7 non è < 5 , dunque 5 sarà inserito nell'array A . Alla successiva iterazione avremo 7 dovrà essere confrontato con l'elemento successivo a 5, che è ∞ , ed essendo $7 < \infty$ allora anche 7 verrà inserito nell'array A .

Il ciclo termina e quella porzione dell'array A risulterà ordinata.

```

void mergesort(int A[], int n){
    mergesort(A,0,n-1);
}

void mergesort(int A[], int p, int r){
    int q; //mid
    if(p<r){
        q = (p+r)/2;
        mergesort(A,p,q);
        mergesort(A,q+1,r);
        merge(A,p,q,r);
    }
}

void merge(int A[], int p, int q, int r){
    int n = q-p+1;
    int m = r-q;
    int* L = new int[n+1]; //sovradimensionato di 1
    int* R = new int[m+1]; //sovradimensionato di 1

    int i, j;
    for(i=0; i<n; i++)
        L[i] = A[p+i];
    for(j=0; j<m; j++)
        R[j] = A[q+j+1];
    L[n] = +INF;
    R[m] = +INF;

    int k; i=0; j=0;
    for(k=p; k<r+1; k++)
        if(L[i] <= R[j])
            A[k] = L[i++];
        else
            A[k] = R[j++];

    delete[] L; delete[] R;
}

```

nb: ci sono delle differenze rispetto allo pseudocodice, in quanto qui consideriamo array 0-based e non 1-based.

• Merge a due vie

Dati due insiemi ordinati in input, possiamo **combinarli** in un unico insieme ordinato semplicemente tenendo traccia del **più piccolo** elemento di ogni insieme ed entrando in un ciclo in cui il minore dei due elementi è **spostato in un insieme di output**, continuando fino a che entrambi non siano esauriti.

Il **tempo di esecuzione** è **lineare** nel numero di elementi del file di output, purché siamo in grado di trovare il prossimo minimo nel file di input in tempo costante.

```
void mergesortTwoWays(int A[], int n){
    mergeSort(A,0,n-1);
}

void mergeSort(int A[], int left, int right){
    if(left<right){
        int mid = (left+right)/2;
        mergeSort(A,left,mid);
        mergeSort(A,mid+1,right);

        //alloco vettore di appoggio
        int n = right-left+1;    //size output
        int *out = new int[n];

        //richiamo merge
        int n1 = mid-left+1;    //size left
        int n2 = right-mid;    //size right
        merge(out, A+left, n1, A+(mid+1), n2);

        //ricopio vettore di appoggio
        for(int i=0; i<n; i++)
            A[left+i] = out[i];
        delete[] out;
    }
}

void merge(int c[], int a[], int n, int b[], int m){
    for(int i=0, j=0, k=0; k<n+m; k++){
        if(i==n) { c[k] = b[j++]; continue; }
        if(j==m) { c[k] = a[i++]; continue; }
        c[k] = (a[i]<b[j]) ? a[i++] : b[j++];
    }
}
```

nb: L'istruzione **continue** può essere utilizzata solo all'interno di un ciclo e, a differenza di **break**, riporta il controllo all'inizio del ciclo.

Il mergeSort è simile alla sua variante precedente, solo che in questo caso ci prendiamo qui il disturbo dell'allocazione dell'array di appoggio.

Il merge d'altra parte è molto più compatto.

L'array c rappresenta ovviamente l'array di output, e le sue dimensioni sono date da $N+M$ (ovvero le dimensioni dei due sottoarray). Per il funzionamento del mergesort, questi due sottoarray sono sempre ordinati (si parte da sottoarray di size 1, che sono ordinati per definizione, e via via che proseguiamo col mergesort questi sottoarray continueranno a mantenere la proprietà dell'ordinamento (vedere Invariante di ciclo: Camastra)).

i è l'indice con cui scorriamo sugli elementi di $a[]$, mentre j è l'indice con cui scorriamo sugli elementi di $b[]$. Ad ogni passo inseriamo nell'array di output il più piccolo fra $a[i]$ e $b[j]$, e aumentiamo i o j in base a se abbiamo inserito un elemento di $a[]$ o di $b[]$.

Se $i == N$, significa che l'array $a[]$ è finito, dunque dovremo inserire i rimanenti elementi di $b[]$. Viceversa, se $j == M$.

LEZ 8 – Quick Sort (e varianti)

QUICK SORT

Il **QuickSort** è l'algoritmo di ordinamento con generalmente le prestazioni migliori fra quelli basati sul confronto. Si tratta di un algoritmo in place, facilmente implementabile, ed è general purpose, ovvero che ha un buon comportamento in un'ampia varietà di situazioni.

Nonostante il tempo di esecuzione nel caso peggiore sia $\Theta(n^2)$ con un array di input di n numeri, il quicksort è comunque spesso la soluzione pratica migliore per effettuare un ordinamento perché mediamente è molto efficiente, con un tempo di esecuzione atteso di $\Theta(n \log n)$.

Lo svantaggio sta nel fatto che non è stabile, e appunto che nel caso peggiore ha un comportamento quadratico.

(Rip. Camastra)

Il **Quick Sort** è un algoritmo di ordinamento che esiste in varie versioni: l'idea dell'algoritmo si basa sulla scelta di un elemento detto pivot, o anche elemento partizionatore.

Dopo aver scelto il pivot, si scorre il vettore disordinato in modo da spostare da una parte tutte le chiavi minori del partizionatore e dall'altra parte tutte quelle che sono maggiori. Una volta fatta questa separazione, l'elemento partizionatore deve stare in mezzo.

Il partizionatore si chiama così perché poi partiziona il vettore in 2 sottovettori disordinati caratterizzati dal fatto che la porzione di sinistra contiene le chiavi minori e la porzione di destra le chiavi maggiori. Il Divide et Impera sta nel fatto che il problema iniziale è stato scisso in 2 sottoproblemi di ordinamento

(Se non si presentano due sottoporzioni bilanciate vuol dire che non è stato scelto nel modo giusto il partizionatore).

Essendo un algoritmo di tipo **Divide et impera**, possiamo dividere il funzionamento del QuickSort in 3 parti. Consideriamo un array $A[p \dots r]$, ove p e r rappresentano il primo e l'ultimo elemento, ed indichiamo con q l'elemento partizionatore:

1. **Divide**: partizionare l'array $A[p..r]$ in due sottoarray $A[p..q-1]$ e $A[q+1..r]$ (eventualmente vuoti) tali che ogni elemento di $A[p..q-1]$ sia minore o uguale ad $A[q]$ che, a sua volta, è minore o uguale a ogni elemento di $A[q+1..r]$. Calcolare l'indice q come parte di questa procedura di partizionamento.
2. **Impera**: ordinare i due sottoarray $A[p..q-1]$ e $A[q+1..r]$ chiamando ricorsivamente quicksort.
3. **Combina**: poiché i sottoarray sono ordinati sul posto, non occorre alcun lavoro per combinarli: l'intero array $A[p..r]$ è ordinato.

Randomized Quicksort

Il **Randomized Quicksort** è fondamentalmente identico al Quicksort, con la differenza che il pivot non sarà più un elemento fisso (nb: ultimo elemento, $A[r]$) ma un elemento casuale ($A[p..r]$).

È preferibile utilizzare il randomized q. rispetto alla versione originale in quanto tale elemento di casualità rende più probabile avere prestazioni migliori nel caso medio. Infatti, essendo il pivot scelto a caso, prevediamo che in media la ripartizione dell'array di input potrà essere ben bilanciato.

Per quanto riguarda la complessità del randomized q., è analoga a quella del quicksort.

Miglioramenti per il Quicksort

Un miglioramento consiste nell'evitare che il Quicksort operi su un gran numero di array parziali di piccole dimensione.

Pertanto, invece di “terminare” il quicksort quando raggiungiamo il caso base ($\text{if } (r \leq p) \text{ return};$), ovvero quando la dimensione dell'array è 1, potremmo dire che se la dimensione dell'array ha raggiunto un certo valore M , andremo a chiamare l'**insertionsort**, che sappiamo comportarsi molto bene per array di piccole dimensioni, con una complessità di tempo lineare nel caso migliore.

Utilizzando per M un valore compreso tra 5 e 25, si riesce ad ottenere circa un risparmio del 10% in esecuzione.

Un ulteriore miglioramento consiste nel **partizionamento col metodo della mediana fra tre elementi**.

Esso consiste nell'individuare il primo, l'ultimo e l'elemento centrale dell'array, e scegliere il mediano da questi tre elementi come pivot.

Per ottenere l'effetto “completo” con il metodo della mediana, dobbiamo inoltre *ordinare* questi tre elementi. Ciò non andrà ad influenzare la scelta corrente del pivot, ma potrà influenzare la scelta del pivot all'iterazione successiva, diminuendo così la probabilità che possa presentarsi il caso peggiore.

Tale metodo riduce il tempo medio di esecuzione di circa il 5%(in media).

Se combiniamo i due miglioramenti, ossia utilizziamo il quicksort per trattare gli array di piccole dimensioni, ed applichiamo il metodo della mediana, possiamo ottenere un miglioramento complessivo che si aggira tra il 25-30%.

Un quicksort così composto prende il nome di **Hybrid Sort**.

ALGORITMO

Originale

```
void quicksort(int A[], int n);
void quicksort(int A[], int p, int r);
int partition(int A[], int p, int r);

void quicksort(int A[], int n){
    quicksort(A,0,n-1);
}

void quicksort(int A[], int p, int r){
    if(p<r){
        int q = partition(A,p,r);
        quicksort(A,p,q-1);
        quicksort(A,q+1,r);
    }
}

int partition(int A[], int p, int r){
    int q = A[r];           //Il pivot è l'ultimo elemento dell'array
    int i = p-1;           //Usiamo i e j per spostarci nell'array
    for(int j=p; j<r; j++){
        if(A[j] <= q){      //Se A[j] è minore del pivot
            i++;
            swap(A[i],A[j]);
        }
    }
    swap(A[i+1],A[r]);      //Il pivot viene inserito fra le due partizioni
    return i+1;            //posizione aggiornata del pivot
}

//Prima dell'ultimo scambio, i valori da 'p' a 'i' sono tutti <= di q.
//mentre i valori da 'i+1' a 'j-1' sono tutti > di q.
```

Randomized

Aggiungiamo degli step preliminari al Partition. Scambiamo un elemento a caso con l'ultimo elemento (infatti il pivot sarà sempre l'ultimo elemento).

```
int randPartition(int A[], int p, int r){
    srand(time(NULL));

    int range = r-p+1;
    int i = rand()%range+p;
    swap(A[r],A[i]);
    return partition(A,p,r);
}
```

Hybrid Sort

```
void hybridsort(int A[], int p, int r){

    if(r-p < 5){
        insertionsort(&A[p],r-p+1); //r-p+1 è il size di A, che parte da p
    } else {
        int q = partitionHS(A,p,r);
        hybridsort(A,p,q-1);
        hybridsort(A,q+1,r);
    }
}

int partitionHS(int A[], int p, int r){

    int q = MOT(A,p,r);    //pivot selezionato col Metodo della mediana dei 3 elem
    int i = p-1;           //Usiamo i e j per spostarci nell'array
    for(int j=p; j<r; j++){
        if(A[j]<=q){        //Se A[j] è minore del pivot...
            i++;            //Incremento i
            swap(A[i],A[j]); //Scambio A[i] con A[j]
        }
    }
    swap(A[i+1],A[r]);      //Il pivot viene inserito fra le due partizioni
    return i+1;            //Posizione aggiornata del pivot
}
```

```

int MOT(int A[], int p, int r){
    //Ordino i 3 elementi, e restituisco quello nel mezzo
    //Trovo innanzitutto l'elemento più grande. Il secondo più grande
    //sarà il mediano. Lo scambieremo con l'elemento alla fine.

    int mid = (p+r)/2;

    int x = max(A[p],A[mid],A[r]);

    if(x==A[p]){                //se A[p] è il 1 più grande
        x = max(A[r],A[mid]);
        if(x == A[mid])         //...e se A[mid] è il 2 più grande
            swap(A[mid],A[r]); //lo scambiamo con A[r].
    }
    else if(x == A[mid]){       //se A[mid] è il 1 più grande
        x = max(A[r],A[p]);
        if(x == A[p])           //...e se A[p] è il 2 più grande
            swap(A[p],A[r]);    //lo scambiamo con A[r].
    }
    else{ //x == A[r]           //se A[r] p il 1 più grande
        x = max(A[p],A[mid]);
        if(x==A[p])             //...e se A[p] è il 2 più grande
            swap(A[p],A[r]);    //lo scambiamo con A[r]
        else                    //altrimenti A[mid] è il 2 più grande
            swap(A[mid],A[r]);  //lo scambiamo con A[r]
    }

    return A[r];
}

```

LEZ 9 – ALBERI & HEAP

ALBERI

L'**albero** è una struttura dati gerarchica, in quanto è possibile accedere alle informazioni solo partendo dal nodo radice, cioè la radice dell'albero: dal nodo *radice* possiamo discendere ai livelli inferiori. I nodi finali del livello più basso sono dette nodi *foglie*.

Nell'albero, i nodi si raggruppano in livelli: nel livello 1 troviamo solo la radice. Il nodo che stiamo prendendo in considerazione si chiama nodo corrente e i nodi che si possono raggiungere da esso sono i nodi figli. Il nodo padre del nodo corrente è il nodo grazie al quale siamo potuti arrivare al nodo corrente. Al livello più basso troviamo certamente nodi foglie. Quindi: la radice è l'unico nodo che non ha padre, e le foglie sono gli unici nodi che non hanno figli.

Per ogni nodo di un albero si definisce grado come il numero dei figli: quindi le foglie hanno tutte grado 0 perché non hanno figli.

Si dice sottoalbero un albero che ha come radice il nodo corrente che vogliamo considerare.

Gli algoritmi di visita di un albero sono descritti in maniera *ricorsiva*. Ricordiamo che in un albero non è possibile passare direttamente da un nodo al fratello o risalire da un nodo al nodo padre. Per poter attraversare i nodi dello stesso livello dobbiamo conservare il loro padre, così che attraverso di lui potremo riscendere da un suo figlio ad un altro figlio.

Negli *alberi binari* noi abbiamo il nodo radice e due sottoalberi, dunque ogni nodo può avere al massimo 2 figli, rispetto agli alberi "generici" dove per ogni nodo potevamo avere un numero imprecisato di figli.

Questi due sottoalberi prendono il nome di: sottoalbero sinistro e sottoalbero destro.

Ci sono *tre tipi* principali di **alberi binari**:

- **ALBERO BINARIO COMPLETO**: Si tratta di un albero nel quale tutti i livelli sono completamente riempiti, ad eccezione possibilmente dell'ultimo livello, che dovrà avere tutti i nodi il più a sinistra possibile.

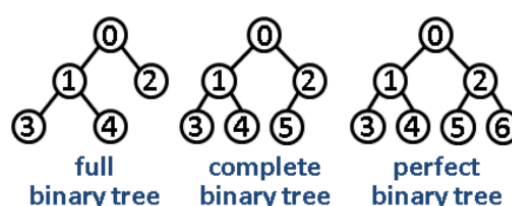
Tutte le foglie avranno profondità h o $h-1$.

Il numero dei nodi al livello $(h-1)$ deve essere 2^{h-1} .

Tutti i nodi interno hanno grado 2.

- **ALBERO BINARIO PIENO**: Si tratta di un albero dove ogni nodo ha 0 o 2 figli.

- **ALBERO BINARIO PERFETTO**: Si tratta di un albero nel quale tutti i nodi interni hanno due figli e tutte le foglie sono allo stesso livello.



ALBERI BINARI HEAP

Un **heap** (binario) è una struttura dati composta da un **array** che possiamo considerare come un **albero binario completo**. Ogni nodo dell'albero corrisponde a un elemento dell'array che memorizza il valore del nodo.

Un array A che rappresenta un heap è un *oggetto* con due attributi:

1. $lunghezza[A]$: indica il numero di elementi dell'array;
2. $heap-size[A]$: indica il numero di elementi dell'heap che sono registrati nell'array A .

nb: ciò significa che, anche se $A[1 \dots lunghezza[A]]$ contiene numeri validi, nessun elemento dopo $A[heap-size[A]]$, dove $heap-size[A] \leq lunghezza[A]$, è un elemento dell'heap.

La radice dell'albero è $A[1]$. Se i è l'indice di un nodo, gli indici di suo padre $PARENT(i)$, del figlio sinistro $LEFT(i)$ e del figlio destro $RIGHT(i)$ possono essere calcolati come:

(nb: tra parentesi le funzioni per array zero-based)

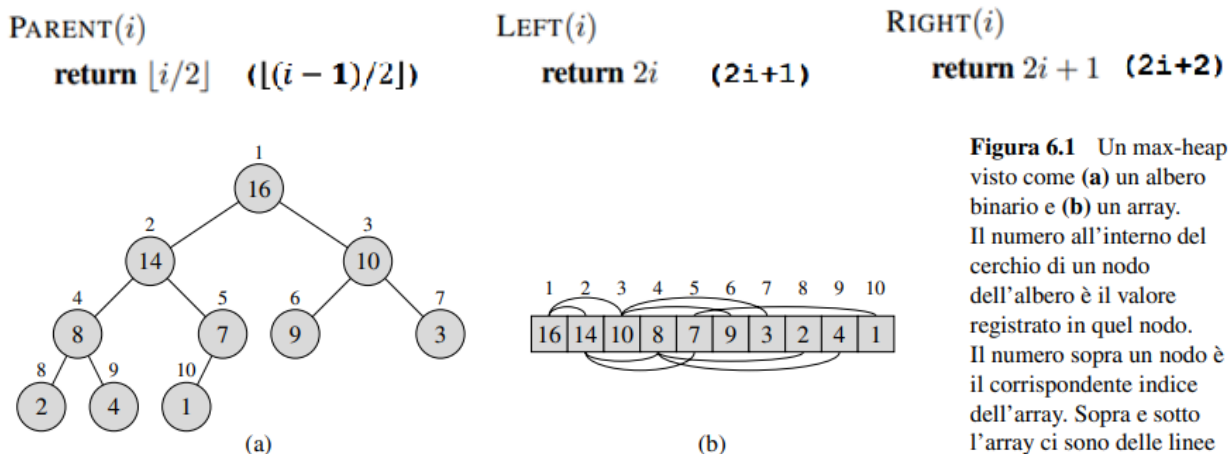


Figura 6.1 Un max-heap visto come (a) un albero binario e (b) un array. Il numero all'interno del cerchio di un nodo dell'albero è il valore registrato in quel nodo. Il numero sopra un nodo è il corrispondente indice dell'array. Sopra e sotto l'array ci sono delle linee che rappresentano le relazioni padre-figlio; i padri sono sempre a sinistra dei loro figli. L'albero ha altezza 3; il nodo con indice 4 e valore 8 ha altezza 1.

Ci sono due tipi di heap binari: max-heap e min-heap. In entrambi i tipi, i valori dei nodi soddisfano una **proprietà dell'heap**:

1. La proprietà del **max-heap** dice che il suo valore più grande è memorizzato alla radice, e il sottoalbero di un nodo contiene valori non maggiori di quello contenuto nel nodo stesso. $(A[PARENT(i)] \geq A[i])$
2. La proprietà del **min-heap** (al contrario) dice che il suo valore più piccolo è memorizzato alla radice, e il sottoalbero di un nodo contiene valori maggiori di quello contenuto nel nodo stesso. $(A[PARENT(i)] \leq A[i])$

Definiamo **altezza di un heap** l'altezza della sua radice, calcolata come $\log n$.

Le **procedure per gestire un heap** sono tre:

1. La procedura **Max-Heapify**, di complessità $O(\log n)$, che è un algoritmo che serve per mantenere la proprietà dell'Heap.
2. La procedura **Build-Max-Heap**, di complessità $O(n)$, che è un algoritmo che costruisce un Heap da zero (partendo da un array).
3. La procedura **Heapsort**, di complessità $O(n \log n)$, che è un algoritmo che ordina sul posto un array.

Max-Heapify

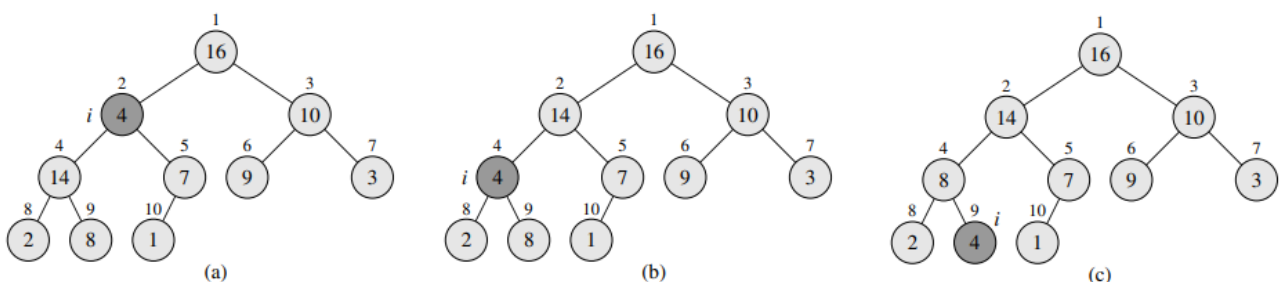
MAX-HEAPIFY è un'importante subroutine per manipolare i max-heap. I suoi input sono un array A e un indice i dell'array. Quando viene chiamata questa subroutine, si suppone che gli alberi binari con radici in $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ siano max-heap, ma che $A[i]$ possa essere più piccolo dei suoi figli, violando così la proprietà del max-heap. La funzione di MAX-HEAPIFY è quella di consentire al valore $A[i]$ di "scendere" nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

In particolar modo, ad ogni passaggio, viene determinato il più grande degli elementi fra $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$, e se ne salva l'indice in una variabile chiamata *max*.

- Se $A[i]$ è più grande, allora il sottoalbero con radice nel nodo i è un max-heap, e quindi la procedura termina.
- Altrimenti, uno dei due figli ha l'elemento più grande e $A[i]$ viene scambiato con $A[\text{max}]$.

Tuttavia, dopo questo scambio, il sottoalbero con indice *massimo* ha il valore originale $A[i]$, e ciò significa che potrebbe comunque esserci il rischio che il sottoalbero con radice in *massimo* possa violare la proprietà del max-heap. Per questo motivo, richiameremo nuovamente MAX-HEAPIFY per questo sottoalbero.

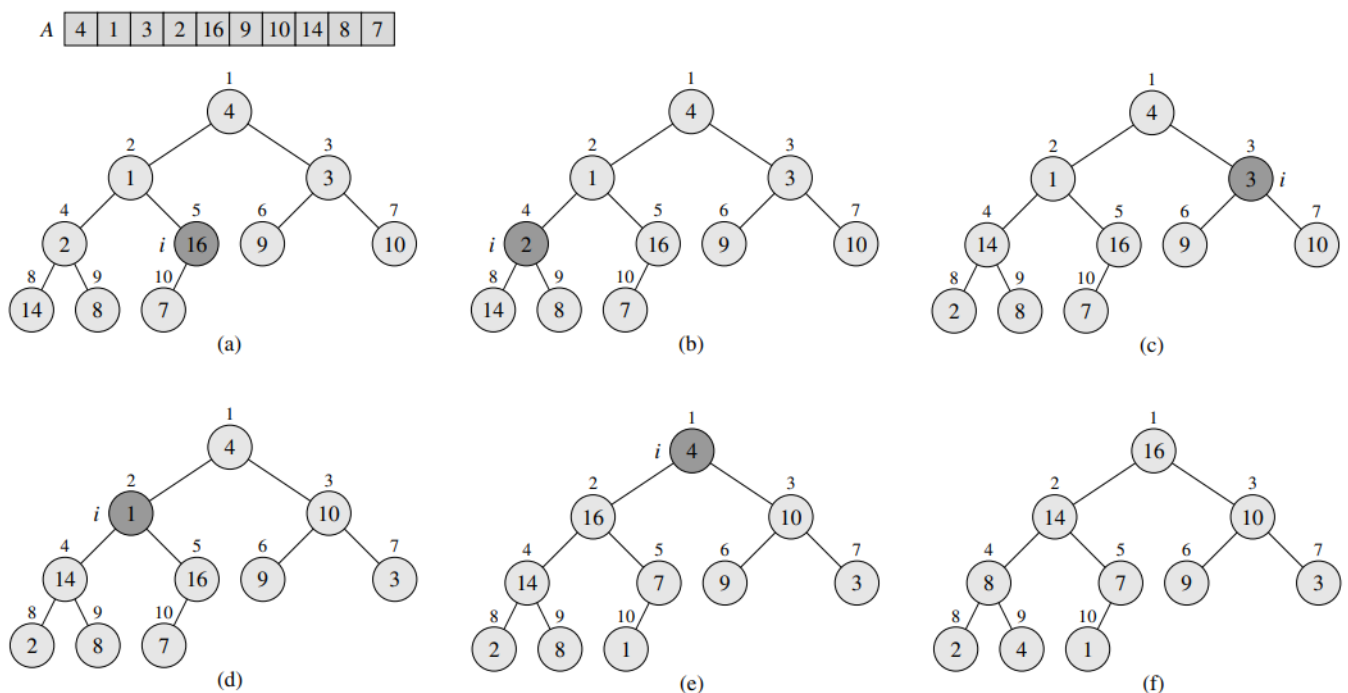
Inoltre, quando confrontiamo il valore massimo, dovremmo anche controllare che l'indice del figlio sinistro e destro di un determinato nodo non sia maggiore rispetto alla dimensione dell'heap. Infatti, se così fosse, vorrebbe dire che tale nodo figlio non esiste, e lo ignoriamo.



Build-Max-Heap

Per costruire un heap possiamo utilizzare la procedura MAX-HEAPIFY dal basso verso l'alto (bottom-up) per convertire un'array $A[1 \dots n]$, con $n = \text{lunghezza}[A]$, in un max-heap. Da notar bene che tutti gli elementi nel sottoarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ sono foglie dell'albero (le foglie in effetti compongono la metà dei nodi dell'albero) e quindi ciascuno è un heap di un elemento da cui iniziare.

La procedura BUILD-MAX-HEAP attraversa i restanti nodi dell'albero ed esegue la procedura MAX-HEAPIFY in ciascuno di essi.



nb: Nota infatti come l'array abbia size 10, e infatti il primo elemento elemento che prendiamo in considerazione è al posto $i=5$. Da qui consideriamo $2i$ (ovvero 7) e $2i+1$ (che non esiste (infatti è $>$ del size di A), e non inseriamo alcun nodo).

Al passo successivo, consideriamo $i=4$, poi $i=3 \dots$ e così via.

ALGORITMI

```
int parent(int i){ return (i-1)/2; }
int left(int i){ return 2*i+1; }
int right(int i){ return 2*i+2; }

void maxHeapify(int A[], int n, int i){
    int l = left(i);
    int r = right(i);
    int max;

    if(l<n && A[l]>A[i])
        max = l;
    else
        max = i;

    if(r<n && A[r]>A[max]){
        max = r;
    }

    if(max != i){
        swap(A[i],A[max]);
        maxHeapify(A,n,max);
    }
}

void buildMaxHeap(int A[], int n){
    int heapSize = n; //non è necessario
    for(int i=(n/2)-1; i>=0; i--){
        maxHeapify(A,n,i);
    }
}
```

LEZ 10 – HEAPSORT & CODE DI PRIORITÀ

HEAPSORT

L'Heapsort è un eccellente algoritmo di ordinamento. Inizia utilizzando BUILD_MAX_HEAP per costruire un max-heap dell'array.

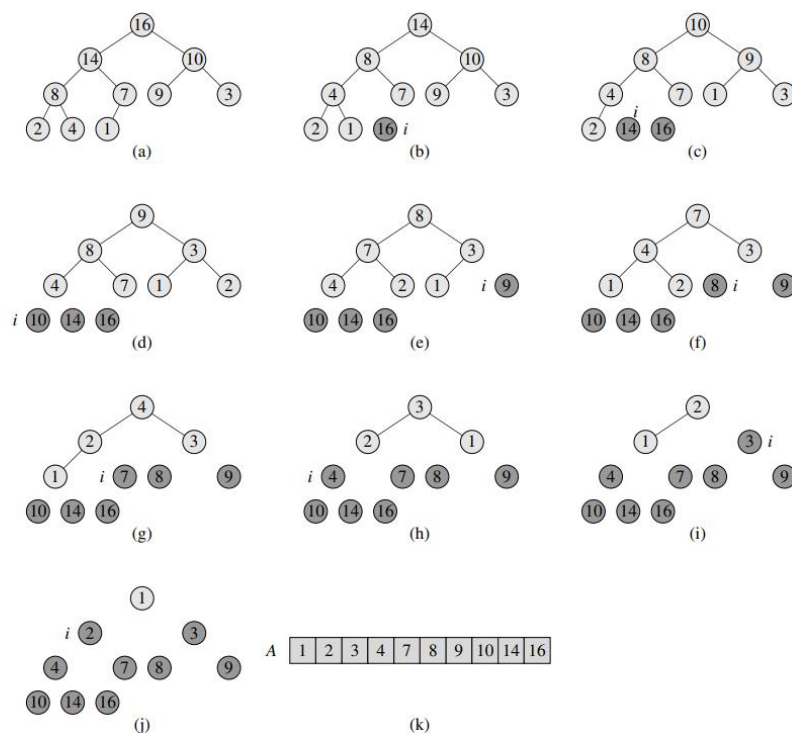
Poiché l'elemento più grande dell'array è memorizzato nella radice $A[0]$, esso può essere inserito nella posizione finale corretta scambiandolo con $A[n-1]$. Se adesso “scartiamo” il nodo $n-1$ dall'heap (diminuendone il size) notiamo che $A[0...n-2]$ può essere facilmente trasformato in un max-heap.

I figli della radice restano max-heap, ma la nuova radice potrebbe violare la proprietà del max-heap, che può tuttavia essere facilmente ripristinata richiamando MAXHEAPIFY($A, 0$).

L'algoritmo heapsort poi ripete questo processo per il max-heap di dimensione $n - 1$ fino a un heap di dimensione 2 (finché $i \geq 1$).

Tale procedura impiega un tempo $O(n \log n)$, in quanto impiega le procedure buildMaxHeap e maxHeapify (che impiegano rispettivamente un tempo $O(n)$ e $O(\log n)$).

```
void heapsort(int A[], int n){
    buildMaxHeap(A,n);
    for(int i=n-1; i>=1; i--){
        swap(A[0],A[i]);
        n--;
        maxHeapify(A,n,0);
    }
}
```



CODE DI PRIORITÀ

Una **coda di priorità** è una struttura dati *astratta* che serve a mantenere un insieme S di elementi, ciascuno con un valore associato detto **chiave**.

Analogamente all'heap, distinguiamo code di *max-priorità* e *min-priorità*.

Una **coda di max-priorità** supporta diverse operazioni:

1. **Insert(S, x)**: inserisce l'elemento x nella coda S .
2. **Maximum(S)**: restituisce l'elemento con chiave massima.
3. **Extract-Max(S)**: rimuove e restituisce l'elemento con chiave massima.
4. **Increase-priority(S, x, k)**: aumenta il valore della chiave di x al nuovo valore k (che si suppone essere grande almeno quanto il valore attuale della chiave di x).

Inoltre, possiamo distinguere un'ulteriore procedura **buildHeap(Max)** che ci permette di costruire un heap sfruttando **Insert(S, x)**, dunque creeremo l'heap inserendo un elemento per volta.

Analogamente, una *coda di min-priorità* supporterà operazioni come: *insert*, *minimum*, *extract_minimum*, *decrease_key*, *buildHeap(Min)*.

Per l'implementazione di questi metodi, possiamo pensare allo sviluppo di una classe **PriorityQueue**.

```
int parent(int i){ return (i-1)/2; }
int left(int i){ return 2*i+1; }
int right(int i){ return 2*i+2; }

class priorityQueue{
    int *A;
    int n;

    void maxHeapify(int i);
    void buildMaxHeap();

public:
    priorityQueue(int *A, int heapSize);
    priorityQueue();

    void insert(int key);
    int max();
    int extractMax();
    void increasePriority(int i, int key);

    //utilities
    void changeHeapSize(int newsize);
    void print();
};
```

nb: ci sono alcune differenze in quanto stiamo considerando adesso array 0-based. Inoltre, non abbiamo più bisogno di passare l'array e il suo size ad ogni funzione in quanto questi attributi fanno già parte della classe stessa.

```
//Implementation
typedef priorityQueue pq;

pq::priorityQueue(int *arr, int heapSize){
    this->n = heapSize;
    this->A = new int[n];
    for(int i=0; i<n; i++)
        A[i] = arr[i];
    buildMaxHeap();
}

pq::priorityQueue(){
    A = nullptr;
    n = 0;
}

void pq::maxHeapify(int i){
    int l = left(i);
    int r = right(i);
    int max;

    max = (l<n && A[l]>A[i]) ? l : i;
    max = (r<n && A[r]>A[max]) ? r : max;

    if(max != i){
        swap(A[i],A[max]);
        maxHeapify(max);
    }
}

void pq::buildMaxHeap(){
    for(int i=(n/2)-1; i>=0; i--)
        maxHeapify(i);
}

void pq::insert(int key){
    changeHeapSize(n+1);
    A[n-1] = -INF;
    increasePriority(n-1,key);
}

int pq::max(){
    return A[0];
}
```

```

int pq::extractMax(){
    if(n<1){
        std::cout<<"heap underflow";
        exit(1);
    }

    int max = A[0];
    A[0] = A[n-1];
    changeHeapSize(n-1);
    maxHeapify(0);
    return max;
}

void pq::increasePriority(int i, int key){
    if(key<A[i]){
        std::cout<<"key must be greater than current value";
        exit(1);
    }

    A[i] = key;
    while(i>0 && A[parent(i)] < A[i]){
        swap(A[i],A[parent(i)]);
        i = parent(i);
    }
}

//Other

void pq::changeHeapSize(int newsize){
    int *B = new int[newsize];
    for(int i=0; i<newsize; i++){
        if(i<n)
            B[i] = A[i];
        else
            B[i] = 0;
    }
    n = newsize;
    delete[] A;
    A = B;
}

```

nb: changeHeapSize cambia il size dell'array semplicemente inizializzando un array più grande/piccolo B, copia i primi newsize elementi di A in B (eventuali posizioni vuote di B saranno riempite da 0), dealloca i dati a cui puntava A, e fa puntare A al nuovo array B.

nb: le code di priorità possono essere implementate anche con strutture dati diverse dagli heap, sebbene questa risulti la soluzione più comune ed efficiente.
Per questo, la coda di priorità è considerata una struttura dati *astratta*.

LEZ 11 – ALGORITMI DI ORDINAMENTO LINEARI

Gli algoritmi di ordinamento visti fino ad ora effettuano un ordinamento basato solo su confronti fra gli elementi di input, e il quale limite inferiore è $n \log n$.

Gli algoritmi in tempo lineare usano operazioni diverse dai confronti e quindi non vale il precedente limite inferiore.

Esempi di tali algoritmi sono il: *counting sort*, *radix sort* e *bucket sort*.

COUNTING SORT

Il Counting Sort è un **algoritmo di ordinamento per valori numerici interi** con complessità $O(n+m)$, dove n è la lunghezza dell'array e m è pari a $\max(A) - \min(A) + 1$. I valori $\max(A)$ e $\min(A)$ sono rispettivamente l'elemento più grande e l'elemento più piccolo dell'array.

Tale algoritmo **non** è basato su confronti e scambi, e conviene utilizzarlo quando il valore di m è più piccolo rispetto ad n , altrimenti risulterebbero più veloci altri algoritmi. Esso prende in **ingresso** un vettore e **restituisce** un secondo vettore ordinato, ed utilizza un vettore di appoggio per l'elaborazione.

L'algoritmo conta il **numero di occorrenze** di ciascun valore presente nell'array da ordinare, memorizzando questa informazione in un array temporaneo di dimensioni pari all'intervallo di valori.

Si calcolano i valori massimo e minimo dell'array, e si prepara un array ausiliario C di dimensioni pari all'intervallo dei valori con $C[i]$, che rappresenta la frequenza dell'elemento $i + \min(A)$ nell'array di partenza A .

Si visita l'array A aumentando l'elemento di C corrispondente. Infine, si visita l'array C in ordine e si scrivono su A , $C[i]$ copie del valore di $i + \min(A)$.

COMPLESSITÀ

L'algoritmo esegue due **iterazioni**:

- una di lunghezza **n** per il calcolo delle occorrenze dei valori.
- una di lunghezza **m** per l'impostazione delle posizioni finali dei valori.

La complessità totale è quindi $O(n+m)$, e conviene utilizzarlo quando il valore di **m** è **$O(n)$** , nel qual caso l'algoritmo è $O(n)$.

COUNTING SORT - IMPLEMENTAZIONE IN C++

Una volta individuati minimo e massimo, e di conseguenza il size dell'array di supporto, creiamo tale nuovo array inizializzando tutti gli elementi a 0.

Ogni qualvolta che incontriamo un certo valore di posizione $A[i]$ -min di C, andiamo ad incrementare di 1 il valore in quella posizione, questo per tutto il size di A.

Fatto ciò, avremo trovato il valore di tutte le posizioni di C.

Al prossimo passo, utilizzeremo un nuovo contatore k con il quale ci sposteremo lungo l'array A. Finché $C[i]$ sarà maggiore di 0, inseriremo il giusto valore in $A[k]$, che corrisponde a $i+\text{min}$, ed è anche il valore definitivo. Poi incrementeremo k e decremteremo $C[i]$ di 1.

Finito il ciclo, tutto l'array A risulterà ordinato.

```
void countingsort(int A[], int n){  
  
    //trovo min e max  
    int max = A[0]; int min = A[0];  
    for(int i=0; i<n; i++){  
        if(A[i]>max) max = A[i];  
        else if(A[i]<min) min = A[i];  
    }  
  
    //inizializzo a 0 un vettore di appoggio C  
    int sizeC = max-min+1;  
    int *C = new int[sizeC]{}; //inizializzo a 0  
  
    //conto le ricorrenze  
    for(int i=0; i<n; i++){  
        C[A[i]-min]++;  
    }  
  
    //il for scrive C[i] volte il valore i+min in A  
    //il while inserisce il giusto valore in A[k], e decrementa C[i]  
    int k=0;  
    for(int i=0; i<sizeC; i++){  
        while(C[i]>0){  
            A[k] = i+min;  
            C[i]--;  
            k++;  
        }  
    }  
  
    delete[] C;  
}
```

Esempio di esecuzione

```
//inizio

A = {1,0,3,1,3,1}
n = 6;

max = 3; min = 0;

sizeC = max-min+1 = 3-0+1 = 4
C = {0,0,0,0}

//conto...
for(i=0; i<n; i++)
    C[A[i]-min]++;

cioè...
i=0 , C[A[0]-min]++ -> C[1-0]++ -> C[1]++ --|
i=1 , C[A[1]-min]++ -> C[0-0]++ -> C[0]++      |    C[0] = 1
i=2 , C[A[2]-min]++ -> C[3-0]++ -> C[3]++      |--> C[1] = 3
i=3 , C[A[3]-min]++ -> C[1-0]++ -> C[1]++      |    C[2] = 0
i=4 , C[A[4]-min]++ -> C[3-0]++ -> C[3]++      |    C[3] = 2
i=5 , C[A[5]-min]++ -> C[1-0]++ -> C[1]++ --|

//inseriamo il giusto valore di A[k]
for(i=0; i<n; i++)
    while(C[i]>0){
        A[k] = i+min;
        C[i]--;
        k++;
    }

cioè...
i=0, k=0, C[0] = 1 -> A[k], A[0] = 0 | C[0]--          //0
i=0, k=1, C[0] = 0 -> next for it

i=1, k=1, C[1] = 3 -> A[k], A[1] = 1 | C[1]--          //1
i=1, k=2, C[1] = 2 -> A[k], A[2] = 1 | C[1]--          //1
i=1, k=3, C[1] = 1 -> A[k], A[3] = 1 | C[1]--          //1
i=1, k=4, C[1] = 0 -> next for it

i=2, k=4, C[2] = 0 -> next for it

i=3, k=4, C[3] = 2 -> A[k], A[4] = 3 | C[3]--          //3
i=3, k=5, C[3] = 1 -> A[k], A[5] = 3 | C[3]--          //3
i=3, k=6, C[3] = 0 -> next for it

i=4, end for
```

A = {0,1,1,1,3,3}

RADIX SORT

Il Radix Sort è un algoritmo di ordinamento usato per ordinare **record con chiavi multiple**.

Un esempio di record con chiavi multiple è dato dalla *data gg/mm/aaaa*. Infatti, per ordinare per data si deve prima ordinare l'anno, a parità di anno si deve ordinare per mese, e a parità di mese si deve ordinare per giorno..

Il Radix Sort opera in modo controintuitivo, ordinando **prima le cifre meno significative**, e poi via via quelle più significative.

Ad esempio, supponiamo di dover ordinare una sequenza di numeri a 3 cifre. Possiamo considerare le cifre di un intero come chiavi separate.

Utilizzando un **ordinamento di tipo stabile** possiamo procedere ordinando prima le unità, poi le decine e in ultimo le centinaia.

Ad ogni passo, la stabilità ci garantisce che le cifre precedenti siano già ordinate.

COMPLESSITÀ

Considerato che dobbiamo eseguire d ordinamenti, ove d rappresenta il numero di colonne, il **tempo complessivo** è $\Theta(d(n+k))$, dove k è il valore massimo dei valori nelle celle.

Nel caso dei *numeri decimali*, il numero delle colonne è uguale al numero delle cifre, ed ogni cifra può avere come valore massimo 9. Ergo, la complessità sarà $\Theta(d * n)$. Se d è costante, la complessità sarà $\Theta(n)$.

Lo svantaggio sta nel fatto che il metodo non è *in place*, e ha bisogno più del doppio della memoria di tali metodi.

Radix-Sort(A, d)

for $i=1$ to d do

metodo di ordinamento stabile su cifra i

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

NB: Algoritmi di ordinamento stabili sono, ad esempio, il Merge Sort, il Counting Sort, l'Insertion Sort e il Bubble Sort.

Algoritmi di ordinamento instabili sono, ad esempio, il Quicksort, il Selection Sort e l'Heapsort.

In sostanza, il radix sort:

- può essere usato solo per ordinare numeri;
- l'ordinamento avviene dalla cifra meno significativa a quella più significativa;
- viene usato solitamente insieme al counting sort.

Esempio 2:

170	45	75	90	802	24	2	66
170	90	802	2	24	45	75	66
802	2	24	45	66	170	75	90
2	24	45	66	75	90	170	802

1. ordiniamo prima per le unità.
2. l'array è ordinato per le unità. Notare che 170 viene prima di 90 in quanto il counting sort è stabile, e al passo precedente 170 veniva prima di 90.
ordiniamo per le decine.
3. l'array è ordinato per le decine. Ordiniamo per le centinaia.
4. l'array è ordinato.

Notare che eventuali spazi vuoti sono considerati come "0".

```
void radixsort(int A[], int n){
    int m = max(A,n);
    for(int exp=1; m/exp > 0; exp*=10)
        countingsortRadix(A,n,exp);
}
```

Per il counting sort dobbiamo usare una versione che è in grado di distinguere le varie cifre di un numero (unità, decine, centinaia...).

Adattare la versione precedente è risultato essere complicato, dunque utilizzerò questa versione presa online.

nb: per ottenere la i -sima cifra di un numero n , la possiamo calcolare come:

$$n = (n/\text{exp})\%10$$

dove exp è dato da: $\text{exp} = \text{pow}(10, i)$;

```
void countingsortRadix(int array[], int size, int place) {
    const int MAX = 10;
    int output[size];
    int count[MAX];

    for (int i = 0; i < MAX; ++i)
        count[i] = 0;

    // Calculate count of elements
    for (int i = 0; i < size; i++)
        count[(array[i] / place) % 10]++;

    // Calculate cumulative count
    for (int i = 1; i < MAX; i++)
        count[i] += count[i - 1];

    // Place the elements in sorted order
    for (int i = size - 1; i >= 0; i--) {
        output[count[(array[i] / place) % 10] - 1] = array[i];
        count[(array[i] / place) % 10]--;
    }

    for (int i = 0; i < size; i++)
        array[i] = output[i];
}
```

BUCKET SORT

Il Bucket Sort, o *bin sort*, è un algoritmo di ordinamento il cui funzionamento consiste nel suddividere un array in un **numero finito di buckets**.

Ciascun bucket è ordinato individualmente, o utilizzando un differente algoritmo di ordinamento, o applicando ricorsivamente il bucket sort.

Inizialmente si definisce un array di “buckets” inizialmente vuoti, poi si mette ogni elemento dell’array nel suo bucket e viene ordinato ogni bucket non vuoto.

Infine, vengono spostati gli elementi dai buckets all’array originale.

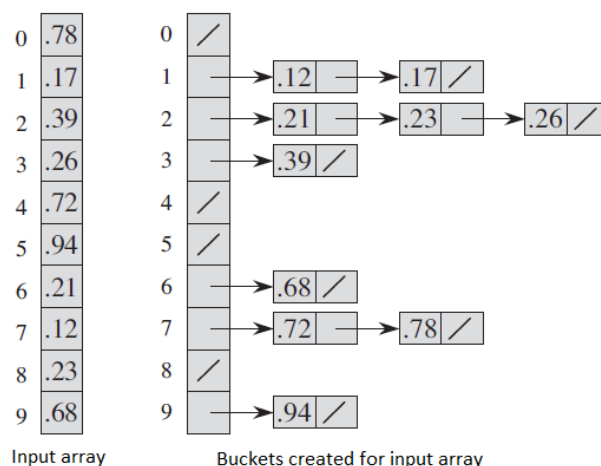
La sua complessità media è $\Theta(n)$.

ESEMPIO:

Consideriamo un array di 10 elementi.

Inseriremo l’elemento $arr[i]$ nel bucket $[n \cdot arr[i]]$. Una volta inseriti tutti gli elementi nei loro rispettivi bucket, ordineremo i singoli bucket, e infine sposteremo tali elementi nell’array originale.

Il numero di bucket è n .



nb: la versione generale (Cormen) presuppone che gli elementi siano compresi fra 0 ed 1, noi invece implementeremo una versione che consente di ordinare l’array qualsiasi sia il range di valori facendo qualche conversione.

Un’accoppiata BucketSort/InsertionSort può risultare eccellente. Infatti sappiamo che l’InsertionSort si comporta molto bene con array di piccole dimensioni, in quanto sarà probabile che questi saranno già parzialmente ordinati, e l’InsertionSort è in grado di “rendersene conto”. Quello che fa il BucketSort è prendere un array e suddividerlo in più gruppi, o “bucket”, quindi creando una serie di array più piccoli (che verranno ordinati dall’InsertionSort).

ALGORITMO

Possiamo implementare un bucketsort che possa ordinare qualsiasi range di valori. Si tratta di aggiungere un passo preliminare: dobbiamo trovare il valore massimo dell'array, aggiungere 1, e poi dividere tutti gli elementi per questo valore $max+1$. Quello che otterremo sarà l'array normalizzato (cioè ogni valore è compreso fra 0 e 1).

Aggiungiamo 1 cossiché il risultato sia $max/max+1$ sia minore 1. Il numero di bucket resterà n .

Quando andremo a verificare in quale bucket debba andare un certo valore, non considereremo $bucket[n*arr[i]]$, ma $bucket[n*normalizedArr[i]]$, nel quale “spingeremo” però $arr[i]$.

Per normalizzare un numero ci basterà fare $A[i]/(max(A[i])+1)$.

Es.

$A = \{ 81, 48, 23, 41, 18, 76, 37, 40 \}$

$maxp1 = 81 + 1 = 82$

$A \text{ normalizzato} = \{ .98, .58, .28, .5, .21, .92, .45, .48 \}$

```
void bucketsort(int A[], int n){
    vector<int> bucket[n];           //crea n bucket vuoti

    //inseriamo gli elementi nei bucket
    int maxp1 = max(A,n)+1;
    for(int i=0; i<n; i++){
        bucket[n*(A[i]/maxp1)].push_back(A[i]);

    //ordiniamo i bucket con l'insertion sort, ma per usarlo
    //ho bisogno di un array, quindi ne uso uno di appoggio
    int* B;
    int m;
    for(int i=0; i<n; i++){
        B = bucket[i].data();
        m = bucket[i].size();
        insertionsort(B,m);
    }

    //concateno i bucket in ordine
    int k=0;
    for(int i=0; i<n; i++){
        for(int j=0; j<bucket[i].size(); j++){
            A[k++] = bucket[i][j];
        }
    }
}
```

LEZ 12 – ALBERI BINARI DI RICERCA (ABR)

Gli **alberi binari di ricerca** (o alberi binari ordinati) sono alberi binari che vengono costruiti e mantenuti in modo tale che tra le informazioni sia soddisfatta una certa proprietà: la proprietà dell'ordinamento. (nb: figlio sx più piccolo del nodo corrente, mentre figlio dx più grande)

Sia x un nodo di un albero binario di ricerca:

- Se y è un nodo del sottoalbero sinistro di x , allora $\rightarrow$ $chiave[x] \geq chiave[y]$;
- Se y è un nodo del sottoalbero destro di x , allora $\rightarrow$ $chiave[x] \leq chiave[y]$;

Tali alberi binari sono strutture dati che possono essere rappresentate come un **concatenamento di nodi**, e sui quali vengono definite varie operazioni, quali: Search, Minimum, Maximum, Predecessor, Successor, Insert e Delete.

Inoltre, gli ABR possono essere utilizzati per implementare sia **dizionari** che **code di priorità**.

Ogni nodo di un ABR contiene: la **chiave**, il **puntatore al PADRE**, il **puntatore al FIGLIO DESTRO**, il **puntatore al FIGLIO SINISTRO** ed eventuali dati satelliti.

Per quanto riguarda la **visita di un albero binario**, sono tre gli algoritmi fondamentali che ci permettono di visitare un albero, che si differenziano principalmente per la visita della radice rispetto alla visita dei due sottoalberi:

- Visita **preorder** (ordine anticipato) che visita in ordine:
 - 1- la radice;
 - 2- il sottoalbero sinistro;
 - 3- il sottoalbero destro.
- Visita **inorder** (ordine simmetrico) che visita in ordine:
 - 1- il sottoalbero sinistro;
 - 2- la radice;
 - 3- il sottoalbero destro.
- Visita **postorder** (ordine differito) che visita in ordine:
 - 1- il sottoalbero sinistro;
 - 2- il sottoalbero destro;
 - 3- la radice.

Tali visite possono essere implementate con un semplice algoritmo ricorsivo, la quale complessità è $\Theta(n)$, in quanto la procedura viene chiamata ricorsivamente esattamente due volte per ogni nodo dell'albero.

La visita presenta una forma del tipo:

```
VISITA_ANTICIPATA(x)
if x != NIL
    visita(x)
    VISITA_ANTICIPATA(Left[x])
    VISITA_ANTICIPATA(Right[x])
```


Operazioni su ABR

TREE-SEARCH

Si usa per ricercare un nodo con una data chiave all'interno di un ABR.

In Input si passa il *puntatore alla radice dell'albero* (o sottoalbero) e la *chiave k*, e restituisce in output il *puntatore al nodo con chiave k* (se esiste).

Il tempo di esecuzione è: $O(h)$, dove h è l'altezza dell'albero.

Metodo ricorsivo

TREE_SEARCH(x,k)

```
1.  if x = NIL or k = x.key
2.      return x
3.  if k < x.key
4.      return TREE_SEARCH(x.left,k)
5.  else
6.      return TREE_SEARCH(x.right,k)
```

Metodo iterativo

TREE_ITERATIVE_SEARCH(x,k)

```
1.  tmp = x
2.  while tmp ≠ NIL and k ≠ tmp.key
3.      if k < tmp.key
4.          tmp = tmp.left
5.      else
6.          tmp = tmp.right
7.  return tmp
```

nb: tmp rappresenta una variabile di appoggio. Infatti vogliamo evitare di cambiare involontariamente il nodo puntato da x.

Nel metodo ricorsivo ritorniamo direttamente il figlio sinistro o destro (dunque non abbiamo bisogno di un "appoggio"), mentre nel metodo iterativo aggiorniamo di volta in volta tmp.

TREE_MINIMUM & TREE_MAXIMUM

Per la ricerca del minimo e del massimo ci basta ricordare la proprietà dell'ARB, ovvero che la chiave del figlio sinistro è minore rispetto a quella del nodo corrente, mentre la chiave del figlio destro è maggiore rispetto a quella del nodo corrente.

Quindi:

- ci basterà scendere sempre a sinistra per ottenere il minimo (se il figlio sinistro del nodo corrente è un nullptr, significa che il nodo corrente è il minimo);
- ci basterà scendere sempre a destra per ottenere il massimo (se il figlio destro del nodo corrente è un nullptr, significa che il nodo corrente è il massimo).

Basterà passare come input il *puntatore al nodo della radice* dell'albero (o sottoalbero) e otterremo in output il puntatore al nodo con chiave minima/massima.

Il tempo di esecuzione è: $O(h)$, dove h è l'altezza dell'albero.

TREE_MIN(x)

```
1. tmp = x
2. while tmp.left ≠ NIL
3.     tmp = tmp.left
4. return tmp
```

TREE_MAX(x)

```
1. tmp = x
2. while tmp.right ≠ NIL
3.     tmp = tmp.right
4. return tmp
```

SUCCESSORE & PREDECESSORE

Il **successore di un nodo v** è il più piccolo nodo maggiore di v .

Possiamo distinguere due casi:

- Se v ha un figlio destro, allora il successore è il minimo del sottoalbero destro di v .
- Se v non ha un figlio destro, allora il successore è il primo antenato di v (si risale l'albero) tale che v sia figlio sinistro del proprio antenato.

Il **predecessore di un nodo v** è il più piccolo nodo maggiore di v .

Possiamo distinguere due casi:

- Se v ha un sinistro, allora il predecessore è il massimo del sottoalbero sinistro di v .
- Se v non ha un figlio sinistro, allora il predecessore è il primo antenato di v (si risale l'albero) tale che v sia figlio destro del proprio antenato.

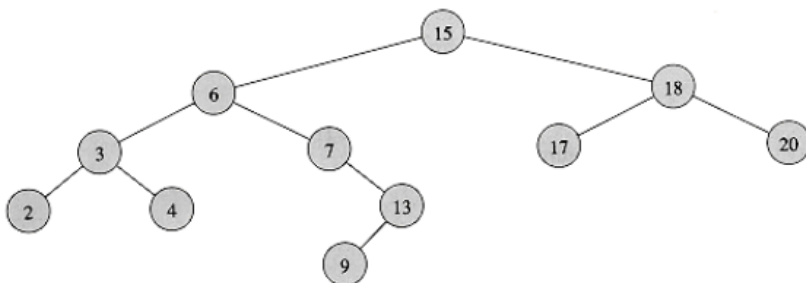
TREE_SUCCESSOR(x)

```
1. if x.right ≠ NIL
2.     return TREE_MIN(x.right)
3. xp = x.p
4. while xp ≠ NIL and x = xp.right
5.     x = xp
6.     xp = xp.p
7. return xp
```

TREE_PREDECESSOR(x)

```
1. if x.left ≠ NIL
2.     return TREE_MAX(x.left)
3. xp = x.p
4. while xp ≠ NIL and x = xp.left
5.     x = xp
6.     xp = xp.p
7. return xp
```

Il tempo di esecuzione, in entrambi i casi, è: $O(h)$, dove h è l'altezza dell'albero.



ESEMPIO:

Successore di 7? di 4? di 6? di 20? → 9; 6; 7; no succ. (perché obv è il max dell'albero)

Predecessore di 6? di 17? di 2? → 4; 15; no succ. (perché obv è il min dell'albero)

INSERT

La procedura di inserimento richiede di passare l'albero nel quale inserire il nodo (T) e il nodo da inserire (z).

Cominciamo dalla radice dell'albero e scendiamo cercando di individuare la giusta posizione del nodo z. Usiamo due nodi d'appoggio: x (nodo corrente) e xp (padre del nodo corrente).

Finché x non è nullptr (ovvero non abbiamo trovato una posizione libera per z) continuiamo a scendere, aggiornando x e il padre di x. Il nodo x diventerà il suo figlio sinistro se la chiave di z è minore della sua chiave attuale, altrimenti diventerà il suo figlio destro.

Trovata una posizione per z, aggiorniamo il padre di z in modo che punti al padre di x (**nb**: essendo x alla fine del ciclo un nodo NIL, allora xp sarà una foglia).

Adesso xp si troverà con un nuovo figlio, dunque dobbiamo anche aggiornare questa informazione:

- se xp è NIL, significa che l'albero era vuoto. Z diventa la nuova radice dell'albero.
- se z ha una chiave più piccola di xp, significa che z è il figlio sinistro di xp;
- altrimenti, significa che z è il figlio destro di xp.

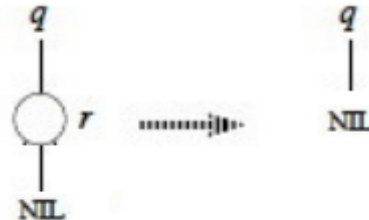
TREE_INSERT(T,z)

```
1.  x = T.root
2.  xp = NIL
3.  while x ≠ NIL
4.      xp = x
5.      if z.key < x.key
6.          x = x.left
7.      else
8.          x = x.right
9.  z.p = xp
10. if z.p = NIL
11.     T.root = z
12. else if z.key < z.p.key
13.     z.p.left = z
14. else
15.     z.p.right = z
```

DELETE (con Transplant)

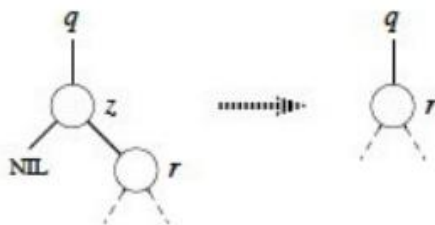
Quando vogliamo eliminare un nodo z da un ABR, possono trovarci di fronte a 3 possibilità:

- **Il nodo z da eliminare è una foglia:** modifichiamo il **padre di z** in modo che il suo puntatore a figlio sia un nullptr.

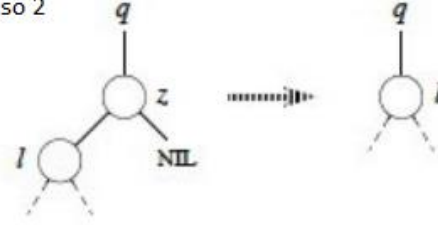


- **Il nodo z da eliminare ha un figlio:** sostituiamo z con suo **figlio**, facendo puntare al **padre di z** il **figlio di z** (distinguiamo 2 casi in base a se il **figlio di z** si tratta del figlio destro o sinistro).

caso 1

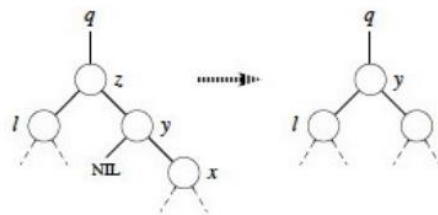


caso 2

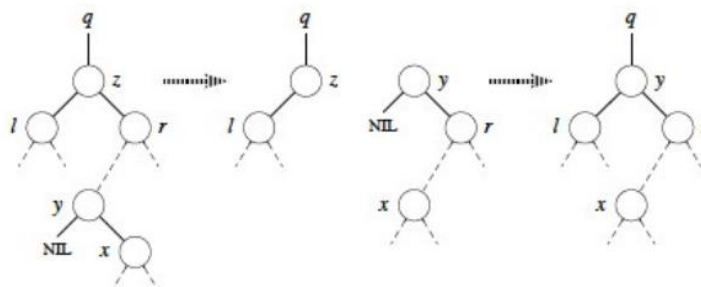


- **Il nodo z da eliminare ha due figli:** troviamo il **successore y** di z (che si troverà nel sottoalbero di destra, e sappiamo che non avrà un figlio sinistro*):
 - o se y è anche il figlio destro di z , sostituiamo z con y .
 - o altrimenti, sostituiamo y con il figlio destro di z , e poi sostituiamo z con y .

y è figlio destro di z



y NON è figlio destro di z



Transplant

La procedura Transplant consente di **sostituire** un sottoalbero con un altro sottoalbero. In sostanza, significa modificare “chi punta a chi”. Se vogliamo sostituire il nodo z con y , significa che il padre di z considera come figlio (punta a) y , e y considera come padre (punta a) il padre di z . In pseudocodice:

TRANSPLANT(T, u, v)

```
1.  p = u.parent
2.  if p = NIL
3.      T.root = v
4.  else if u = p.left
5.      p.left = v
6.  else
7.      p.right = v
8.  if v ≠ NIL
9.      v.parent = p
```

nb: l'istruzione 1 non è necessaria ai fini dell'algoritmo (possiamo anche sostituire p con $u.parent$), è usata solo per una questione di comodità.

Se p è NIL, significa che u è la radice dell'albero. Sostituirla significa banalmente far diventare v la nuova radice.

Se invece il padre esiste, dobbiamo cambiare il suo figlio in modo che adesso non punti più a u , ma punti a v . Se u era il figlio sinistro di p , allora v diventerà il nuovo figlio sinistro di p , viceversa v diventerà il nuovo figlio destro di p .

Infine dobbiamo assegnare il nuovo padre a v , ma questo possiamo farlo solo se v non è un nodo NIL (per questo il controllo preliminare). Infatti possiamo anche sostituire un sottoalbero con “nulla”.

nb: alla fine di questo algoritmo, il nodo u viene “tagliato fuori” dall'albero, nel senso che visitando l'albero non sarà più possibile raggiungerlo (infatti quello che era suo padre adesso punta a v). Tuttavia u mantiene ancora i suoi “puntatori” intatti (infatti punta ancora a quello che era suo padre, e punta ancora ai suoi figli), ed è importante per il funzionamento della delete.

TREE_DELETE(T,z)

```
1.  if z.left = NIL
2.      TRANSPLANT(T,z,z.right)
3.  else if z.right = NIL
4.      TRANSPLANT(T,z,z.left)
5.  else
6.      y = TREE_MIN(z.right)          //y is the successor of z
7.      if y ≠ z.right
8.          TRANSPLANT(T,y,y.right)
9.          y.right = z.right
10.         y.right.parent = y
11.     TRANSPLANT(T,z,y)
12.     y.left = z.left
13.     y.left.parent = y
```

*il motivo per cui sappiamo che il successore y di z non ha un figlio sinistro, è perché y è il più piccolo nodo maggiore di z (ed essendo tutti i nodi nel sottoalbero destro di z maggiori di z , allora questo si trasforma nel problema di **trovare il minimo nel sottoalbero destro**). Come ben ricordiamo, per ottenere il nodo più piccolo di un sottoalbero ci basterà scendere sempre a sinistra (finché il figlio sinistro del nodo corrente non è un nullptr, dunque il nodo corrente è il minimo).

L'obiettivo della delete è quella di “tagliare fuori” dall'albero il nodo z , e tale obiettivo possiamo raggiungerlo attraverso una transplant.

La riga 1 copre sia il caso in cui z sia una foglia, che il caso in cui abbia solo figlio destro. Infatti anche se z non avesse destro, andremmo a sostituirlo con un nodo NIL.

La riga 3 copre la situazione speculare. z ha figlio sinistro ma non ha figlio destro, dunque il figlio sinistro prenderà il posto di z .

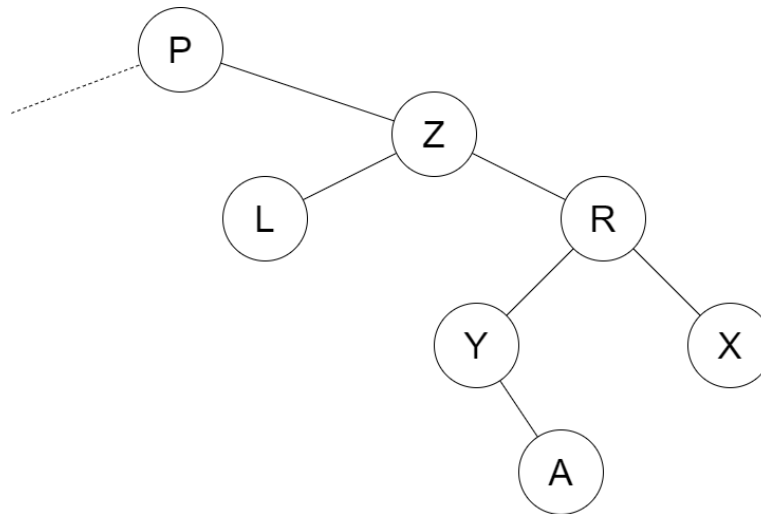
La riga 5 copre il caso in cui z ha entrambi i figli. Dobbiamo sostituire z con il suo successore:

- se y è anche figlio destro di z , lo sostituiamo semplicemente con una transplant. Dobbiamo però stare attenti a non perdere il sottoalbero sinistro di z (che è ancora attaccato a z). Per questo il figlio sinistro di z diventerà il figlio sinistro di y , e ovviamente il nuovo padre di quel nodo sarà y .
- se y non è il figlio destro di z , dobbiamo prima far prendere a y il posto di z . Più che uno “scambio” vero e proprio, giochiamo un po' coi puntatori. Innanzitutto, dobbiamo prima fare una transplant fra y e $y.right$, con l'effetto che il figlio destro di y diventerà il nuovo figlio sinistro di $z.right$, e momentaneamente ci troviamo y fuori dall'albero. Ora che il sottoalbero destro di y è “al sicuro”, dobbiamo far diventare $z.right$ il nuovo figlio destro di y . Completato questo passaggio preliminare, ci resta solo da eseguire il punto sopra.

nb: per la riga 10, siccome non abbiamo ancora modificato i puntatori di z , potremmo anche eseguire “ $z.right.parent = y$ ” e andrebbe bene comunque.

ESEMPIO di esecuzione del 3 caso della delete:

Immaginiamo di avere la seguente situazione, e vogliamo eliminare il nodo Z.



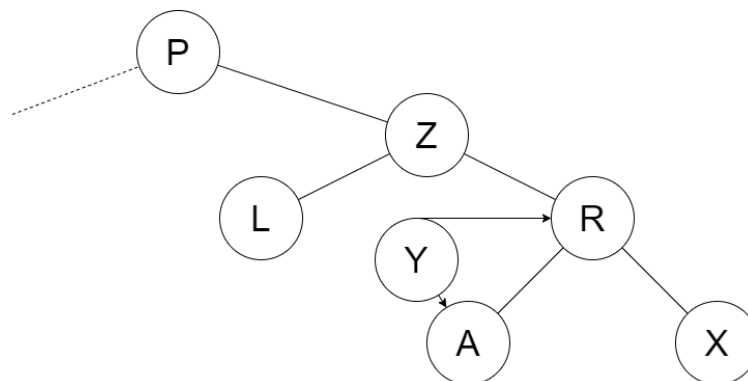
Notiamo che il nodo Z ha due figli: L e R; dunque ci troviamo nel 3 caso.

Troviamo il successore di Z, che risulta essere Y. Notiamo come $Y \neq Z.\text{right}$, dunque dobbiamo prima effettuare il passo preliminare (far prendere a Y il posto di Z.right, ovvero R).

Andando riga per riga:

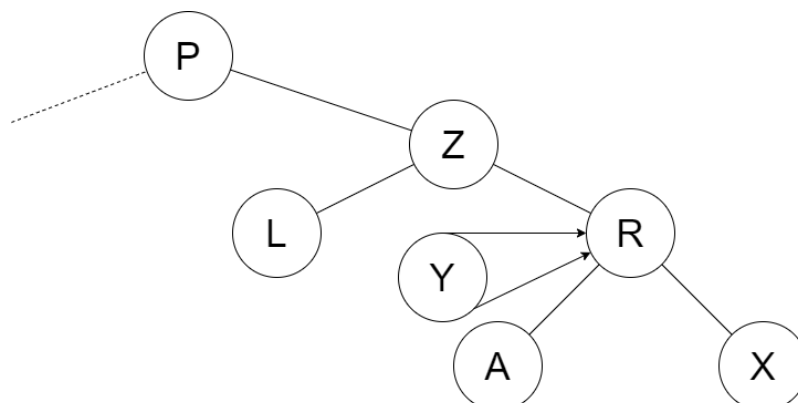
8. `TRANSPLANT(T,y,y.right)`

R considera A come figlio sinistro, e A considera R come padre:

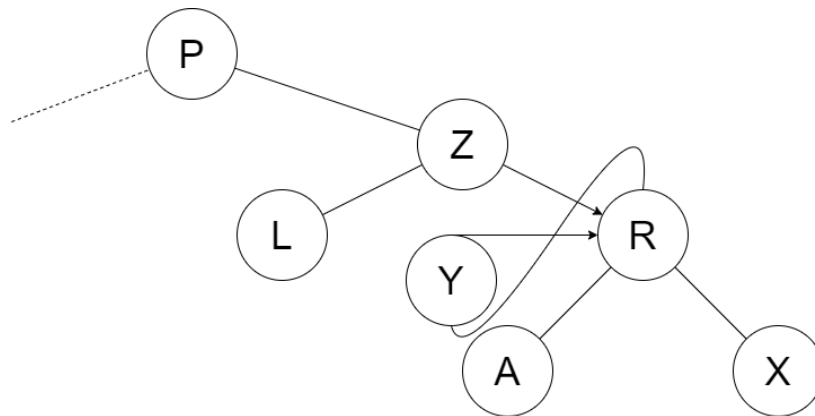


9. $y.\text{right} = z.\text{right}$

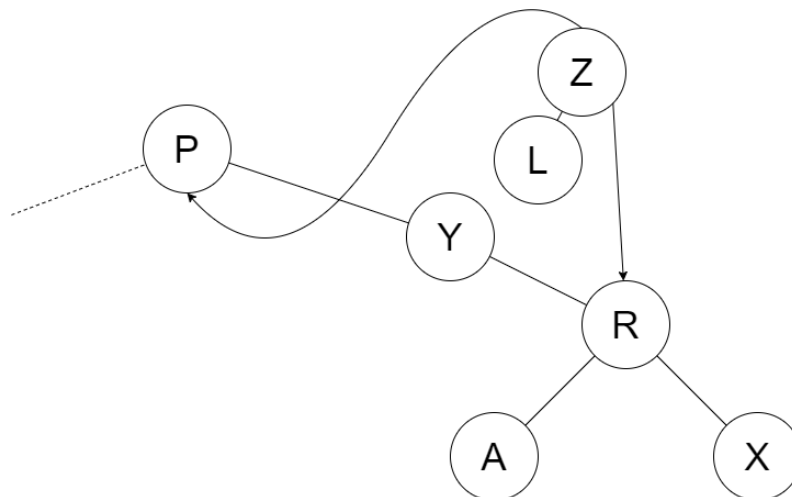
Y considera R come figlio destro (per Y, per il momento, R è sia il padre che il figlio destro):



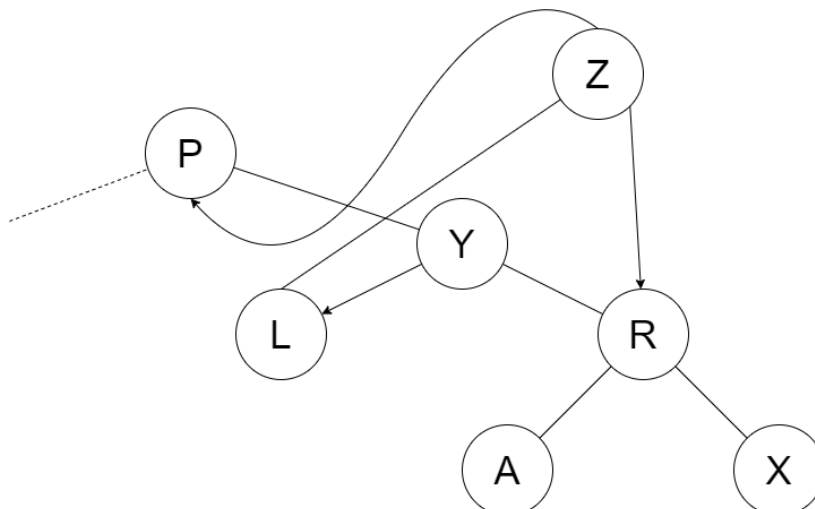
10. $y.\text{right.parent} = y$
 R considera Y come padre.



11. $\text{TRANSPLANT}(T, z, y)$
 Y considera P come padre, e P considera Y come nuovo figlio destro.
 Sia Z che il suo sottoalbero sinistro rimangono fuori dall'albero, tuttavia non vogliamo perdere questo sottoalbero (lo daremo a Y nei punti successivi).

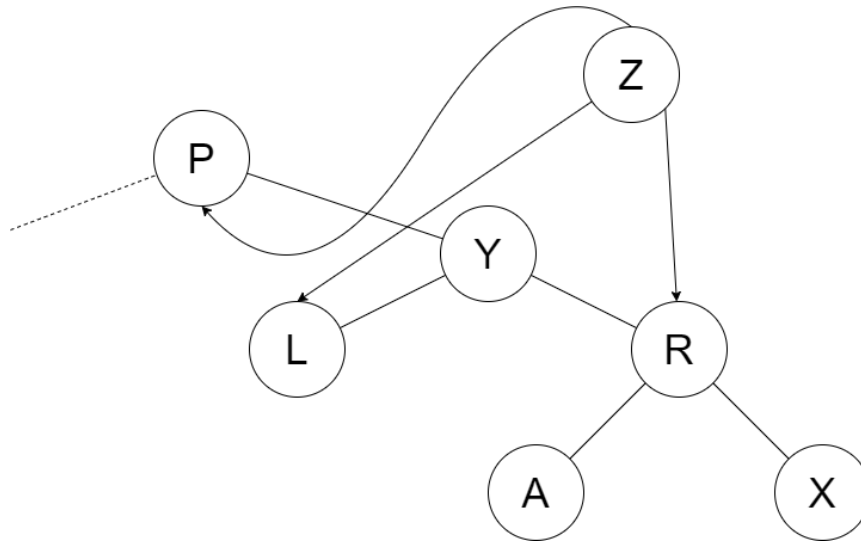


12. $y.\text{left} = z.\text{left}$
 Y considera L come figlio sinistro.

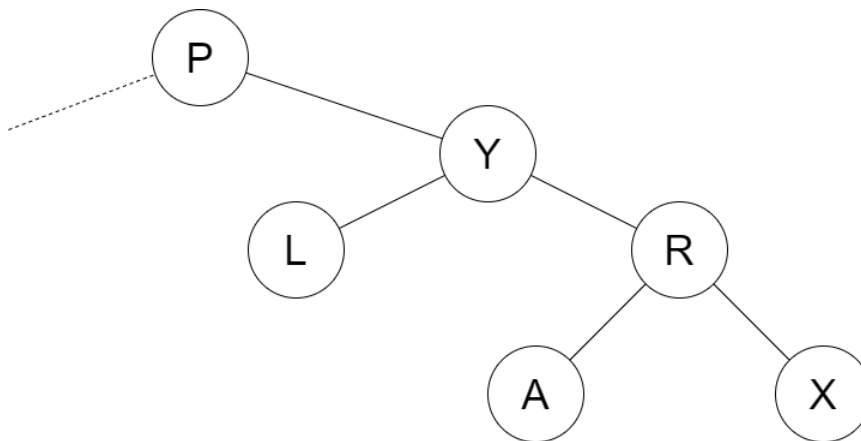


13. `y.left.parent = y`

L considera Y come nuovo padre.



Abbiamo completamente isolato il nodo Z. Se lo **rimuoviamo** dal piano, abbiamo la seguente situazione.



ABR IN C++

Creeremo una classe Node ed una classe BST che implementa tutte le operazioni viste prima. BST sarà divisa in due parti:

- *parte privata*: contiene la radice e tutti i metodi che NON vogliamo siano direttamente accessibili dall'utente, in quanto operano utilizzando i nodi.
- *parte pubblica*: contiene i metodi "wrapper" che richiamano i metodi private, astruendo il concetto di nodo.

Non vogliamo che l'utente debba avere a che fare direttamente con i nodi (ovvero che debba gestire dei puntatori a nodo). La struttura interna dell'albero deve rimanere nascosta all'utente, che dovrà occuparsi di gestire unicamente i dati.

Anche per questo motivo la classe Node è privata, e consente l'accesso ai propri membri solo alla classe BST.

Sia la classe Node che BST sono template.

Es. Main

```
int main(int, char**){
    BST<int> bst;
    bst.insert(15);
    bst.insert(6);
    bst.insert(3);
    bst.insert(2);
    bst.insert(4);
    bst.insert(7);
    bst.insert(13);
    bst.insert(9);
    bst.insert(18);
    bst.insert(17);
    bst.insert(20);
    bst.preorder();
    cout<<endl;

    BST<string> bst2;
    bst2.insert("dado");
    bst2.insert("birra");
    bst2.insert("alba");
    bst2.insert("fiume");
    bst2.insert("cinta");
    bst2.insert("dente");
    bst2.preorder();
    return 0;
}
```

risultato

15 6 3 2 4 7 13 9 18 17 20
dado birra alba cinta fiume dente

Definizione Node e BST

```
template<typename T> class BST;
template<typename T> class Node;

template<typename T>
class Node{
    T key;
    Node<T>* parent;
    Node<T>* left;
    Node<T>* right;
    Node(T key) : key{key}{
        parent = left = right = nullptr;
    }
    template<typename Tkey> friend class BST;
};

template<typename T>
class BST{
    Node<T>* root;
    void preorder(Node<T>* root);
    void inorder(Node<T>* root);
    void postorder(Node<T>* root);
    Node<T>* search(Node<T>* root, T key);
    Node<T>* searchIt(Node<T>* root, T key);
    Node<T>* min(Node<T>* root);
    Node<T>* max(Node<T>* root);
    Node<T>* successor(Node<T>* root);
    Node<T>* predecessor(Node<T>* root);
    void insert(Node<T>* z);
    void transplant(Node<T>* u, Node<T>* v);
    void deletion(Node<T>* z);
public:
    BST() : root{nullptr}{}
    void preorder();
    void inorder();
    void postorder();
    bool search(T key);
    T min();
    T max();
    void insert(T key);
    bool deletion(T key);
};
```

Implementazione metodi privati di BST

```
template<typename T>
void BST<T>::preorder(Node<T>* root){
    if(root != nullptr){
        std::cout << root->key << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

template<typename T>
void BST<T>::inorder(Node<T>* root){
    if(root != nullptr){
        inorder(root->left);
        std::cout << root->key << " ";
        inorder(root->right);
    }
}

template<typename T>
void BST<T>::postorder(Node<T>* root){
    if(root != nullptr){
        postorder(root->left);
        postorder(root->right);
        std::cout << root->key << " ";
    }
}

template<typename T>
Node<T>* BST<T>::search(Node<T>* root, T key){
    if(root == nullptr || key == root->key)
        return root;
    if(key < root->key)
        return search(root->left, key);
    else
        return search(root->right, key);
}

template<typename T>
Node<T>* BST<T>::searchIt(Node<T>* root, T key){
    Node<T>* tmp = root;
    while(tmp != nullptr && key != tmp->key)
        tmp = (key < tmp->key)? tmp->left : tmp->right;
    return tmp;
}
```

```

template<typename T>
Node<T>* BST<T>::min(Node<T>* root){
    Node<T>* tmp = root;
    while(tmp->left != nullptr)
        tmp = tmp->left;
    return tmp;
}

template<typename T>
Node<T>* BST<T>::max(Node<T>* root){
    Node<T>* tmp = root;
    while(tmp->right != nullptr)
        tmp = tmp->right;
    return tmp;
}

template<typename T>
Node<T>* BST<T>::successor(Node<T>* root){
    //caso 1: il nodo "v" ha un figlio destro, dunque il successore
    //è il minimo del sottoalbero destro di v
    if(root->right != nullptr)
        return min(root->right);

    //caso 2: il nodo "v" NON ha un figlio destro, dunque il successore
    //è il primo antenato di v tale che v sia figlio sx del proprio antenato
    Node<T>* parent = root->parent;
    while(parent != nullptr && root == parent->right){ //...risaliamo l'albero
        root = parent;
        parent = parent->parent;
    }
    return parent; //se succ non trovato, parent è un nullptr
}

template<typename T>
Node<T>* BST<T>::predecessor(Node<T>* root){
    //caso 1: il nodo "v" ha un figlio sinistro, dunque il predecessore
    //è il massimo del sottoalbero sinistro di v
    if(root->left != nullptr)
        return max(root->left);

    //caso 2: il nodo "v" NON ha un figlio sinistro, dunque il predecessore
    //è il primo antenato di v tale che v sia figlio dx del proprio antenato
    Node<T>* parent = root->parent;
    while(parent != nullptr && root == parent->left){ //...risaliamo l'albero
        root = parent;
        parent = parent->parent;
    }
    return parent; //se pred non trovato, parent è un nullptr
}

```

```

template<typename T>
void BST<T>::insert(Node<T>* z){
    Node<T>* x = this->root;
    Node<T>* xp = nullptr;
    while(x != nullptr){
        xp = x;
        x = (z->key < x->key)? x->left : x->right;
    }
    z->parent = xp;
    if(z->parent == nullptr)
        this->root = z;
    else if(z->key < z->parent->key)
        z->parent->left = z;
    else
        z->parent->right = z;
}

```

```

template<typename T>
void BST<T>::transplant(Node<T>* u, Node<T>* v){
    Node<T>* p = u->parent;
    if(p == nullptr)
        this->root = v;
    else if(u == p->left)
        p->left = v;
    else
        p->right = v;
    if(v != nullptr)
        v->parent = p;
}

```

```

template<typename T>
void BST<T>::deletion(Node<T>* z){
    if(z->left == nullptr)
        transplant(z, z->right);
    else if(z->right == nullptr)
        transplant(z, z->left);
    else{
        Node<T>* y = min(z->right);
        if(y != z->right){
            transplant(y, y->right);
            y->right = z->right;
            y->right->parent = y;
        }
        transplant(z, y);
        y->left = z->left;
        y->left->parent = y;
    }
}

```

Implementazione metodi pubblici di BST

```
template<typename T>
void BST<T>::preorder(){
    preorder(this->root);
}

template<typename T>
void BST<T>::inorder(){
    inorder(this->root);
}

template<typename T>
void BST<T>::postorder(){
    postorder(this->root);
}

template<typename T>
bool BST<T>::search(T key){
    Node<T>* found = search(this->root, key);
    return (found != nullptr)? true : false;
}

template<typename T>
T BST<T>::min(){
    Node<T>* m = min(this->root);
    return m->key;
}

template<typename T>
T BST<T>::max(){
    Node<T>* m = max(this->root);
    return m->key;
}

template<typename T>
void BST<T>::insert(T key){
    insert(new Node(key));
}

template<typename T>
bool BST<T>::deletion(T key){
    Node<T>* z = search(this->root, key);
    if(z == nullptr)
        return false;
    deletion(z);    //cut out 'z' from the tree
    delete(z);      //deallocation
    return true;
}
```


LEZ 13/14 – ALBERI RED BLACK (ARB)

Un albero Red-Black (o *balanced search tree*) è un particolare tipo di albero di ricerca in grado di **autobilanciare** la propria struttura, e che consiste in una struttura dati tipicamente usata per implementare array associativi.

Si tratta di una struttura complessa, ma è molto efficiente: effettua ricerche, inserimenti e cancellazioni in un tempo **$O(\log n)$** , dove n è il numero di elementi dell'albero.

In tale albero:

- Le **chiavi** sono mantenute solo nei **nodi interni** dell'albero;
- Le **foglie** sono costituite da **nodi nulli** (nb: nodi NIL).

Tale albero deve rispettare **5 proprietà**, quali:

1. Ogni nodo è colorato di **rosso** o di **nero**;
2. La radice è **nera**;
3. I **nodi NIL** (foglie) sono **neri**;
4. Se un nodo è **rosso**, allora i suoi **figli** sono **neri**;
5. Ogni **percorso** da un nodo interno ad una foglia ha **lo stesso numero di nodi neri** (**b-altezza**).

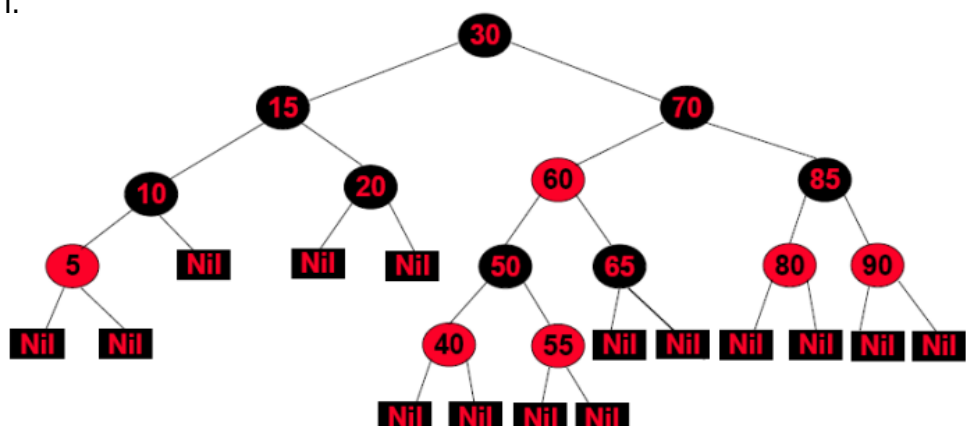
ed ogni nodo ha:

- Puntatore al padre;
- Puntatore al figlio destro;
- Puntatore al figlio sinistro;
- Chiave;
- **Colore**;
- Dati satelliti.

Se un **nodo è rosso**, allora entrambi i **suoi figli sono neri**; da notar bene, tuttavia, che un nodo nero può avere figli neri.

nb: un albero RB può avere più colorazioni possibili.

nb: un nodo rosso non può avere un padre o un figlio rosso.



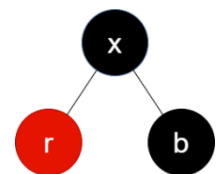
TEOREMA

Ogni albero RB, con radice x , ha almeno $2^{bh(x)} - 1$ nodi interni, dove $bh(x)$ è l'altezza *nera* del nodo x , e dove l'altezza nera $bh(x)$ è il numero di nodi neri in un cammino semplice dal nodo x ad una foglia (escluso x).

DIMOSTRAZIONE

Si dimostra per induzione:

- Se l'altezza di x è 0, allora x è una foglia ed il sottoalbero ha $2^{bh(x)} - 1 = 1 - 1 = 0$ nodi interni.
- Per il passo induttivo si può considerare un nodo interno x con due figli:
 - **figlio rosso r** : altezza nera = $bh(x)$;
è la stessa del padre in quanto non contribuisce alla b-altezza di x .
 - **figlio nero b** : altezza nera = $bh(x) - 1$.
contribuisce alla b-altezza di x , dunque la sua b-altezza sarà uguale a quella del padre *meno* il contributo che lui fornisce (1).



Ogni figlio ha almeno $2^{bh(x)-1} - 1$ nodi interni. Pertanto il sottoalbero ha $(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$ nodi interni.

ALTRI TEOREMI

- In un albero RB, almeno la metà dei nodi dalla radice alla foglia deve essere nera;
- In un albero RB, nessun percorso da un nodo v ad una foglia è lungo più del doppio del percorso da v ad un'altra foglia;
- In un albero RB, con n nodi, l'altezza può essere al massimo $2\log(n+1)$.

DIMOSTRAZIONI

Sia h l'altezza dell'albero.

Per la prima proprietà, almeno la metà dei nodi in un cammino dalla radice ad una foglia, esclusa la radice, devono essere neri. Perciò la b-altezza deve essere almeno $\frac{h}{2}$.

Sapendo inoltre che il numero di nodi è superiore (o uguale) a $2^{bh(x)} - 1$, avremo che:

- $n \geq 2^{\frac{h}{2}} - 1$
- $n + 1 \geq 2^{\frac{h}{2}}$
- $\log(n + 1) \geq h/2$
- $2\log(n + 1) \geq h$

Ciò che rende speciale questa struttura è proprio la sua capacità di autobilanciarsi. Infatti, supponiamo di voler effettuare una ricerca in un ABR. Potrebbe capitare che tale ABR abbia tutti i nodi sulla sinistra (o sulla destra), rendendolo non diverso da una lista. Ciò significa che se l'elemento da ricercare si trovasse in fondo a tale albero, impiegheremmo un tempo n per trovarlo.

Ad esempio, un albero costruito con una sequenza di numeri decrescente (10,8,5,1) si ritroverà con tutti i nodi sulla sinistra, assumendo la forma di una lista. Per trovare 1, quindi, dovremo attraversare tutti i nodi dell'albero.

La risposta a tale problema è data proprio dagli **Alberi Red-Black**, i quali hanno un'altezza garantita di **$O(\log n)$** per n nodi.

Operazioni su Alberi RB

Le operazioni Search, Minimum, Maximum, Successor e Predecessor definite sugli ABR funzionano anche sugli alberi RB e sono eseguite in tempo **$O(h)$** , ovvero **$O(\log n)$** .

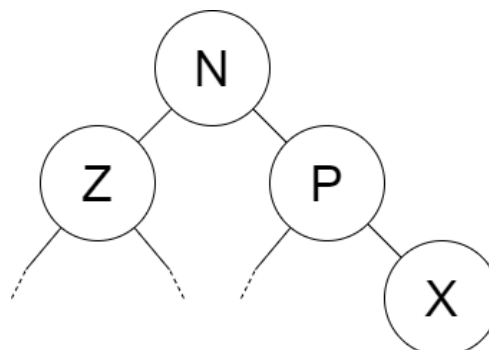
Invece le operazioni di Insert e Delete sono più complicate in quanto modificano la struttura dell'albero e potrebbero quindi violarne le proprietà.

Tali proprietà possono essere ripristinate attraverso modifiche al colore dei nodi, o modifiche alla struttura dei puntatori mediante un'ulteriore operazione chiamata **rotazione**.

Tutte queste operazioni sono eseguite in tempo $O(\log n)$.

NB: RELAZIONI ALL'INTERNO DI UN ALBERO BINARIO

Prendendo in considerazione il seguente albero binario:



Consideriamo il nodo X:

- P è il padre di X;
- N è il nonno di X;
- Z è lo zio di X.

ROTAZIONE

La rotazione altera la struttura dell'albero attraverso la riorganizzazione dei sottoalberi, con l'intento di diminuire l'altezza dell'albero.

Ricordando che gli alberi RB hanno un'altezza massima di $O(\log n)$, possiamo diminuire l'altezza spostando i sottoalberi più grandi verso l'alto e i sottoalberi più piccoli verso il basso.

Da notare che la rotazione non impatta l'ordine degli elementi: quelli più grandi saranno sempre sulla destra, mentre i più piccoli saranno sempre sulla sinistra.

La rotazione consiste sostanzialmente nell'invertire la relazione "padre-figlio", e possiamo distinguere **due tipi**:

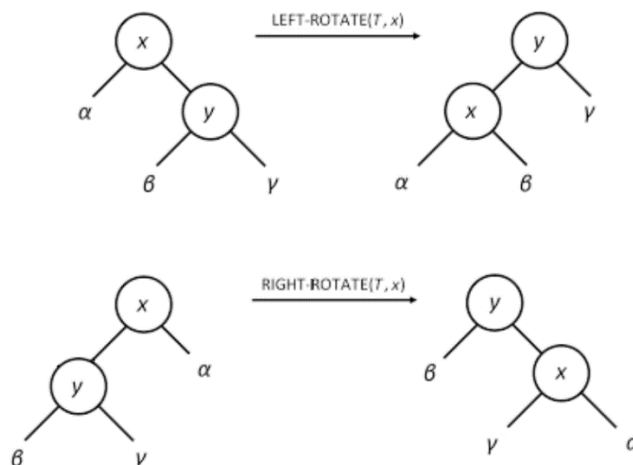
left-rotate: sul nodo x , significa che il figlio destro di x (y) prenderà il posto di x , ed x diventerà il figlio sinistro di y .

y potrebbe però avere un figlio sinistro, dunque il figlio sinistro di y diventerà il nuovo figlio destro di x .

right-rotate: sul nodo x , significa che il figlio sinistro di x (y) prenderà il posto di x , ed x diventerà il figlio destro di y .

y potrebbe però avere un figlio destro, dunque il figlio destro di y diventerà il nuovo figlio sinistro di x .

Esempio: Supponiamo di avere questi due alberi



1) eseguiamo una left-rotate sul nodo x . Ciò significa che il suo figlio destro y prenderà il suo posto, ed x diventerà figlio sinistro di y . Il figlio sinistro di y , ovvero β , diventerà il nuovo figlio destro di x .

2) eseguiamo una right-rotate sul nodo x . Ciò significa che il suo figlio sinistro y prenderà il suo posto, ed x diventerà figlio destro di y . Il figlio destro di y , ovvero γ , diventerà il nuovo figlio sinistro di x .

Notiamo che effettuiamo tali passaggi proprio per mantenere la proprietà per cui tutti i nodi maggiori di quello corrente vanno a destra, mentre quelli inferiori vanno a sinistra.

ROTAZIONE - PSEUDOCODICE

Left-Rotate(T,x)

```
1. y = x.right          //y è un puntatore a nodo di appoggio
2. x.right = y.left
3. if y.left ≠ NIL
4.     y.left.p = x
5. y.p = x.p
6. if x.p == NIL
7.     T.root = y
8. else if x == x.p.left
9.     x.p.left = y
10. else
11.     x.p.right = y
12. y.left = x
13. x.p = y
```

L'istruzione 1 crea un puntatore a nodo di appoggio **y**, e lo fa puntare al nodo destro di **x**.

La 2 comincia già ad impostare il figlio destro di **x** che punta al figlio sinistro di **y**. Ciò però è ancora solo un'assegnazione unilaterale (infatti non abbiamo ancora modificato i puntatori a nodo di **y** e del suo figlio sinistro).

La 3/4 imposta il padre del figlio sinistro di **y** come **x** (adesso quel nodo ed **x** si puntano in modo bilaterale). Ovviamente ciò possiamo farlo solo se il nodo non è NIL. Se fosse NIL, non dovremmo preoccuparci di questa assegnazione.

Dalla 5 alla 11 ci occupiamo di far prendere ad **y** il posto di **x**. In particolare:

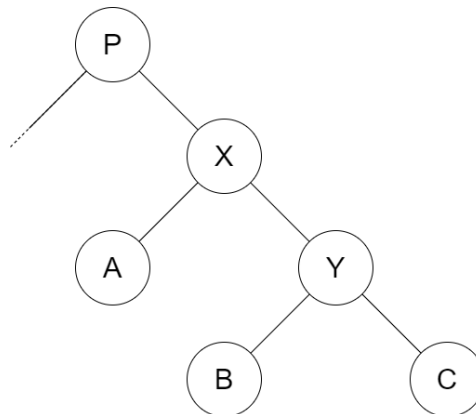
- la 5 imposta il padre di **y** come il padre di **x**.
- dalla 6 alla 11 ci occupiamo di impostare **y** come nuovo figlio del padre di **x**.
Possono verificarsi 3 casi:
 - il padre di **x** è NIL, ovvero **x** è la radice e dunque impostiamo **y** come nuova radice dell'albero.
 - **x** è il figlio sinistro di suo padre, dunque imposteremo **y** come figlio sinistro del padre di **x**.
 - **x** è il figlio destro di suo padre, dunque imposteremo **y** come figlio destro del padre di **x**.

La 12 imposta il figlio sinistro di **y** come **x**.

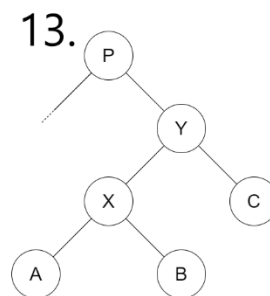
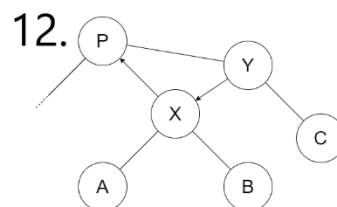
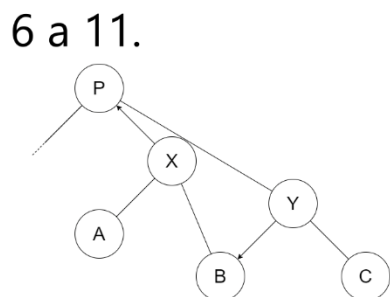
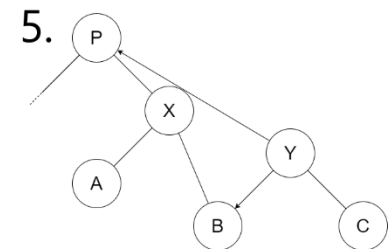
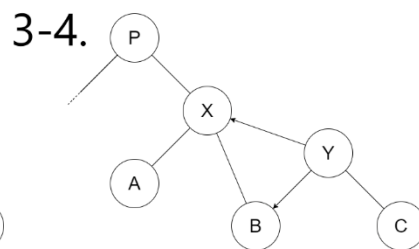
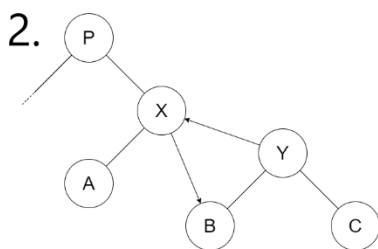
La 13 imposta il padre di **x** come **y**.

Esempio di Left Rotate

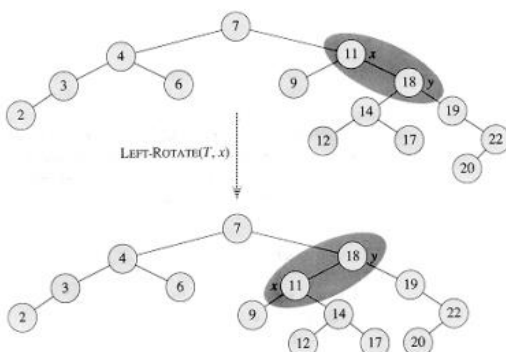
Prendiamo il seguente albero di partenza:



Eseguiamo i vari passi:



Esempio 2:



Right-Rotate(T,x)

```
1. y = x.left      //y è un puntatore a nodo di appoggio
2. x.left = y.right
3. if y.right ≠ NIL
4.     y.right.p = x
5. y.p = x.p
6. if x.p == NIL
7.     T.root = y
8. else if x == x.p.left
9.     x.p.left = y
10. else
11.     x.p.right = y
12. y.right = x
13. x.p = y
```

Notiamo come il codice sia analogo al left-rotate(T,x), con la differenza che invertiamo left e right (nb: righe 1 a 4, e riga 12). Le righe da 5 a 11, e la riga 13 rimangono invariate.

INSERT

Per inserire un nodo all'interno di un albero RB dobbiamo seguire **tre passi** fondamentali:

1. Colorare il nuovo nodo di **rosso**;
2. Ricercare la giusta posizione del nodo all'interno dell'albero, usando la stessa procedura usata per i normali ABR;
3. Ricolorare e ruotare i nodi per "aggiustare" la violazione.

Durante un inserimento, **possono essere violate 2** delle 5 **proprietà** di un ARB:

prop 2: la radice è nera;

prop 4: se un nodo è rosso, allora entrambi i suoi figli sono neri.

Una volta colorato di rosso il nodo X, e una volta inserito all'interno dell'albero, dobbiamo correggere eventuali violazioni. Consideriamo **2 situazioni**, ovvero se il padre di X sia il figlio sinistro o destro del nonno di X, e per ogni situazione possiamo considerare **3 casi**.

1. Se il padre di X è il figlio sinistro del nonno:

- **caso 1:** lo zio Z di X è rosso.
Cambiamo i colori di alcuni nodi, preservando la proprietà 5 (tutti i percorsi sotto a questi nodi hanno altezza nera uguali): il padre e lo zio diventano neri, e il nonno diventa rosso.
Poi si continua verso l'alto facendo del nonno (padre del padre) di X il nuovo X, e rieseguiremo il ciclo ma su questo X.
- **caso 2:** lo zio Z di X è nero, ed X è un figlio destro.
Facciamo del padre di X il nuovo X, e trasformiamo questo caso nel caso 3 effettuando una left-rotate sul nuovo X. Poi, procediamo col caso 3.
- **caso 3:** lo zio Z di X è nero, ed X è un figlio sinistro.
Cambiamo i colori di alcuni nodi: il padre diventa nero, e il nonno diventa rosso.
Eseguiamo poi una right-rotate sul nonno.
Dopo la right-rotate, il padre avrà preso il posto del nonno, ed essendo il padre nero non avremo bisogno di effettuare ulteriori iterazioni.

2. Se il padre di X è il figlio destro del nonno, il discorso è simile a prima:

- **caso 1:** *uguale a prima*.
- **caso 2:** lo zio Z di X è nero, ed X è un figlio sinistro.
Facciamo del padre di X il nuovo X, e trasformiamo questo caso nel caso 3. effettuando una right-rotate sul nuovo X. Poi, procediamo col caso 3.
- **caso 3:** lo zio Z di X è nero, ed X è un figlio destro.
Cambiamo i colori di alcuni nodi: il padre diventa nero, e il nonno diventa rosso.
Eseguiamo poi una left-rotate sul nonno.

NB:

Nel caso il nodo X fosse la radice dell'albero, allora ci limitiamo a colorarlo di nero (**caso 0**).

L'unico caso in cui si procede a ripristinare verso l'alto è il caso 1. Negli altri 2, il padre di X diventerà nero, dunque finita l'iterazione si uscirà dal WHILE e si termina.

La situazione 1 e la situazione 2 sono molto simili, solo che hanno LEFT e RIGHT invertite (incluse le operazioni left-rotate e right-rotate). Inoltre, nella situazione 1 il caso 2 ci porta al caso 3, mentre nella situazione 2 è il caso 3 a portarci nella situazione 2.

PSEUDO-CODICE

TREE_INSERT_RB(T,x)

```
1.  x.color = RED //passo 1, colorare X di rosso
2.  TREE_INSERT(T,x) //passo 2, aggiungere X all'albero

3.  while x ≠ T.root and x.p.color ≠ BLACK //passo 3, correggiamo violazioni
4.      if x.p = x.p.p.left //il padre di x è il figlio sx del nonno, sit. 1
5.          z = x.p.p.right //allora lo zio sarà il figlio dx del nonno
6.          if z.color = RED //caso 1, lo zio è rosso
7.              x.p.color = BLACK
8.              z.color = BLACK
9.              x.p.p.color = RED
10.             x = x.p.p
11.         else //lo zio non è rosso
12.             if x = x.p.right //caso 2, zio n. e x è il f. dx, tr in 3
13.                 x = x.p
14.                 LEFT_ROTATE(T,x)
15.                 x.p.color = BLACK //caso 3, zio nero e x è il figlio sx
16.                 x.p.p.color = RED
17.                 RIGHT_ROTATE(T,x.p.p)
18.         else //il padre di x è il figlio dx del nonno, sit. 2
19.             z = x.p.p.left //allora lo zio sarà il figlio sx del nonno
20.             if z.color = RED //caso 1, lo zio è rosso
21.                 x.p.color = BLACK
22.                 z.color = BLACK
23.                 x.p.p.color = RED
24.                 x = x.p.p
25.             else //lo zio non è rosso
26.                 if x = x.p.left //caso 2, zio n. e x è il f. sx, trasforma in 3
27.                     x = x.p
28.                     RIGHT_ROTATE(T,x)
29.                     x.p.color = BLACK //caso 3, zio nero e x è il figlio dx
30.                     x.p.p.color = RED
31.                     LEFT_ROTATE(T,x.p.p)

32. T.root.color = BLACK //caso 0, la radice è nera
```

NB:

Da un punto di vista pratico, possiamo trovare problemi quando vogliamo aggiungere un secondo o terzo nodo all'albero (Il nonno di questi nodi sarebbe il padre della radice, ovvero un nodo NIL, e i ragionamenti visti prima non funzionano).

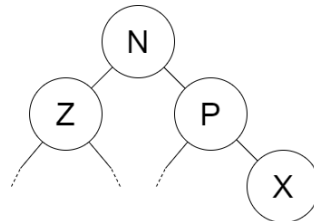
La soluzione dipende da come vogliamo gestire i nodi NIL, ma una semplice consiste nell'aggiungere al while un'ulteriore condizione: il padre del nodo da aggiungere deve essere diverso dalla radice.

Avremo dunque:

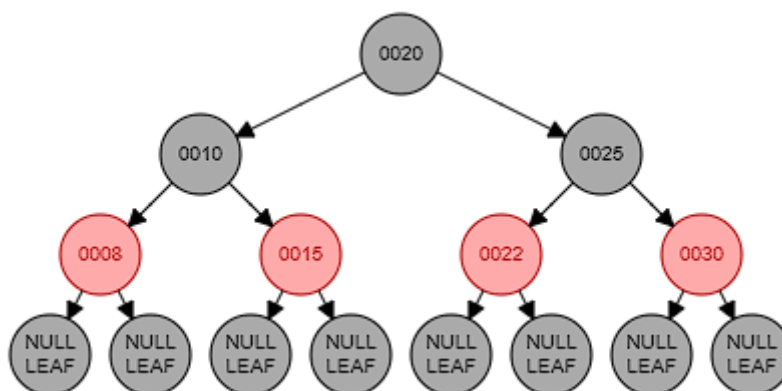
```
while x ≠ T.root and x.p.color ≠ BLACK and x.p != T.root
```

ESEMPI

Ricordando che: X è il nodo appena inserito, P è il padre di X, N è il nonno di X, e Z è lo zio di X, avremo una struttura del tipo:



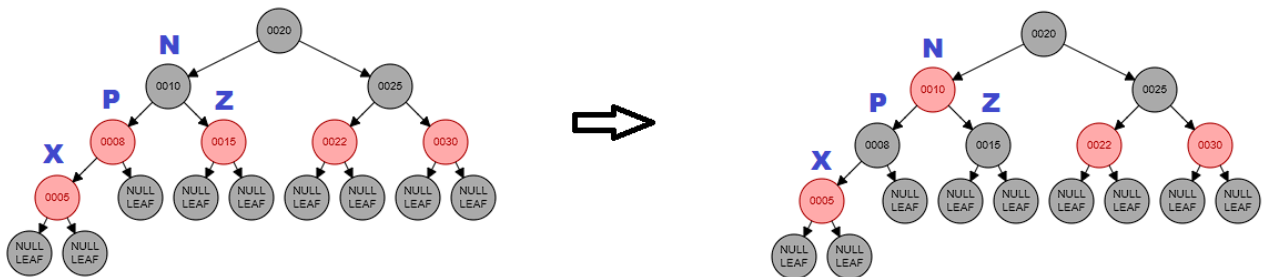
Albero di partenza:



Esempi di situazioni in cui il padre è figlio sinistro del nonno

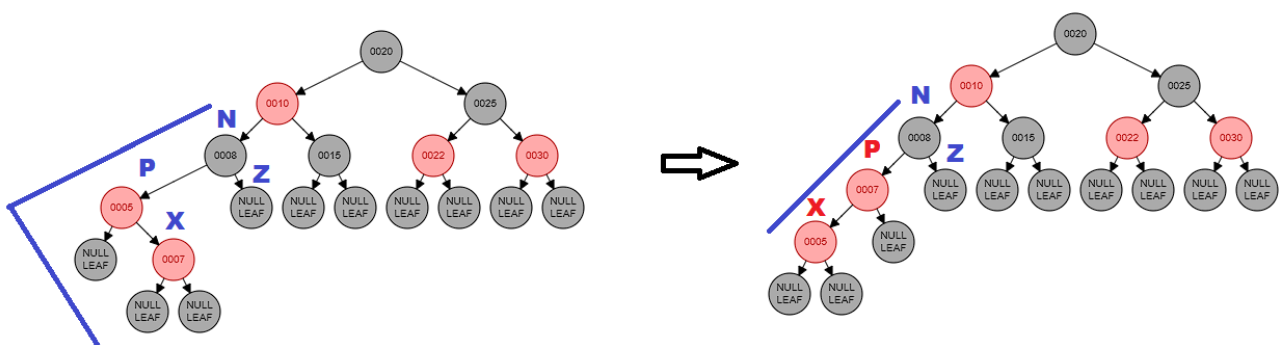
Inseriamo 5 nell'albero di riferimento:

1.1. lo zio è rosso



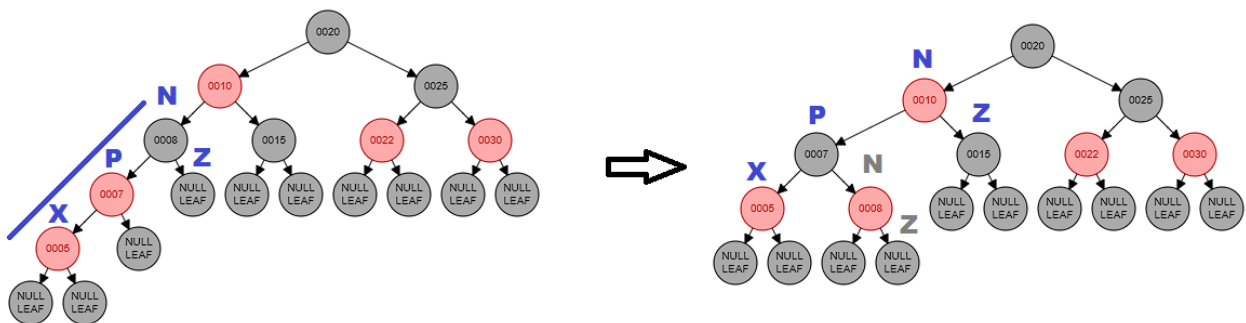
Inseriamo anche 7 nell'albero ottenuto prima:

1.2 lo zio è nero, e x è figlio destro



Continuiamo il caso di prima. Notiamo come dal caso 2 siamo passati al 3:

1.3 lo zio è nero, e x è figlio sinistro



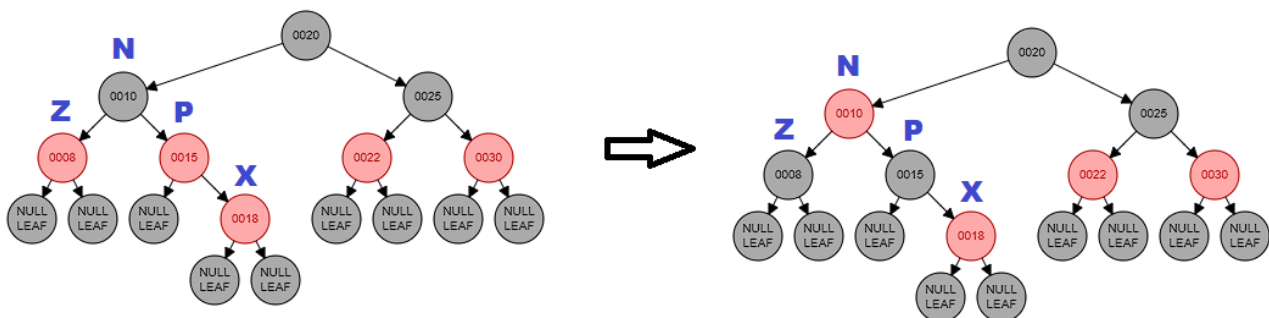
IN SOSTANZA:

- **caso 1, lo zio è rosso**: dobbiamo solo cambiare i colori del padre, dello zio e del nonno. Padre e zio diventano neri, e il nonno diventa rosso. La violazione viene passata al nonno ($x = x.p.p$).
- **caso 2, zio nero** e *in pratica* fra nonno, padre e figlio si forma un **rombo** (x è figlio destro). Facciamo diventare il padre il nuovo x, ed effettuiamo una LEFT_ROTATE su questo nuovo x (il padre diventa ora figlio sx di del vecchio x). La violazione è quindi spostata al padre prima di cadere nel caso 3 (notare come fra nonno, padre e figlio si sia formata una linea dritta).
- **caso 3, zio nero** e *in pratica* fra nonno, padre e figlio si forma una **linea dritta** (x è figlio sinistro). Anche in questo voglio che padre e zio siano neri, ma essendo che lo zio è già nero dovrò solo far diventare il padre nero. Poi faccio diventare il nonno rosso. Effettuiamo poi una RIGHT_ROTATE sul nonno, e quindi il padre diventerà la nuova radice. Essendo che il padre di X è nero, non c'è più alcuna violazione da risolvere.

Esempi di situazioni in cui il padre è figlio destro del nonno (sono speculari)

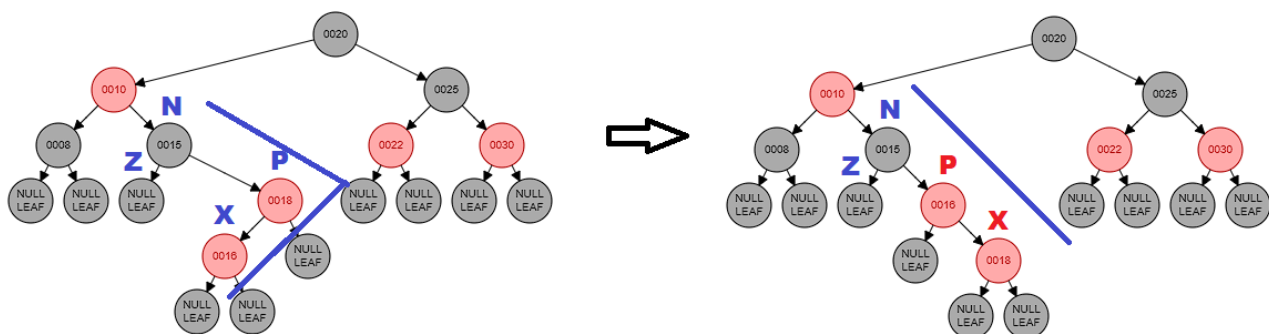
Inseriamo 18 nell'albero di riferimento:

2.1 lo zio è rosso



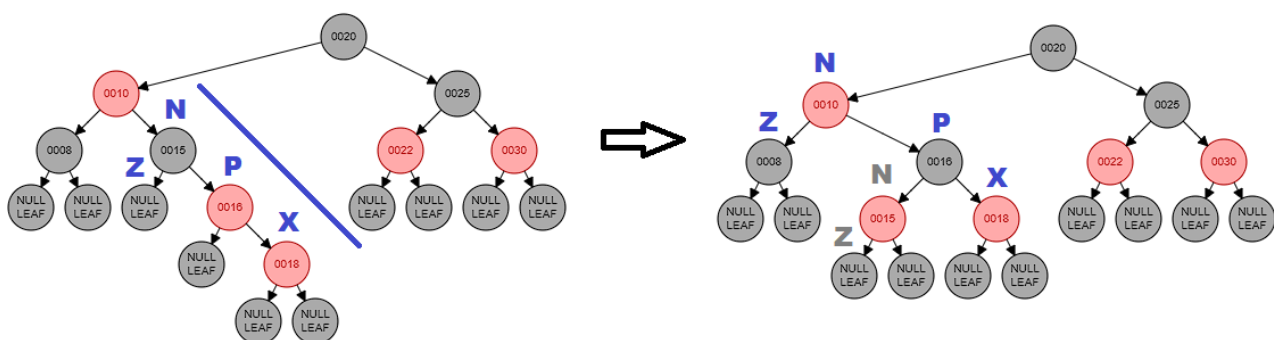
Inseriamo anche 16 nell'albero ottenuto prima:

2.2 lo zio è nero, e x è figlio sinistro



Continuiamo il caso di prima. Notiamo come dal caso 2 siamo passati al 3:

2.3 lo zio è nero, e x è figlio destro



IN SOSTANZA:

Vale lo stesso di quanto detto con la situazione 1. Ricordiamo che se fra nonno, padre e figlio si forma un **rombo** siamo nel caso 2, e vogliamo farlo diventare una **linea dritta** (facendo diventare il padre il nuovo x, e IN QUESTO CASO facendo una RIGHT_ROTATE su questo nuovo x). Se fra nonno, padre e figlio si forma una **linea dritta**, siamo nel caso 3.

DELETION

L'algoritmo di cancellazione per alberi RB è costruito sulla base di quello per ABR, ma stavolta dovremo fare anche attenzione a preservare il bilanciamento dell'albero.

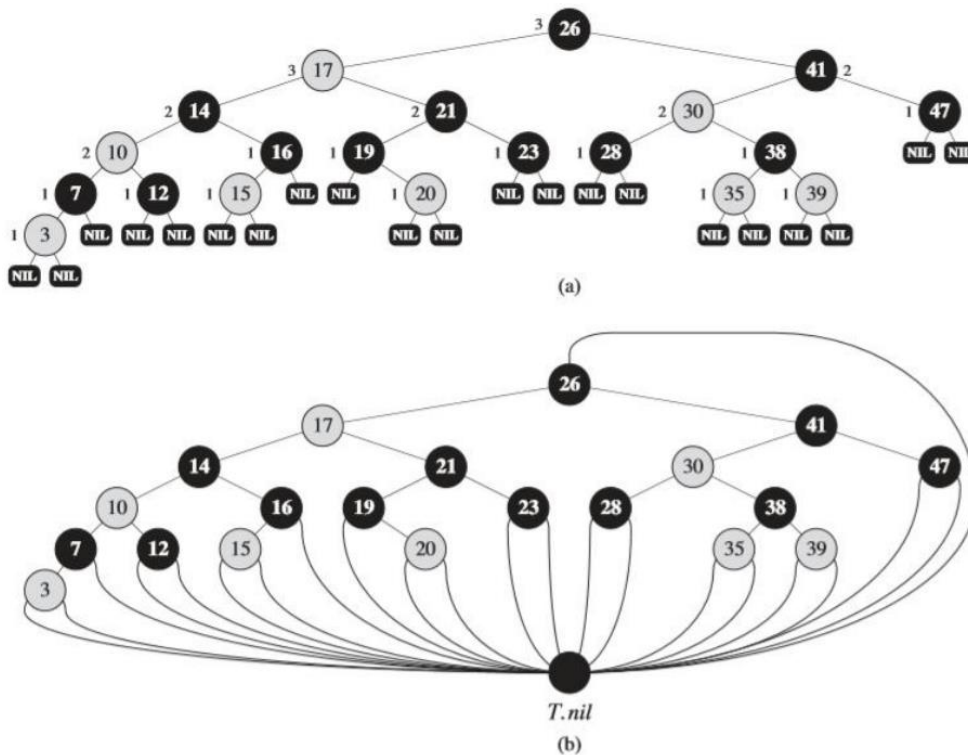
Utilizziamo:

- una **variante di TRANSPLANT**.
- Una **sentinella T.NIL**: si tratta di un nodo di colore nero in cui gli attributi (p, left, right, e key) possono assumere valori arbitrari (non ci interessano).

Il nodo T.NIL

La comodità di avere una sentinella T.NIL è che ci permette di evitare di allocare un nodo NIL per ogni foglia. Infatti, tutti i "nodi prima delle foglie" non punteranno ad un diverso nodo foglia NIL, ma tutti punteranno allo stesso, cioè T.NIL (lo stesso vale per il "parent" della radice).

esempio:



Transplant per alberi RB

Ricordiamo che:

la procedura Transplant consente di **sostituire** un sottoalbero con un altro sottoalbero.

In sostanza, significa modificare “chi punta a chi”. Se vogliamo sostituire il nodo z con y , significa che il padre di z considera come figlio (punta a) y , e y considera come padre (punta a) il padre di z .

La differenza principale con la Transplant “base” è che in questo caso **utilizziamo T.NIL** al posto di NIL. Essendo T.NIL un nodo a tutti gli effetti, alla fine non dobbiamo preoccuparci di verificare se v è un nodo NIL prima di assegnargli il padre di u .

RB_TRANSPLANT(T, u, v)

```
1.  p = u.p
2.  if p = T.NIL
3.      T.root = v
4.  else if u = p.left
5.      p.left = v
6.  else
7.      p.right = v
8.  v.p = p
```

//facciamo direttamente l'assegnazione, senza fare prima il controllo

Cancellazione in RB

La procedura RB-DELETE presenta delle istruzioni in più rispetto a TREE-DELETE, che servono a tenere traccia di due particolari nodi, **y** e **x**. Infatti il nodo **y** può creare delle violazioni nell'albero, che andranno corrette sfruttando il nodo **x**.

Quando vogliamo cancellare un nodo **z**:

- se **z** ha meno di due figli, **y** punta a **z** (che viene rimosso);
- se **z** ha due figli, **y** punta al successore di **z** e sostituirà **z**.

In entrambi i casi dobbiamo ricordare il **colore di y** prima di rimuoverlo o spostarlo, e dobbiamo tenere traccia del nodo **x** che prende il posto di **y** poiché potrebbe causare delle violazioni (**x** è un nodo di appoggio che può puntare a nodi diversi in base all'if, e viene richiamato solo se il colore originale di **y** era nero).

Dopo aver eliminato **z**, se il nodo **y** era nero, è necessario richiamare la procedura RB-DELETE-FIXUP sul nodo **x** per ripristinare le proprietà degli alberi cambiando i colori di alcuni nodi ed effettuando delle rotazioni.

RB_DELETE(T,z)

```
1.  y = z
2.  yOC = y.color    //y Original Color
3.  if z.left = T.NIL
4.      x = z.right
5.      RB_TRANSPLANT(T,z,z.right)
6.  else if z.right = T.NIL
7.      x = z.left
8.      RB_TRANSPLANT(T,z,z.left)
9.  else
10.     y = TREE_MIN(z.right)
11.     yOC = y.color
12.     x = y.right
13.     if y.p = z
14.         x.p = y          //se x == T.NIL
15.     else
16.         RB_TRANSPLANT(T,y,y.right)
17.         y.right = z.right
18.         y.right.p = y
19.         RB_TRANSPLANT(T,z,y)
20.         y.left = z.left
21.         y.left.p = y
22.         y.color = z.color    //y prende il colore di z
23. if yOC = BLACK
24.     RB_DELETE_FIXUP(T,x)
```

La procedura FIXUP

La procedura di FIXUP viene richiamata solo se il colore originale di **y** era nero. Infatti:

- se **y** era rosso:
 - le altezze nere dell'albero non sono cambiate;
 - non sono stati creati nodi rossi consecutivi:
 - se **y** era figlio destro di **z**, in quanto rosso non poteva essere figlio di un nodo rosso.
 - se **y** non era figlio destro di **z**, il figlio destro di **y**, cioè **x**, prende il posto di **y**. Se **y** è rosso, **x** deve essere nero per cui la sostituzione non crea problemi.
- se **y** era nero:
 - **y** potrebbe essere la radice ed avere un figlio rosso che diventa la nuova radice.
 - rimuovendo **y**, avremo che **x** e **x.p** potrebbero essere entrambi rossi.
 - Spostando **y** (nero), qualunque cammino che conteneva **y** avrà un nero "mancante". Bisogna dunque aggiungere un **nero "extra"** ad **x** (che sostituisce **y**).

nb: da questo punto in poi consideriamo che i nodi possano avere un "doppio strato di colore".

L'**obiettivo** è di spostare verso l'alto il nero "extra" finché:

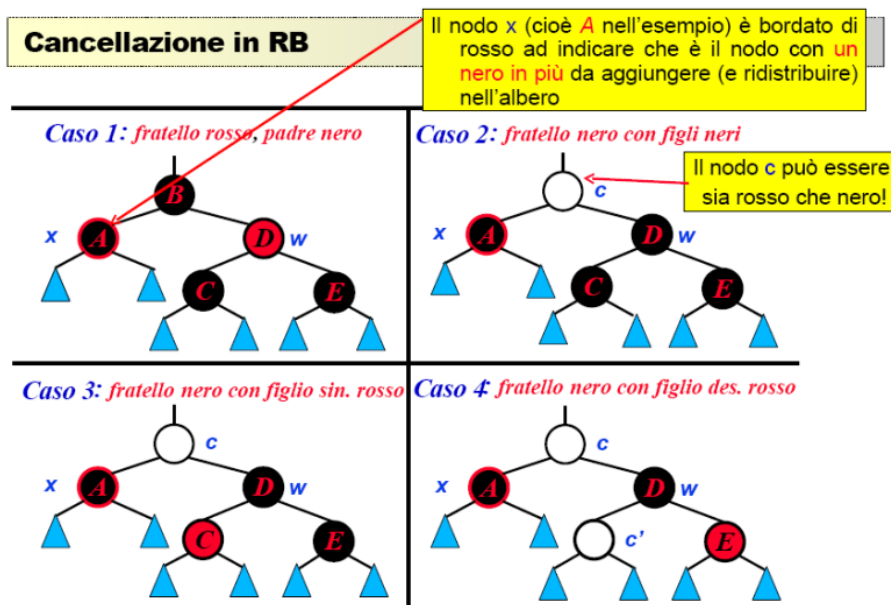
- **x** punta ad un nodo rosso-nero, nel qual caso **x** diventa solo nero (singolo);
- **x** punta alla radice, nel qual caso il nero extra può essere rimosso.

Finché una di queste due condizioni non viene soddisfatta, **x** continuerà a puntare ad un nodo doppiamente nero che non sia la radice.

Da un punto di vista implementativo, sono 4 i casi che possono presentarsi, i quali possono presentarsi in due versioni simmetriche (dipende se **x** è figlio sinistro o destro). Dunque 2 situazioni * 4 casi.

Oltre **x**, dovremo anche mantenere un puntatore al fratello di **x**, che chiamiamo **w**.

1. Se il padre di X è il figlio sinistro del nonno:



caso 1: il fratello w è rosso, e il padre $x.p$ è nero.

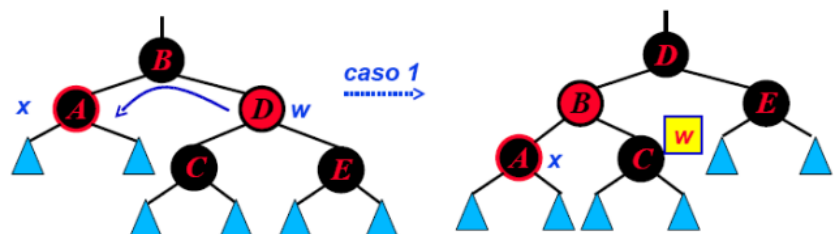
Se w è rosso, significa che deve avere entrambi i figli neri. Possiamo scambiare i colori di w e $x.p$ e poi effettuare una LEFT_ROTATE su $x.p$.

Il nuovo fratello di x , che era uno dei figli di w , è nero.

Da questo caso si ricade poi nel caso 2, oppure nei casi 3 e/o 4.

In sostanza, avremo:

```
if w.color = RED
    w.color = BLACK
    x.p.color = RED
    LEFT_ROTATE(T, x.p)
    w = x.p.right
```



caso 2: il fratello w è nero, ed entrambi i suoi figli sono neri.

Togliamo un nero sia da w che da x , ottenendo x singolarmente nero e w rosso.

Aggiungiamo poi un nero a $x.p$ (che diventerà rosso-nero, oppure doppio-nero).

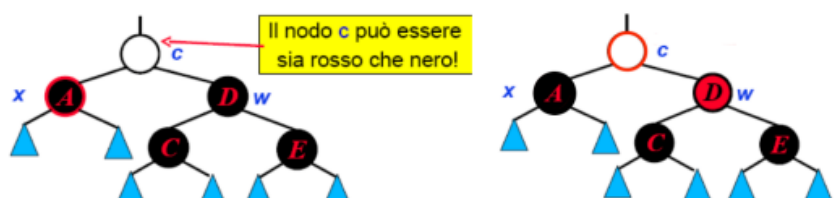
Valutiamo la condizione del ciclo con $x = x.p$:

Da questo caso, si va direttamente alla prossima iterazione del ciclo.

- se x è rosso-nero, il ciclo non viene ripetuto ed il nero extra viene rimosso;
- se x è nero-nero, il ciclo viene ripetuto.

In sostanza, avremo:

```
if w.left.color = BLACK and w.right.color = BLACK
    w.color = RED
    x = x.p
```



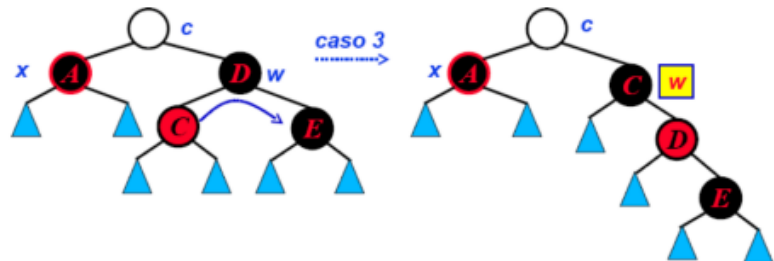
caso 3: il fratello **w** è nero, e ha il figlio sinistro rosso.

Questo significa che il figlio destro di **w** è nero. Scambiamo i colori di **w** e **w.left** e poi effettuiamo una RIGHT_ROTATE su **w**.

Il nuovo fratello **w** di **x** è nero con un figlio rosso, per cui ricadiamo nel caso 4.

In sostanza, avremo:

```
else if w.right.color = BLACK
    w.left.color = BLACK
    w.color = RED
    RIGHT_ROTATE(T,w)
    w = x.p.right
```



caso 4: il fratello **w** è nero, e ha il figlio destro rosso.

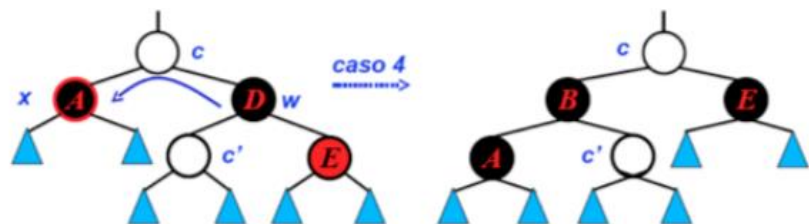
Trasferiamo a **w** il colore del padre, che successivamente diventa nero così come il figlio destro di **w**.

Rimuoviamo il nero extra da **x**.

Da questo caso, il ciclo *termina* perché **x** diventa **T.root**.

In sostanza, avremo:

```
w.color = x.p.color
x.p.color = BLACK
w.right.color = BLACK
LEFT_ROTATE(T,x.p)
x = T.root //end cicle
```



2. Se il padre di **X** è il figlio destro del nonno:

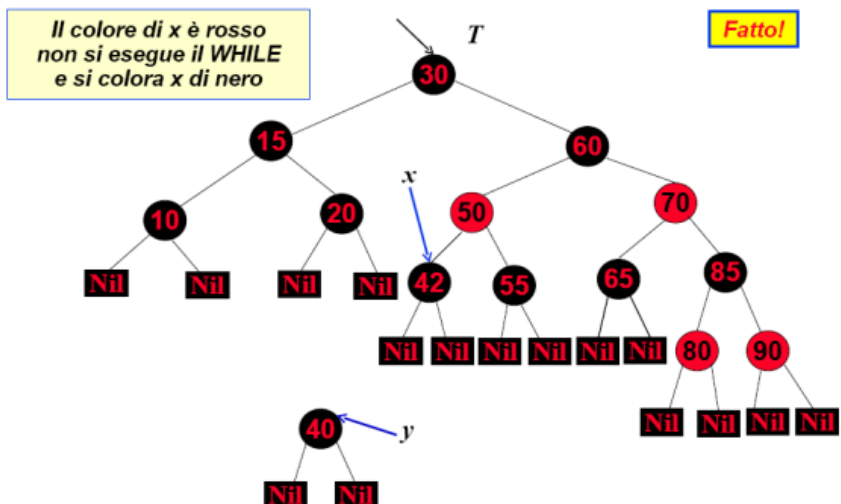
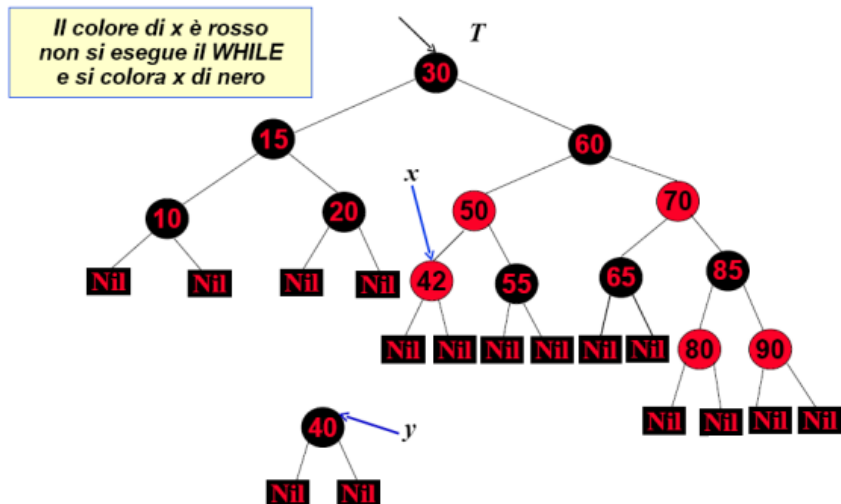
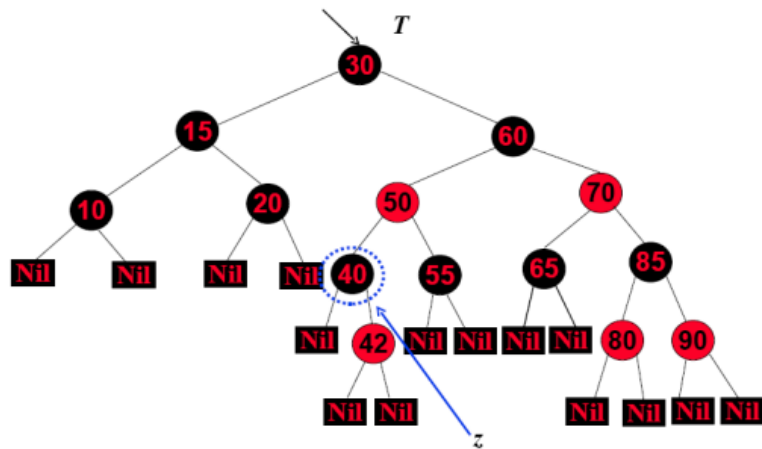
Si trattano degli stessi casi, ma simmetrici (destro e sinistro sono scambiati).

PSEUDO-CODICE

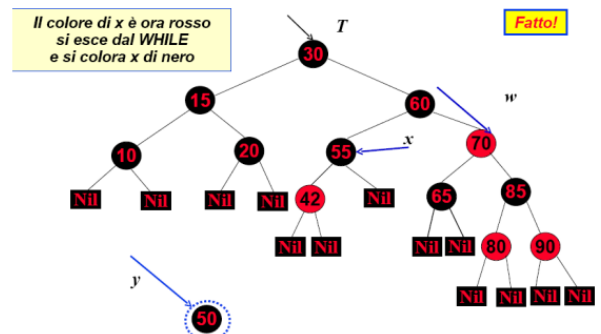
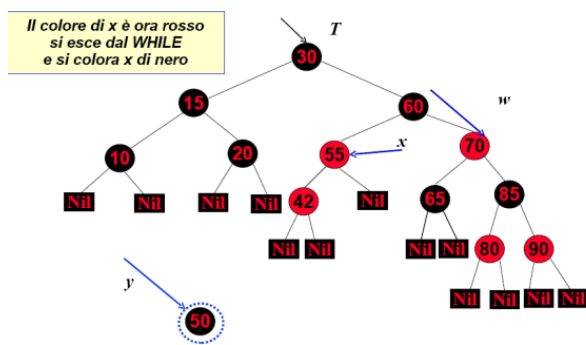
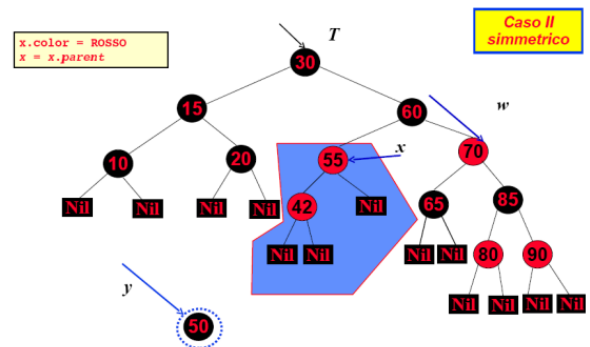
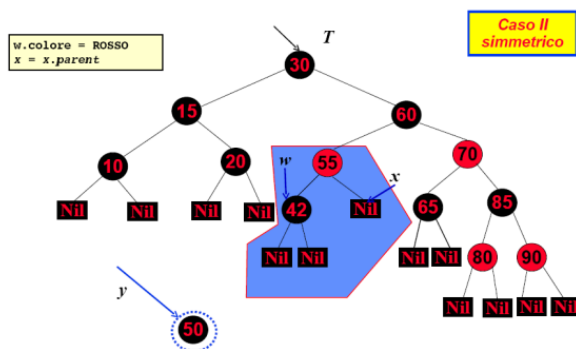
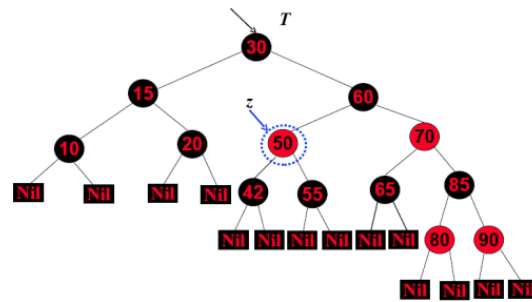
RB_DELETE_FIXUP(T,x)

```
1.  while x ≠ T.root and x.color ≠ RED
2.      if x = x.p.left      //x è il figlio sx
3.          w = x.p.right    //dunque w è il figlio dx
4.          if w.color = RED    //caso 1
5.              w.color = BLACK
6.              x.p.color = RED
7.              LEFT_ROTATE(T,x.p)
8.              w = x.p.right
9.          if w.left.color = BLACK and w.right.color = BLACK    //caso 2
10.             w.color = RED
11.             x = x.p
12.          else
13.              if w.right.color = BLACK    //caso 3
14.                  w.left.color = BLACK
15.                  w.color = RED
16.                  RIGHT_ROTATE(T,w)
17.                  w = x.p.right
18.              w.color = x.p.color    //caso 4
19.              x.p.color = BLACK
20.              w.right.color = BLACK
21.              LEFT_ROTATE(T,x.p)
22.              x = T.root
23.      else    //x è il figlio dx. TUTTI I CASI SIMMETRICI
24.          w = x.p.left    //dunque w è il figlio sx
25.          if w.color = RED
26.              w.color = BLACK
27.              x.p.color = RED
28.              RIGHT_ROTATE(T,x.p)
29.              w = x.p.left
30.          if w.left.color = BLACK and w.right.color = BLACK
31.              w.color = RED
32.              x = x.p
33.          else
34.              if w.left.color = BLACK
35.                  w.right.color = BLACK
36.                  w.color = RED
37.                  LEFT_ROTATE(T,w)
38.                  w = x.p.left
39.              w.color = x.p.color
40.              x.p.color = BLACK
41.              w.left.color = BLACK
42.              RIGHT_ROTATE(T,x.p)
43.              x = T.root
44.  x.color = BLACK
```

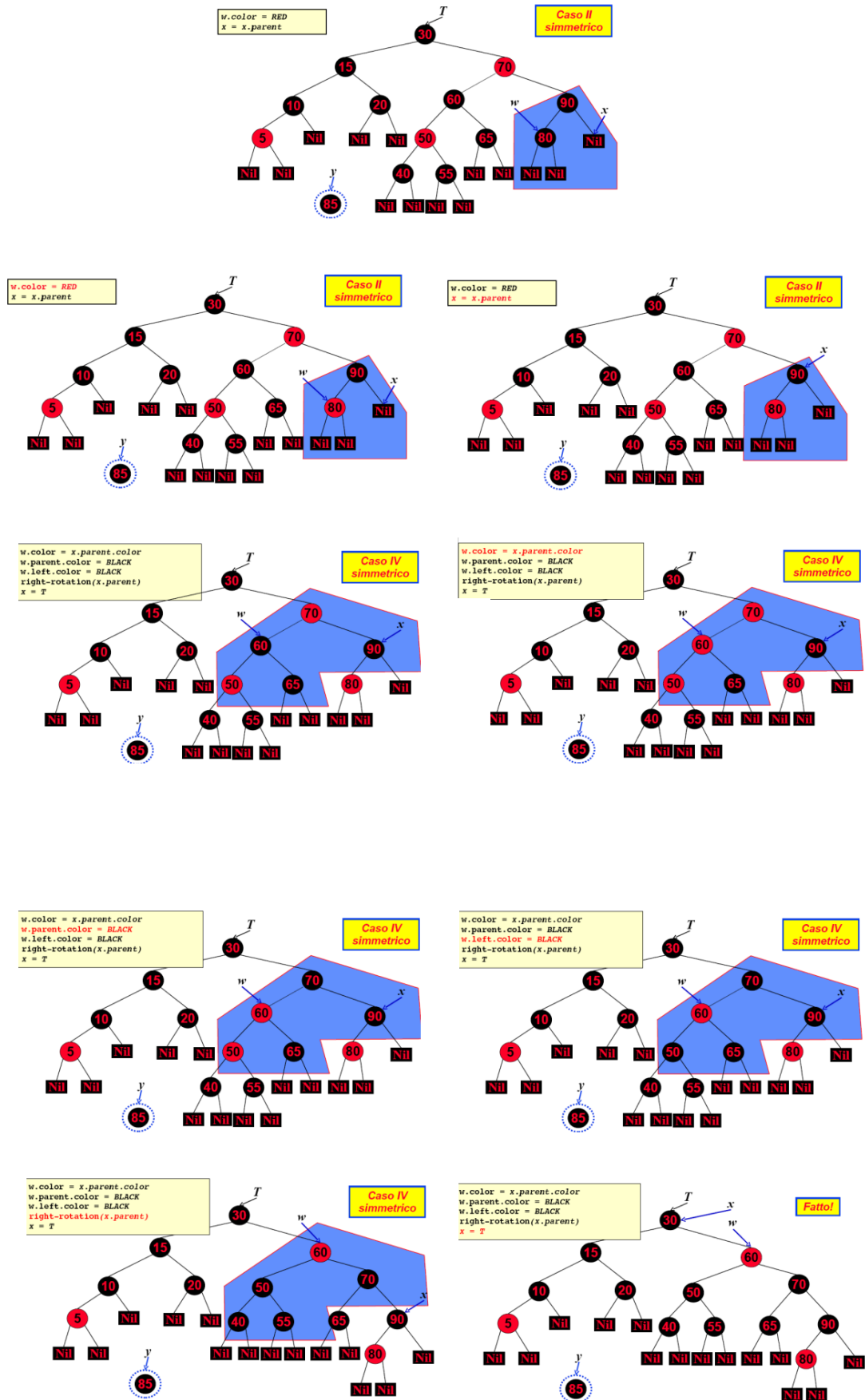
ESEMPIO 1



ESEMPIO 2



ESEMPIO 3



LEZ 15 – PD (FIBONACCI e PROBLEMA CATENA DI MONTAGGIO)

FIBONACCI in C++

```
//RICORSIVO

int fibo_ric(int i){
    return (i>1? fibo_ric(i-1)+fibo_ric(i-2) : 1);
}

//PD CON TABULAZIONE

int fibo_tab(int i){
    int KnFibonacci[i+2]{0};           //contiene almeno fibo(0) e fibo(1)
    KnFibonacci[0] = KnFibonacci[1] = 1;
    for(int j=2; j<=i; j++)
        KnFibonacci[j] = KnFibonacci[j-1] + KnFibonacci[j-2];
    return KnFibonacci[i];
}

//PD CON MEMOIZZAZIONE

const int MAXNUM = 256;
int KnFibo[MAXNUM];
int fibo_mem(int i){
    for(int j=0; j<=i; j++)
        KnFibo[j] = 0;
    return realFibo(i);
}

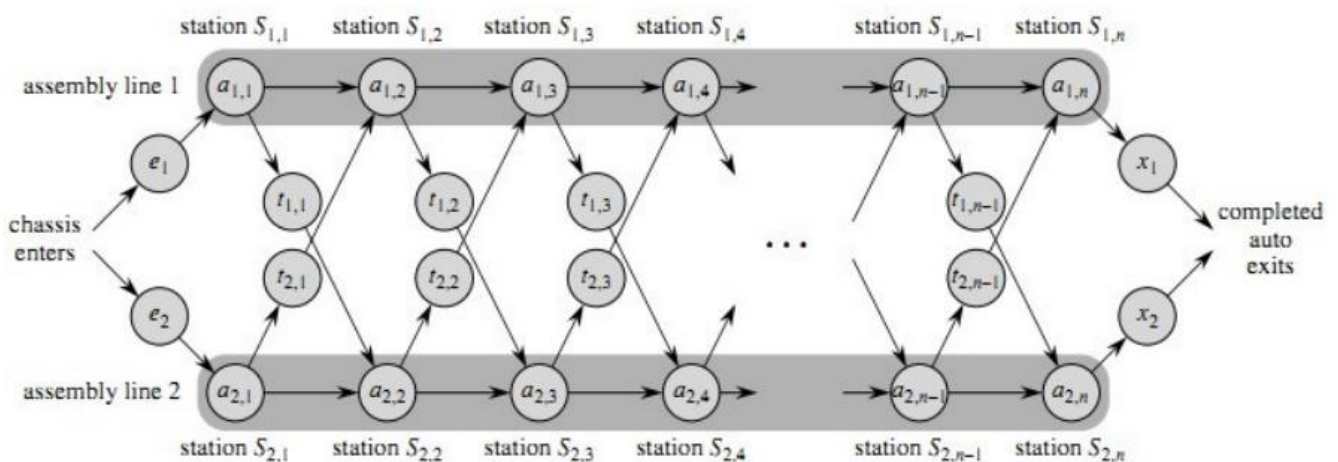
int realFibo(int i){
    if(KnFibo[i] != 0)
        return KnFibo[i];
    if(i==0 or i==1)
        KnFibo[i] = 1;
    else
        KnFibo[i] = realFibo(i-1) + realFibo(i-2);
    return KnFibo[i];
}
```

PROBLEMA DELLA CATENA DI MONTAGGIO

Un'azienda di automobili produce automobili in uno stabilimento con due catene di montaggio. Il telaio di un'automobile entra in una catena di montaggio, riceve nuovi componenti in un certo numero di stazioni, e, infine, l'auto completa esce dalla catena di montaggio.

Ogni catena di montaggio ha n stazioni;

- $S_{i,j}$ è la j -esima stazione della catena i -esima, ove $i = 1, 2$ e $j = 1, \dots, n$.
Da notar bene che $S_{1,j}$ svolge la stessa funzione di $S_{2,j}$, tuttavia possiamo immaginare che le due catene siano state costruite in tempi diversi e con tecnologie diverse, e per questo motivo il tempo richiesto per ciascuna stazione varia a seconda della catena;
- $a_{i,j}$ è il tempo di montaggio richiesto nella stazione;
- e_i è il tempo di ingresso nella catena i -esima;
- x_i è il tempo di uscita della catena i -esima;
- $t_{i,j}$ è il tempo per trasferire il telaio da una catena di montaggio i dopo aver attraversato $S_{i,j}$.



Assumendo che il tempo per passare dalla stazione $S_{i,j-1}$ alla stazione $S_{i,j}$ sia trascurabile, il **PROBLEMA** è determinare quali stazioni scegliere dalla catena 1 e dalla catena 2 per minimizzare il tempo totale per completare l'assemblaggio dell'auto.

Da notar bene che **NON** è possibile applicare la tecnica della forza bruta quando ci sono molte stazioni. Infatti, elencando tutti i modi possibili di scegliere le stazioni e calcolando il tempo richiesto da ciascuno di essi, occorrerebbe un **tempo** $\Omega(2^n)$, che è inaccettabile per n grande.

CARATTERIZZAZIONE DELLA SOTTOSTRUTTURA OTTIMA

Se $j=1$ c'è un solo percorso che il telaio può seguire per arrivare alla stazione $S_{1,j}$.

Per $j=2,3,\dots,n$ possono presentarsi due situazioni:

- Il telaio può provenire dalla stazione $S_{1,j-1}$;
- Il telaio può provenire dalla stazione $S_{2,j-1}$, con tempo di trasferimento $t_{2,j-1}$ (in quanto proviene dall'altra catena).

Da notare che se il **percorso più rapido** per arrivare alla stazione $S_{1,j}$ passa per la stazione $S_{1,j-1}$, allora il telaio deve aver seguito quel percorso per raggiungere $S_{1,j-1}$.

Se invece fosse esistito un percorso più veloce, avremmo scelto quello per arrivare a $S_{i,j}$, ma è una **contraddizione**.

Possiamo applicare lo stesso ragionamento anche alla seconda catena di montaggio, pertanto la **soluzione ottima** contiene la soluzione ottima ai suoi sottoproblemi (trovare il percorso più veloce per raggiungere $S_{1,j-1}$ e $S_{2,j-1}$).

Il **percorso più veloce per raggiungere $S_{1,j}$** può essere:

- Il percorso più veloce per raggiungere $S_{1,j-1}$;
- Il percorso più veloce per raggiungere $S_{2,j-1}$, a cui aggiungere il tempo di trasferimento dalla catena 2 alla catena 1 ($t_{2,j-1}$).

Il **percorso più veloce per raggiungere $S_{2,j}$** può essere:

- Il percorso più veloce per raggiungere $S_{2,j-1}$;
- Il percorso più veloce per raggiungere $S_{1,j-1}$, a cui aggiungere il tempo di trasferimento dalla catena 1 alla catena 2 ($t_{1,j-1}$).

Da queste affermazioni possiamo ricavare la **SOLUZIONE RICORSIVA**:

L'obiettivo è determinare il tempo minimo che impiega un telaio per attraversare lo stabilimento; indichiamo con f^* questo tempo.

Il telaio deve raggiungere la stazione n dalla catena 1 o 2 e poi uscire dallo stabilimento, quindi:

$$- f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

Per calcolare $f_1[j]$ e $f_2[j]$ dobbiamo considerare due casi:

- se $j = 1$, $f_i[1]$ è il minor tempo possibile che impiega un telaio dal punto iniziale fino all'uscita della stazione;

$$f_1[1] = e_1 + a_{1,1}$$

$$f_2[1] = e_2 + a_{2,1}$$

- se $j > 1$, avremo le seguenti equazioni di ricorrenza per una generica stazione.

$$- f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$$

$$- f_2[j] = \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j})$$

Quindi la **soluzione ricorsiva** sarà:

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{se } j = 1 \\ \min(f_1[j-1], f_2[j-1] + t_{2,j-1}) + a_{1,j} & \text{se } j > 1 \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{se } j = 1 \\ \min(f_2[j-1], f_1[j-1] + t_{1,j-1}) + a_{2,j} & \text{se } j > 1 \end{cases}$$

PSEUDOCODICE

FASTEST-WAY(a, t, e, x, n)

$f_1[1] \leftarrow e_1 + a_{1,1}$

$f_2[1] \leftarrow e_2 + a_{2,1}$

for $j \leftarrow 2$ **to** n

do if $f_1[j-1] \leq f_2[j-1] + t_{2,j-1}$

then $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$

$l_1[j] \leftarrow 1$

else $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$

$l_1[j] \leftarrow 2$

do if $f_2[j-1] \leq f_1[j-1] + t_{1,j-1}$

then $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$

$l_2[j] \leftarrow 2$

else $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$

$l_2[j] \leftarrow 1$

if $f_1[n] + x_1 \leq f_2[n] + x_2$

then $f^* \leftarrow f_1[n] + x_1$

$l^* \leftarrow 1$

else $f^* \leftarrow f_2[n] + x_2$

$l^* \leftarrow 2$

Calcolo delle soluzioni ottime

Calcolo della soluzione ottima per la catena 1

Calcolo della soluzione ottima per la catena 2

Gli input di FASTEST-WAY sono gli stessi che abbiamo detto all'inizio:

a = tempo di montaggio; t = tempo di trasferimento; e = tempo di ingresso;
 x = tempo di uscita; n = numero di stazioni.

Il ragionamento di base è calcolarmi passo per passo le varie soluzioni, tali soluzioni saranno conservate in due array: f_1 e f_2 ; inoltre ci serviremo di due ulteriori array di supporto l_1 e l_2 per mostrare le soluzioni ottime, ovvero per mostrare il percorso.

Il primo passo consisterà assegnare ad $f_1[1]$ la soluzione $e_1 + a_{1,1}$, e ad $f_2[1]$ la soluzione $e_2 + a_{2,1}$.

Poi, per ogni j successivo, dobbiamo calcolare la soluzione ottima sia per la catena 1 che per la catena 2.

Nella riga 4 siamo su una certa stazione della catena 1, che non sia la prima, e verifichiamo quale stazione, fra quella precedente sulla stessa catena e quella precedente sulla catena 2 (a cui aggiungiamo anche t), richiede meno tempo (minimo).

Se si verifica il caso 1, allora inseriremo nell'array delle soluzioni $f_1[j]$ il "valore" della stazione precedente della catena 1 + il tempo di montaggio " a " della stazione attuale. Inoltre, nell'array $l_1[j]$ inserirò il valore '1', perché siamo sulla prima catena.

Altrimenti, inseriremo nell'array delle soluzioni $f_1[j]$ il "valore" della stazione precedente della catena 2 + il tempo di trasferimento + il tempo di montaggio della stazione attuale. Inoltre, nell'array $l_1[j]$ inserirò il valore '2', perché siamo sulla seconda catena.

Nella riga 9 facciamo lo stesso, ma per la catena 2.

Ripeteremo tale processo per tutte le stazioni. Fatto ciò, dovremo solo calcolare i tempi di uscita, ed andrò a conservare in una variabile f^* il valore della soluzione ottima, ed andrò a segnarmi in una variabile l^* l'ultima scelta che ho fatto.

Se l^* fosse 1, significa che dovremmo percorrere il percorso dall'array l_1 , se invece l^* fosse 2, dovremo farlo dall'array l_2 . Il numero che troviamo all'interno di l ci dice qual è la soluzione ottima al passo precedente.

Quindi, usando l^* e gli array l_1 e l_2 vado a **ricostruire** la soluzione ottima.

Es.

$f_1 = \{ 9, 18, 20, 24, 32, 35 \}$

$f_2 = \{ 12, 16, 22, 25, 30, 37 \}$

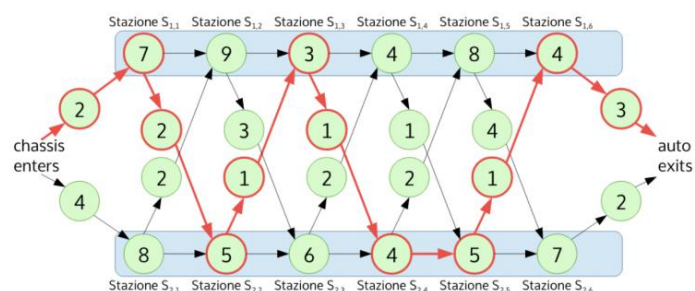
$l_1 = \{ 1, 2, 1, 1, 2 \}$

$l_2 = \{ 1, 2, 1, 2, 2 \}$

$f^* = 38$ ($\min(35+3, 37+2)$)

$l^* = 1$

percorso: 1-2-1-1-2-1



Un'altra soluzione è la seguente: $S_{1,1}-S_{2,2}-S_{1,3}-S_{1,4}-S_{2,5}-S_{1,6}$ oppure 1-2-1-1-2-1 (non quella evidenziata in figura)

FASTEST_WAY – IMPLEMENTAZIONE IN C++

Implementazione fedele allo pseudo-codice:

```
void fastestWay(int a[][MAXN], int t[2][MAXN], int e[2], int x[2], int n){
    int f1[n], f2[n], f;
    bool l1[n-1], l2[n-1], l;

    f1[0] = e[0]+a[0][0];
    f2[0] = e[1]+a[1][0];

    for(int j=1; j<n; j++){

        if(f1[j-1] <= f2[j-1]+t[1][j-1]){           //Catena 1
            f1[j] = f1[j-1]+a[0][j];
            l1[j-1] = 0;
        }
        else{
            f1[j] = f2[j-1]+t[1][j-1]+a[0][j];
            l1[j-1] = 1;
        }

        if(f2[j-1] <= f1[j-1]+t[0][j-1]){           //Catena 2
            f2[j] = f2[j-1]+a[1][j];
            l2[j-1] = 1;
        }
        else{
            f2[j] = f1[j-1]+t[0][j-1]+a[1][j];
            l2[j-1] = 0;
        }

    }

    if( f1[n-1]+x[0] <= f2[n-1]+x[1]){
        f = f1[n-1]+x[0];
        l = 0;
    }
    else{
        f = f2[n-1]+x[1];
        l = 1;
    }

    printFW(l1,l2,l,n);
}
```

nb: ovviamente stiamo considerando array 0-based in questo caso, mentre quelli in pseudocodice erano array 1-based.

È tutto uguale, ma la Catena 1 e la Catena 2 diventano rispettivamente la “Catena 0” e la “Catena 1”.

Implementazione personale sfruttando *tuple*. Ciò mi permette di ritornare 2 valori invece di uno solo, dunque posso direttamente creare una funzione “minimo” che mi restituisce, oltre al minimo, anche se si tratta del primo o del secondo valore passato (0 o 1, ovvero la catena dalla quale otteniamo il minimo).

```
#include <tuple>

tuple<int,int> minTuple(int a, int b){
    return (a<=b? make_tuple(a,0) : make_tuple(b,1));
}

void myFastestWay(int a[][MAXN], int t[2][MAXN], int e[2], int x[2], int n){

    int f1[n], f2[n], f;
    bool l1[n-1], l2[n-1], l;

    f1[0] = e[0]+a[0][0];
    f2[0] = e[1]+a[1][0];

    for(int j=1; j<n; j++){
        tie(f1[j],l1[j-1]) = minTuple(f1[j-1], f2[j-1]+t[1][j-1]); //Catena 1
        f1[j] += a[0][j];

        tie(f2[j],l2[j-1]) = minTuple(f2[j-1], f1[j-1]+t[0][j-1]); //Catena 2
        f2[j] += a[1][j];
    }

    tie(f,l) = minTuple(f1[n-1]+x[0], f2[n-1]+x[1]);

    printFW(l1,l2,l,n);
}
```

La funzione printFW():

```
void printFW(bool l1[], bool l2[], int l, int n){
    if(l==0)
        for(int i=0; i<n-1; i++)
            cout << l1[i] << " - ";
    else
        for(int i=0; i<n-1; i++)
            cout << l2[i] << " - ";
    cout << l << endl;
}
```

Il Main():

```
int main(){

    int n=6;
    int a[2][MAXN] = {
        {7,9,3,4,8,4},
        {8,5,6,4,5,7}
    };
    int t[2][MAXN] = {
        {2,3,1,1,4},
        {2,1,2,2,1}
    };
    int e[2] = {2,4};
    int x[2] = {3,2};

    fastestWay(a,t,e,x,n);
    myFastestWay(a,t,e,x,n);

    return 0;
}
```

LEZ 16 – PD (DISTANZA DI EDITING e ZAINO 01)

DISTANZA DI EDITING (di Levenshtein)

L'equazione di ricorrenza per la distanza di Editing di Levenshtein è:

$$d[i, j] = \begin{cases} i & \text{se } j = 0 \\ j & \text{se } i = 0 \\ \min \begin{cases} d[i, j-1] + 1 \\ d[i-1, j] + 1 \\ d[i-1, j-1] + \delta(X[i], Y[j]) \end{cases} & \text{altrimenti} \end{cases}$$

dove:

- $\delta(X[i], Y[j]) = 0$, se $X[i] = Y[j]$;
- $\delta(X[i], Y[j]) = 1$, se $X[i] \neq Y[j]$.

LevenshteinEditingDistance(X,m,Y,n)

```
1. for i=0 to m
2.   d[i,0] = i
3. for j=1 to n    //la cella d[0,0] è già 0
4.   d[0,j] = j
5. for i=1 to m
6.   for j=1 to n
7.     if X[i] == Y[j]
8.       delta = 1
9.     else
10.      delta = 0
11.      d[i,j] = min(d[i,j-1]+1, d[i-1,j]+1, d[i-1,j-1]+delta)
12. return d[m,n]
```

Esempio

	-	T	G	C	A	T	A
-	0	←1	←2	←3	←4	←5	←6
A	↑1	↖1	←2	↖3	↖3	←4	↖5
T	↑2	↖1	↖2	↖3	↖4	↖3	←4
C	↑3	↑2	↖2	↖2	←3	←4	↖4
T	↑4	↖3	↖3	↖3	↖3	↖3	←4
G	↑5	↑4	↖3	↖4	↖4	↖4	↖4
A	↑6	↑5	↑4	↖4	↖4	↖5	↖4
T	↑7	↖6	↑5	↖5	↖5	↖4	↑5

Codice in C++

```
int LED(string X, int m, string Y, int n){
    vector<vector<int>> d(m+1, vector<int>(n+1)); //crea matrice d MxN
    for(int i=0; i<=m; i++)    //caso base j=0
        d[i][0] = i;
    for(int j=0; j<=n; j++)    //caso base i=0
        d[0][j] = j;

    for(int i=1; i<=m; i++){
        for(int j=1; j<=n; j++){
            int delta = (X[i-1] == Y[j-1])? 0 : 1;
            d[i][j] = min(d[i][j-1]+1, d[i-1][j]+1,d[i-1][j-1]+delta);
        }
    }
    //printVecMatrix(d);
    return d[m][n];
}
```


ZAINO 01

La programmazione dinamica può risolvere il problema dello zaino 0-1 quando sia i pesi che i valori degli oggetti sono interi, perché se non fossero interi lo spazio dei problemi non sarebbe più polinomiale, e quindi anche se ha la sottostruttura ottima, non può essere risolto con la PD in quanto è improbabile che possa trovare problemi ripetuti (e quindi diventa esponenziale).

Numero dei sottoproblemi

Abbiamo **Wn** sottoproblemi della forma $S(i,w)$, dove $1 \leq i \leq n$ e $1 \leq w \leq W$.

Ricordando che il problema originale è $S(n,W)$, dai casi banali avremo:

- **W** sottoproblemi della forma $S(0,w)$;
- **n** sottoproblemi della forma $S(i,0)$.

In totale, avremo: $Wn + W + n = O(Wn)$ sottoproblemi.

Equazione di ricorrenza (calcolo del valore della soluzione ottima)

Sia $v(i,w)$ il valore della soluzione ottima per il sottoproblema $S(i,w)$, dove $i = 0, \dots, n$ e $w = 0, \dots, W$.

Nei casi banali $i=0$ o $w=0$ avremo che:

$$v(i,0) = v(0,w) = 0.$$

Nei casi non banali:

$$V(i,w) = \begin{cases} \max\{V(i-1,w), V(i-1,w-w_i) + v_i\} & \text{se } w_i \leq w \\ V(i-1,w) & \text{altrimenti} \end{cases}$$

Pseudocodice

Knapsack_01(w,v,W,n)

```
1. for w=0 to W
2.   V[0,w] = 0
3. for i=1 to n
4.   V[i,0] = 0
5. for i=1 to n
6.   for w=1 to W
7.     if w[i] ≤ w
8.       V[i,w] = max(V[i-1,w], V[i-1, w-w[i]]+v[i])
9.     else
10.      V[i,w] = V[i-1, w]
1. Return V
```

Codice in C++

nb: W e V sono gli array “peso” e “valore”. TOT è la capienza totale dello zaino. n è il numero di elementi. S sarà la tabella di appoggio.

(ho preferito cambiare la nomenclatura perché quella di prima era ambigua).

La ricorrenza sarà dunque (comprendendo anche i casi banali):

$$S[i, w] = \begin{cases} 0 & \text{se } i = 0 \text{ o } w = 0 \\ \max(S[i-1, w], S[i-1, w - W[i]] + V[i]) & \text{se } W[i] \leq w \\ S[i-1, w] & \text{altrimenti} \end{cases}$$

nb: in C++ gli array sono 0-based, per questo consideriamo a volte (i-1) invece di (i).

```
int** createMatrix(int m, int n){
    int** M = new int*[m];
    for(int i=0; i<m; i++)
        M[i] = new int[n];
    return M;
}

void deleteMatrix(int** M, int m){
    for(int i=0; i<m; i++)
        delete[] M[i];
    delete[] M;
}

int knapsack01(int V[], int W[], int n, int TOT){
    int** S = createMatrix(n+1, TOT+1); //sovradim di 1

    for(int i=0; i<=n; i++)
        S[i][0] = 0;
    for(int w=1; w<=TOT; w++)
        S[0][w] = 0;

    for(int i=1; i<=n; i++)
        for(int w=1; w<=TOT; w++)
            if(W[i-1] <= w)
                S[i][w] = max(S[i-1][w], S[i-1][w-W[i-1]]+V[i-1]);
            else
                S[i][w] = S[i-1][w];

    int res = S[n][TOT];
    printMatrix(S, n+1, TOT+1);
    deleteMatrix(S, n);
    return res;
}
```

Esempio:

i	w	v
1	2	3
2	3	4
3	4	5
4	5	6

iw	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

...dal codice:

```
int W[] = {2,3,4,5};      0      0      0      0      0      0
int V[] = {3,4,5,6};      0      0      3      3      3      3
int n = 4;                0      0      3      4      4      7
int TOT = 5;              0      0      3      4      5      7
```

Valore zaino 01: 7

LEZ 17 – ALGORITMI GREEDY (GAS e CODICI DI HUFFMAN)

GREEDY ACTIVITY SELECTOR (GAS) (Ripetizione Camastra)

Il *problema della selezione di attività* consiste nell'individuazione di un sottoinsieme A che contiene il massimo numero di attività mutuamente compatibili.

La **Greedy Activity Selector** è una procedura che suppone di avere le attività ordinate in base al loro tempo di fine, per poi scegliere la prima attività a_1 . A questo punto è possibile eliminare tutte le attività che intersecano con a_1 , cioè quelle che iniziano prima che a_1 finisca. Si ripetono questi passaggi iterativamente anche per le restanti attività.

Greedy_Activity_Selector($s[1..n]$, $f[1..n]$)

1. $n \leftarrow s.length$
2. $A \leftarrow \{a_1\}$
3. $k \leftarrow 1$
4. **for** $m \leftarrow 2$ **to** n
5. **if** $s[m] \geq f[k]$
6. $A \leftarrow A \cup \{a_m\}$
7. $k \leftarrow m$
8. **return** A

Ad ogni passo, l'algoritmo inserisce in A la prima attività compatibile con l'attività appena terminata, cioè quella localmente ottima. In pratica, l'algoritmo seleziona l'attività a_1 , poi la prima attività compatibile con a_1 , poi la prima attività compatibile con quella appena conclusa, e così via.

nb: il GAS produce un sottoinsieme A che contiene il massimo numero di attività mutuamente compatibili, questo significa che non è detto che conterrà tutte le attività, e anzi è improbabile che accada.

Es.

i	s_i	f_i
1	1	4
2	3	5
3	0	6
4	5	7
5	3	9

Se seguiamo l'algoritmo, ci renderemo conto come l'insieme S sarà costituito solo da $\{a_1, a_4\}$, che rappresenta appunto il massimo numero di attività mutuamente compatibili. Le altre finirebbero per sovrapporsi.

Codice in C++

Utilizzo una **classe Activity** nella quale definisco un campo “s” (start) e “f” (finish) per ogni attività. Per gestire facilmente gli insiemi utilizzo dei vector, e per l’ordinamento utilizzo la funzione predefinita “sort()” andando a specificare come *intervallo di confronto* il tempo di fine.

```
class Activity{
public:
    int s; //start time
    int f; //finish time

    Activity(int s, int f){
        this->s = s;
        this->f = f;
    }
};
```

```
bool finishTimeCompare(Activity a, Activity b){
    return (a.f < b.f);
}

vector<Activity> GreedyActivitySelector(vector<Activity> A){
    sort(A.begin(), A.end(), finishTimeCompare); //ordina risp. tempo fine
    vector<Activity> S;
    S.push_back(A[0]);
    int j = 0;
    for(int i=1; i<A.size(); i++){
        if(A[i].s >= A[j].f){
            S.push_back(A[i]);
            j = i;
        }
    }
    return S;
}
```

```
vector<Activity> A {
    Activity(5,7),
    Activity(3,9),
    Activity(0,6),
    Activity(1,4),
    Activity(3,5)
};
```

Activity 1:	s: 5	f: 7	
Activity 2:	s: 3	f: 9	
Activity 3:	s: 0	f: 6	A, prima
Activity 4:	s: 1	f: 4	
Activity 5:	s: 3	f: 5	

Activity 1:	s: 1	f: 4	
Activity 2:	s: 3	f: 5	A, dopo
Activity 3:	s: 0	f: 6	
Activity 4:	s: 5	f: 7	
Activity 5:	s: 3	f: 9	

attivit� mutamente compatibili		
Activity 1:	s: 1	f: 4
Activity 2:	s: 5	f: 7

nb: non siamo interessati ad ordinare le attivit  in modo permanente, per questo motivo passiamo il vettore ‘A’ per valore, e non per riferimento. In questo modo, il vettore A rimarr  invariato.

Se volessimo modificare anche il vettore originale, dovremmo passarlo per riferimento:

```
vector<Activity> GreedyActivitySelector(vector<Activity> &A){
```

CODICI DI HUFFMAN

L'algoritmo di Huffman è un *algoritmo greedy*, ed è un metodo usato per la compressione dei dati in quanto consente, a seconda delle caratteristiche del file che si vuole comprimere, un risparmio di spazio che va dal 20% al 90%

Per la compressione di stringhe, Huffman utilizza un **codice a lunghezza variabile**, dove si assegna ai caratteri più frequenti una sequenza binaria più corta, ai caratteri meno frequenti una sequenza binaria più lunga. La particolarità di queste stringhe binarie è che nessuna di esse risulta essere prefisso di un'altra stringa binaria. Huffman è, quindi, un **codice prefisso**.

I codici prefisso possono essere rappresentati mediante alberi binari, dove:

- le foglie sono i caratteri da codificare;
- gli archi che scendono a sinistra hanno etichetta 0;
- gli archi che scendono a destra hanno etichetta 1;
- i percorsi radice-foglia corrispondono alla sequenza codificata;

I caratteri più frequenti si troveranno in alto nell'albero (avranno una minore profondità, meno bit).

I caratteri meno frequenti si troveranno in basso nell'albero (avranno una maggiore profondità, molti bit).

La profondità di una foglia indica la lunghezza della stringa binaria utilizzata per codificare il carattere associato alla foglia.

Per decodificare un carattere basta partire dalla foglia associata a quel carattere e risalire fino alla radice.

ESEMPIO

Con un codice a *lunghezza fissa*, per rappresentare 5 caratteri distinti abbiamo bisogno di 3 bit. Infatti E=(000), F=(001), G=(010), H=(011), I=(100). Per 100.000 caratteri, quindi si ha bisogno di 300.000 bit.

Con un codice prefisso, invece, notiamo che la memoria richiesta è *inferiore*, ed è di 210.000 bit.

1. E: $100.000 * 0.40 * 1\text{bit} = 40.000 \text{ bit}$
2. F: $100.000 * 0.05 * 4\text{bit} = 20.000 \text{ bit}$
3. G: $100.000 * 0.25 * 2\text{bit} = 50.000 \text{ bit}$
4. H: $100.000 * 0.10 * 4\text{bit} = 40.000 \text{ bit}$
5. I: $100.000 * 0.20 * 3\text{bit} = 60.000 \text{ bit}$

= 210.000 bit

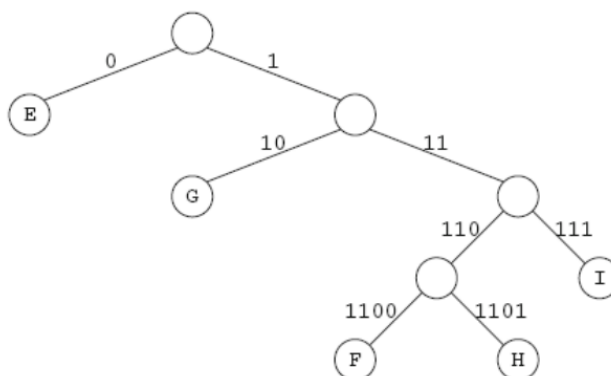


Table 3.1 Frequency of each character in the file

	E	F	G	H	I
Frequency	40 %	5 %	25 %	10 %	20 %
Fix length code	000	001	010	011	100
Huffman code	0	1100	10	1101	111

Costruzione di un codice di Huffman

1. $\text{Huffman}(C)$
2. $n \leftarrow |C|$
3. $Q \leftarrow C$
4. **for** $i \leftarrow 1$ **to** $n - 1$
5. $z \leftarrow \text{ALLOCATE_NODE}()$
6. $x \leftarrow \text{LEFT}[z] \leftarrow \text{EXTRACT_MIN}(Q)$
7. $y \leftarrow \text{RIGHT}[z] \leftarrow \text{EXTRACT_MIN}(Q)$
8. $f[z] \leftarrow f[x] + f[y]$
9. $\text{INSERT}(Q, z)$
10. **return** $\text{EXTRACT_MIN}(Q)$

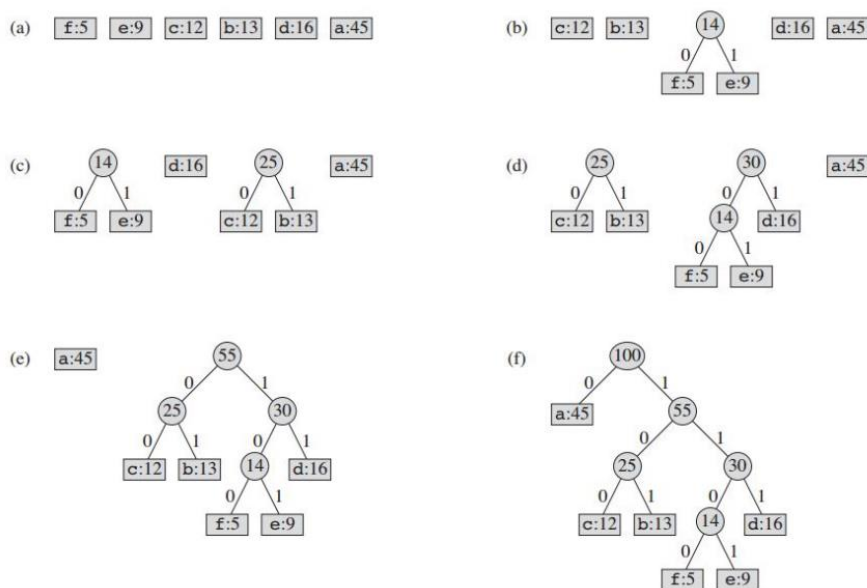
nb: f = frequenza

C rappresenta l'alfabeto (array di caratteri) e n la sua lunghezza. Presupponiamo di inserire tali simboli in una *coda di priorità di minimo*, utilizzando come chiave la *frequenza*. Ad ogni passo creo un nodo, estraggo il minimo e lo metto come figlio sinistro del nodo, poi estraggo di nuovo il minimo e lo metto come figlio destro. Il valore del nodo sarà pari alla somma dei due figli. A questo punto inserisco il nodo nella coda perché per rivalutare la nuova fusione dei due minimi devo considerare anche i nodi. Ripeto tutto finché non sono finiti tutti i caratteri.

Per l'estrazione dei minimi impiego un tempo $\log n$. Il numero dei caratteri è n . Ergo, la complessità dell'algoritmo è $O(n \log n)$.

nb: Sebbene il padre sia sempre maggiore dei nodi figli, *non* si tratta di un heap, in quanto non è completo e non abbiamo il vincolo per cui tutti i nodi terminali siano compresi fra l'ultimo e il penultimo livello.

Esempio:



In C++

Innanzitutto dobbiamo definire la struttura di un **nodo di Huffman**.

Tale nodo conterrà un carattere “ch” e la frequenza di tale carattere “freq”. Conterrà inoltre un puntatore a un figlio sinistro e a un figlio destro (che verranno assegnati durante l’esecuzione dell’algoritmo).

Un nodo di Huffman vuoto è un nodo con carattere “\0” (carattere di escape) e frequenza “freq” = 0.

```
class Node{
public:
    char ch;
    int freq;
    Node* left;
    Node* right;
    Node(char ch, int freq):ch{ch},freq{freq}{
        left = right = nullptr;
    }
    Node():Node('\0',0){}
};
```

codificatore di Huffman

L’alfabeto è rappresentato da una map “carattere/frequenza” (che avremo popolato prima, ad esempio leggendo le occorrenze di ogni carattere all’interno di un file).

L’insidia più grande della conversione da pseudo-codice a C++ è rappresentato dalla coda. Come **coda** possiamo sfruttare la struttura dati stl *priority_queue*, specificando che deve trattarsi di una **coda di puntatori a Node**.

Di default, una *priority_queue* è una coda di priorità di massimo, ma noi vogliamo che sia una **coda di priorità di minimo**. Normalmente basterebbe una sintassi del tipo:

```
std::priority_queue<int, std::vector<int>, greater<int>> Q;
```

dove “greater<int>” è una struttura di confronto standard.

Tuttavia noi non siamo interessati ad interi, ma a “puntatori a Node”, utilizzando come chiave la frequenza dei caratteri. Per questo motivo dobbiamo definire una nostra struttura personale:

```
struct nodeCmp{
    bool operator()(const Node* u, const Node* v) const {
        return u->freq > v->freq;    // > for minQ, < for maxQ
    }
};
```

E la *priority_queue* si presenterà dunque come:

```
std::priority_queue<Node*, std::vector<Node*>, nodeCmp> Q;
```


nb: per come sono implementate le `priority_queue` stl in C++, se vogliamo definire una `priority_queue` “di minimo” dobbiamo usare il segno “>”, per una `priority_queue` “di massimo” dobbiamo usare il segno “<” (dobbiamo ragionare al contrario).
 È anche per questo che nel caso di prima abbiamo considerato la struttura “`greater<int>`” e non qualcosa come “`smaller<int>`”.

Definita la coda, la dobbiamo **popolare** inserendo dei nodi che andremo a creare al momento basandoci sull’alfabeto passato prima.

```
for(auto& c:C)
    Q.push(new Node(c.first,c.second));
```

Dove `c.first` rappresenta il carattere, e `c.second` la frequenza.

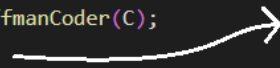
Il resto del codice è equivalente allo pseudo-codice. Infine restituiremo la radice all’albero di Huffman creato.

Il **codice completo** è il seguente:

```
Node* huffmanCoder(std::map<char,int> C){
    int n = C.size();
    std::priority_queue<Node*, std::vector<Node*>, nodeCmp> Q;
    for(auto& c:C)
        Q.push(new Node(c.first,c.second));    //create nodes from map and push in Q

    for(int i=0; i<n-1; i++){
        Node* z = new Node;
        Node* x = z->left = Q.top(); Q.pop();
        Node* y = z->right = Q.top(); Q.pop();
        z->freq = x->freq + y->freq;
        Q.push(z);
    }
    return Q.top();
}
```

risultato:

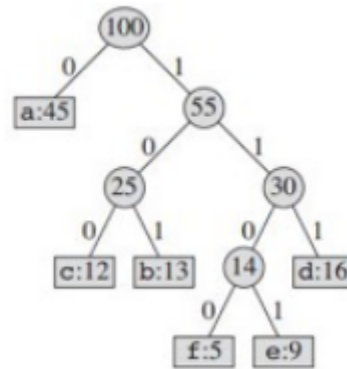
<pre>int main(){ map<char,int> C; C['E'] = 40; C['F'] = 5; C['G'] = 25; C['H'] = 10; C['I'] = 20; Node* huffroot = huffmanCoder(C); printTree(huffroot); return 0; }</pre>		<pre>(:100) +-- (:60) +-- (:35) +-- (I:20) L-- (:15) +-- (H:10) L-- (F:5) L-- (G:25) L-- (E:40)</pre>
--	---	---

Decodifica di un codice di Huffman

I codici di Huffman sono **codici prefissi**: nessuna parola del codice è un prefisso di qualche altra parola dello stesso codice.

Una semplice decodifica consiste in: partire dalla radice e seguire gli archi in base al valore dei biti (0 = sx, 1 = dx) fino alla foglia (dove si trova la lettera), poi si ricomincia dalla radice con il prossimo bit.

- 1011101110011000
- 1->dx, 0->sx, 1->dx = **b**
- 1011101110011000
- 1->dx, 1->dx, 0->sx, 1->dx = **e**
- 1011101110011000
- ...



Da un punto di vista implementativo, dobbiamo scorrere sul testo codificato, che altro non sarà che un insieme di bit. Scorriamo bit per bit, valutando di volta in volta (in base al suo valore) se dobbiamo scendere a sinistra o a destra, che si traduce nell'aggiornare di volta in volta il nodo radice (o meglio, una sua copia) con il figlio sinistro o destro.

Se non ha figli, significa che quel nodo è una foglia, ovvero aggiungiamo quel carattere all'insieme dei caratteri codificati.

decodeHuffman(root, binText)

```
1. text = ""
2. pnode = root
3. for i=1 to binText.size
4.     if binText[i] = 0
5.         pnode = pnode.left
6.     else
7.         pnode = pnode.right
8.     if pnode.left = NIL and pnode.right = NIL
9.         text = text + pnode.data
10.        pnode = root
11. return text
```

nb: se consideriamo i bit come stringhe di 0 e 1, l'implementazione è banale. Tuttavia se vogliamo considerare i singoli bit dobbiamo tener conto che non è possibile operare direttamente su questi in quanto l'unità minima su cui si può operare è il byte.

Un'idea è considerare buffer da 1 byte, e modificare i bit di tale buffer attraverso delle operazioni bitwise.

LEZ 18 – GRAFI (BFS, DFS, ORD. TOPOLOGICO, COMP. FORT. CONN)

Ripetizione di tutta la lezione 15 e lezione 16 (fino a componenti fortemente connesse)

Per l'implementazione in C++, utilizzeremo un'unica classe "Node" che andrà bene sia per la BFS che per la DFS, e utilizzeremo due classi "directedGraph" e "undirectedGraph", che ereditano da una classe astratta Graph.

Notare che un grafo non orientato non è altro che un grafo orientato ma dove ogni coppia di nodi si punta a vicenda.

nb: siccome potremmo passare sia un grafo orientato che non orientato per alcune funzioni, nella funzione richiederemo di passare un "Graph" (che è una classe astratta, dunque in realtà possiamo passare o il Directed o l'Undirected), solo che l'intestazione sarà del tipo `func(Graph* G)`, mentre la chiamata a funzione sarà `func(&myDirG)`.

Classe Node e Classe Edge

```
enum vertexesColor{ WHITE, GRAY, BLACK };
enum edgeType{ NDEFINED, TREE_EDGE, BACK_EDGE, FORWARD_EDGE, CROSS_EDGE };

class Node{
public:
    int key;           //unique identifier
    int rank;          //used for some algs
    std::string label; //node name
    short color;        //WHITE, GRAY, BLACK
    int d;              //distance (or initial distance for some algs)
    int f;              //final distance, used for some algs
    Node* p;            //predecessor, used for some algs
    std::vector<Edge*> adj; //set of edges linking to other nodes
    Node(int key, std::string label) : key{key}, label{label}{}
};

class Edge{
public:
    Node* u;           //node from which the edge start
    Node* v;           //neighbour reached from this edge
    int weight;         //used for some algs
    short type;         //NDEFINED, TREE_EDGE...
    Edge(Node* u, Node* v, int weight = 0, short type = NDEFINED)
        : u{u}, v{v}, weight{weight}, type{type}{}
};
```

nb: "rank" e "weight" sono attributi che ci serviranno per gli algoritmi dei MST.

Classi dei Grafi

Classe astratta Graph

```
class Graph{
public:
    std::vector<Node*> V; //Vertixes, vector to Node pointers
    Node* addNode(std::string label = ""); //assign def labels if not specified
    virtual void addEdge(int ukey, int vkey, int weight = 0) = 0;
    void graphUnion(Edge* uv);
    std::vector<Edge*> getEdges();
    void printInfo();
    void printExtendedInfo();
};

// -- GRAPH implementation

Node* Graph::addNode(std::string label){
    std::string newlabel(1,0); // 1 char from a onwards if label is not spec
    if(label == "") //if not specified...
        newlabel[0] = 97+V.size(); //label will be 1 char from a onwards (97 = a)
    else
        newlabel = label; //else, assign specified label
    V.push_back(new Node(V.size(), newlabel)); //key is unique
    return V.back(); //return added node
}

std::vector<Edge*> Graph::getEdges(){
    std::vector<Edge*> edges;
    for(auto& u:V)
        for(auto& uedge:u->adj)
            edges.push_back(uedge);
    return edges;
}

void Graph::printInfo(){
    for(auto& u:V){
        std::cout<<"vertex{"<<u->key<<"} "<<u->label<<" -> ";
        for(auto& edge:u->adj)
            std::cout<<edge->v->label<<",";
        std::cout<<"\b\n";
    }
}
```

Classe figlia Directed Graph

```
class DirectedGraph : public Graph{
public:
    DirectedGraph(int n=0);           //create random graph if n is specified
    void addEdge(int ukey, int vkey, int weight = 0); //directed edge, u->v
    void getTransposed(DirectedGraph& dgt); //same vertexes, inverted edges
    ~DirectedGraph();
};

// -- DIRECTED GRAPH implementation

DirectedGraph::DirectedGraph(int n){ //create a graph with n predefined nodes
    for(int i=0; i<n; i++)
        addNode();
}

void DirectedGraph::addEdge(int ukey, int vkey, int weight){ //add unidir edge
    V[ukey]->adj.push_back(new Edge(V[ukey],V[vkey],weight));
}

void DirectedGraph::getTransposed(DirectedGraph& dgt){
    for(auto& u:V)
        dgt.addNode(u->label);

    for(auto& u:V)
        for(auto& uedge:u->adj){
            Node* v = uedge->v;
            dgt.addEdge(v->key, u->key, uedge->weight);
        }
}

DirectedGraph::~DirectedGraph(){ //free allocated memory
    for(auto& u:V){
        for(auto& edge:u->adj)
            delete edge;
        delete u;
    }
}
```

Classe figlia Undirected Graph

```
class UndirectedGraph : public Graph{
public:
    UndirectedGraph(int n=0);           //create random graph if n is specified
    void addEdge(int ukey, int vkey, int weight = 0); //undir edge, u->v and v->u
    ~UndirectedGraph();
};

// -- UNDIRECTED GRAPH implementation

UndirectedGraph::UndirectedGraph(int n){ //create a graph with n predefined
nodes
    for(int i=0; i<n; i++)
        addNode();
}

void UndirectedGraph::addEdge(int ukey, int vkey, int weight){ //add bidir edge
    V[ukey]->adj.push_back(new Edge(V[ukey],V[vkey],weight));
    V[vkey]->adj.push_back(new Edge(V[vkey],V[ukey],weight));
}

UndirectedGraph::~~UndirectedGraph(){ //free allocated memory
    for(auto& u:V){
        for(auto& edge:u->adj)
            delete edge;
        delete u;
    }
}
```

BFS

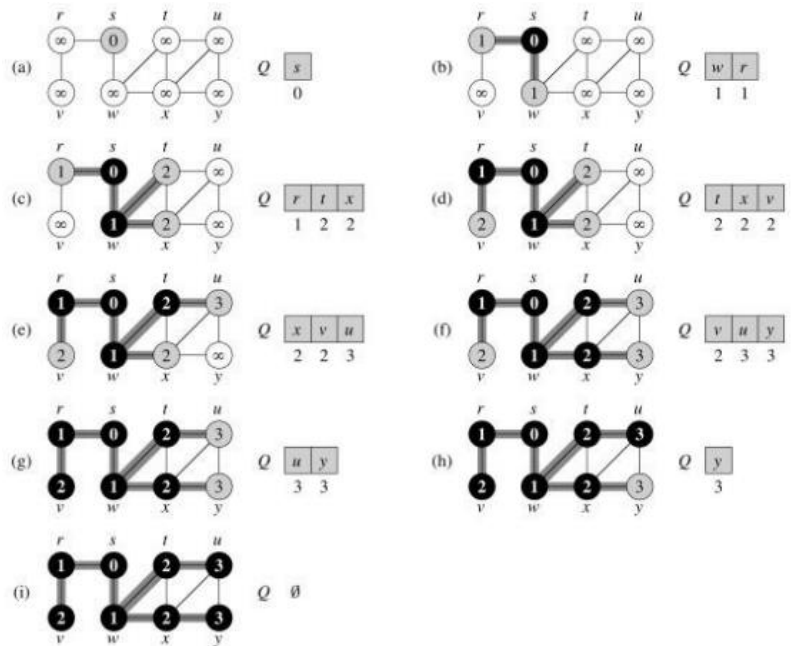
Ricordando lo pseudo-codice

BFS(G,s)

```

1. for each u in (G.V - {s})
2.     u.color = WHITE
3.     u.p = NIL
4.     u.d = ∞
5. s.color = GRAY
6. s.p = NIL
7. s.d = 0
8. Q = ∅
9. ENQUEUE(Q,s)
10. while(Q ≠ ∅)
11.     DEQUEUE(Q,u)
12.     for each v in u.adj
13.         if v.color = WHITE
14.             v.color = GRAY
15.             v.p = u
16.             v.d = u.d + 1
17.             ENQUEUE(Q,v);
18.     u.color = BLACK;

```



```

void BFS(Graph* G, int source){
    Node* s = G->V[source];
    for(auto u:G->V){
        u->color = WHITE;
        u->p = nullptr;
        u->d = INT_MAX;
    }
    s->color = GRAY;
    s->d = 0;
    s->p = nullptr;
    std::queue<Node*> Q;
    Q.push(s);
    while(!Q.empty()){
        Node* u = Q.front(); Q.pop();    //dequeue
        for(auto& uedge:u->adj){         //for each edge (u,v) of u
            Node* v = uedge->v;         //consider the node v of the edge
            if(v->color == WHITE){
                v->color = GRAY;
                v->p = u;
                v->d = u->d + 1;
                Q.push(v);
            }
        }
        u->color = BLACK;
    }
}

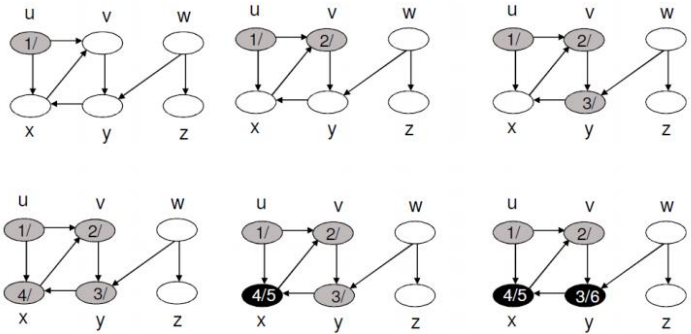
```

DFS

Ricordando lo pseudo-codice

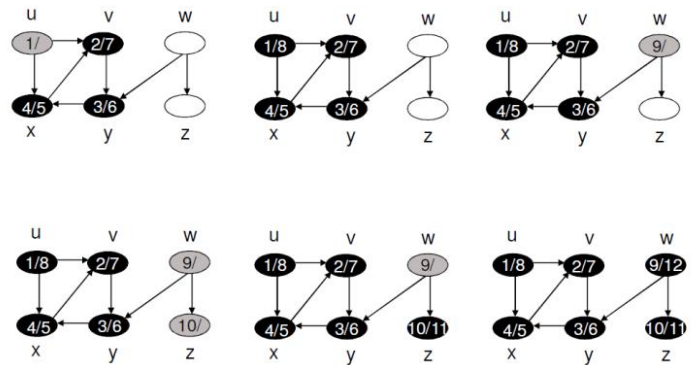
DFS(G)

1. for each u in $G.V$
2. $u.color = WHITE$
3. $u.p = NIL$
4. $time = 0$
5. for each u in $G.V$
6. if $u.color = WHITE$
7. DFS_VISIT(u);



DFS_VISIT(u)

1. $u.color = GRAY$
2. $u.d = time = time + 1$
3. for each v in $u.adj$
4. if $v.color = WHITE$
5. $v.p = u$
6. DFS_VISIT(v);
7. $u.color = BLACK$
8. $u.f = time = time + 1$



In C++

```
int t; //global variable time
void DFS(Graph* G){
    for(auto& u:G->V){
        u->color = WHITE;
        u->p = nullptr;
    }
    t = 0;
    for(auto u:G->V)
        if(u->color == WHITE)
            DFSvisit(u);
}

void DFSvisit(Node* u){
    u->color = GRAY;
    u->d = ++t;
    for(auto& uedge:u->adj){
        Node* v = uedge->v;
        if(v->color == WHITE){
            v->p = u;
            DFSvisit(v);
        }
    }
    u->color = BLACK;
    u->f = ++t;
}
```


CLASSIFICAZIONE DEGLI ARCHI

L'algoritmo di classificazione degli archi può essere visto come un'estensione del DFS. Infatti possiamo ottenere il "tipo" dell'arco verificando nella DFSvisit, per ogni vicino v di u , il colore del nodo v .

L'idea chiave è che ogni arco (u, v) può essere classificato in base al *colore del vertice v* che viene raggiunto quando l'arco viene ispezionato la prima volta:

- BIANCO: indica un **arco d'albero**.
- GRIGIO: indica un **arco all'indietro**.
- NERO: indica un **arco in avanti o trasversale**.

In C++

```
void edgeClassification(DirectedGraph* G){
    for(auto& u:G->V)
        u->color = WHITE;
    for(auto u:G->V)
        if(u->color == WHITE)
            ECvisit(u);
}

void ECvisit(Node* u){
    u->color = GRAY;
    for(auto& uedge:u->adj){
        Node* v = uedge->v;
        if(v->color == WHITE){
            uedge->type = TREE_EDGE;
            ECvisit(v);
        }
        else if(v->color == GRAY)
            uedge->type = BACK_EDGE;
        else
            uedge->type = (u->d < v->d)? FORWARD_EDGE : CROSS_EDGE;
    }
    u->color = BLACK;
}
```

ORDINAMENTO TOPOLOGICO (nuovo)

la DFS può essere utilizzata per realizzare l'**ordinamento topologico** (Top Sort) di un DAG (Direct Acyclic Graph).

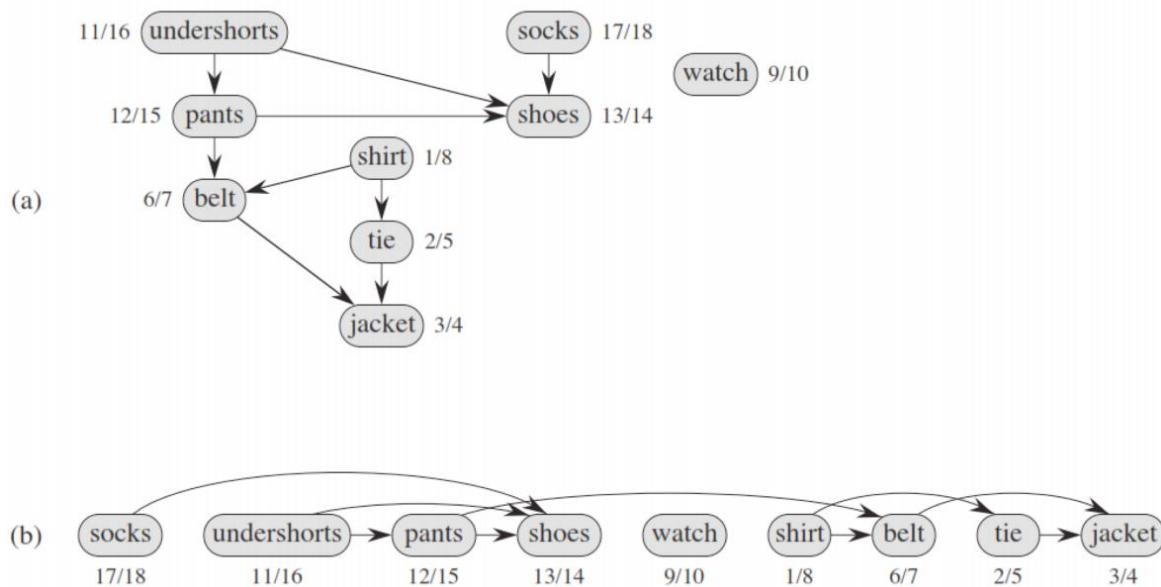
Un ordinamento topologico di un DAG $G=(V,E)$ è un ordinamento lineare di tutti i suoi vertici tale che se G contiene un arco (u,v) allora u appare prima di v nell'ordinamento. Per questo motivo, non possiamo usare un grafo che presenta cicli: se A dipende da B , B dipende da C , e C dipende da A (si è formato un ciclo), non sapremmo da dove cominciare.

Questo algoritmo viene utilizzato in molte applicazioni pratiche per modellare la **precedenza tra eventi**, e si basa sulla visita DFS.

Il Top Sort può trovare **un** ordinamento topologico in un tempo $O(V+E)$.

L'ordinamento topologico non è unico

es.



per mettere la giacca dobbiamo prima mettere la cintura e la cravatta, ma per mettere questi dobbiamo prima mettere la camicia. Inoltre, per mettere la cintura dobbiamo anche già aver messo i pantaloni, ma per metterli dobbiamo prima mettere le mutande.

PSEUDO-CODICE

TS_DFS(DG)

```
1. S = new Stack
2. for each u in DG.V
3.     u.color = WHITE
4.     u.p = NIL
5. time = 0
6. for each u in DG.V
7.     if u.color = WHITE
8.         TS_DFS_VISIT(u,S)
9. return S
```

TS_DFS_VISIT(u,S)

```
1. u.color = GRAY
2. u.d = time = time + 1
3. for each v in u.adj
4.     if v.color = WHITE
5.         v.p = u
6.         TS_DFS_VISIT(v,S);
7. u.color = BLACK
8. u.f = time = time + 1
9. push(S,u)
```

L'ordinamento topologico può essere visto come un'estensione del DFS. L'unica eccezione è che dobbiamo anche considerare uno stack S.

Ogni volta che abbiamo processato un nodo, lo aggiungiamo nello stack (per la natura dello stack, verrà inserito in cima).

Per la natura dell'algoritmo, gli elementi presenti all'interno di S saranno ordinati per tempo di fine in ordine decrescente (*vedi esempio*).

In C++

```
std::stack<Node*> tsDFS(DirectedGraph *G){
    std::stack<Node*> S;
    for(auto& u:G->V){
        u->color = WHITE;
        u->p = nullptr;
    }
    t=0;
    for(auto& u:G->V)
        if(u->color == WHITE)
            tsDFSvisit(u,S);
    return S;
}

void tsDFSvisit(Node* u, std::stack<Node*>& S){
    u->color = GRAY;
    u->d = ++t;
    for(auto& uedge:u->adj){
        Node* v = uedge->v;
        if(v->color == WHITE){
            v->p = u;
            tsDFSvisit(v,S);
        }
    }
    u->color = BLACK;
    u->f = ++t;
    S.push(u);
}
```

COMPONENTI FORTEMENTE CONNESSE

L'algoritmo per trovare le componenti fortemente connesse di un grafo $G = (V, E)$ utilizza il grafo *trasposto* di G , ossia $G^T = (V, E^T)$, dove $E^T = \{(u, v) : (v, u) \in E\}$.

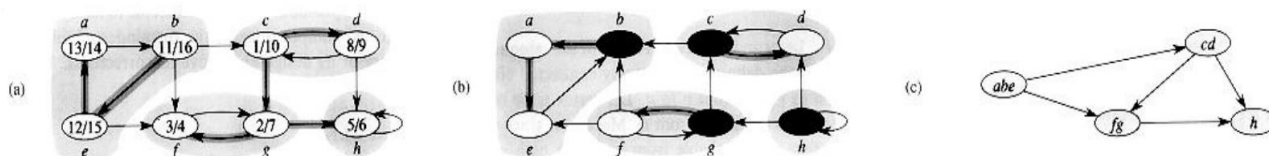
(ossia un grafo dove l'insieme dei vertici è lo stesso, ma l'insieme degli archi è *trasposto*, ovvero quello che era il predecessore diventa il successore, e viceversa. Infatti " E^T è trasposto da tutte le coppie (u, v) tali che l'arco (v, u) appartiene ad E ", per tutti gli archi).

Le componenti connesse possono essere calcolate attraverso il seguente algoritmo, in tempo $\Theta(V + E)$

STRONGLY_CONNECTED_COMPONENTS(G)

1. Chiama DFS(G) per calcolare i tempi di completamento $f[u]$ per ciascun vertice u .
2. Calcola il grafo trasposto G^T .
3. Chiama DFS(G^T), ma nel ciclo principale di DFS, considera i vertici in tempo decrescente rispetto ai tempi calcolati nella riga 1.
(Sostanzialmente, il primo vertice che viene analizzato è quello con il tempo di fine più elevato e si prosegue in ordine decrescente).
4. Genera l'output dei vertici di ciascun albero della foresta DF che è stata prodotta nella riga 3 come una singola componente fortemente connessa.

Alla fine di tale processo, ci ritroveremo un insieme di sottografi (e/o nodi, se ce ne è uno solo).



Il **grado delle componenti** $G^{SCC} = (V^{SCC}, E^{SCC})$ contiene un solo vertice per ogni componente connessa ed i soli archi tra le componenti connesse.

Nella seconda visita stiamo visitando i vertici del grado delle componenti in ordine topologico.

Ogni arco in G^T che congiunge differenti componenti connesse va da una componente con un tempo di completamento antecedente ad una componente con tempo di completamento successivo.

Si dimostra che nella seconda visita si parte dalla componente connessa con tempo di completamento massimo, per cui non ci possono essere archi verso un'altra componente connessa, e dunque si visitano solo i nodi di quella componente connessa.

Per le successive componenti connesse, gli archi potranno essere diretti solo verso delle componenti connesse già visitate, per cui anche in questo si visitano solo i nodi di quella componente connessa.

Si conclude che ogni albero DF conterrà esattamente una sola componente connessa.

Pseudo-codice

SSC(G)

```

1. S = TS_DFS(G)
2. create  $G^T$       (nb: assign color WHITE to all vertexes)
3. while( $S \neq \emptyset$ )
4.     v = S.extract()
5.     if v[ $G^T$ ].color = WHITE
6.         DFS_VISIT(v[ $G^T$ ])
7.         add precedent subgraph as a node of the SCC

```

Riepilogando:

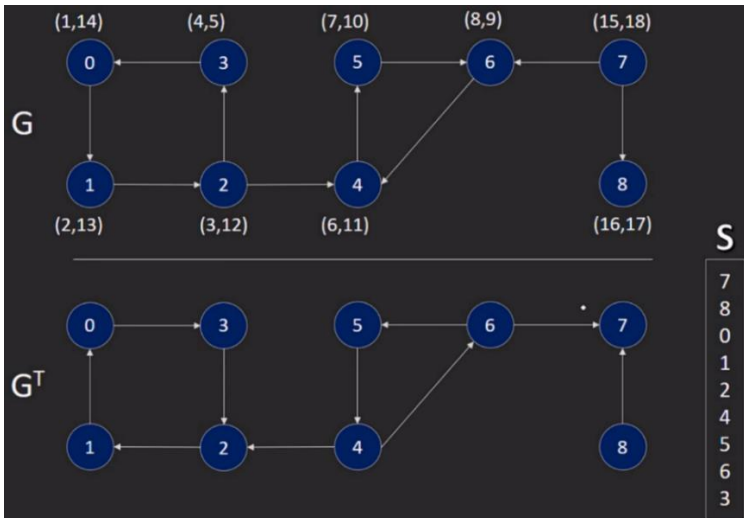
1. effettuiamo l'ordinamento topologico su G ;
2. calcoliamo il grafo trasposto di G , cioè G^T ha gli archi invertiti; vogliamo inoltre che ogni nodo di G^T sia di colore bianco.
- 3/6. Chiamiamo DFS_visit per G^T sui vertici per tempo di fine in ordine decresc. ;
7. Otteniamo i vertici come SCC separate.

nb: per richiamare i vertici in ordine topologico sfruttiamo proprio lo stack che viene restituito dalla funzione `tsDFS()`. Infatti l'elemento che verrà estratto dallo stack, di volta in volta, sarà quello con il tempo di fine maggiore.

La chiamata DFS_VISIT($v[G^T]$) vuole indicare che prendiamo in considerazione il nodo v , ma da parte di G^T e non di G . Infatti vogliamo effettuare ora la DFS sui nodi di G ma con gli archi trasposti.

Dopo la DFS_VISIT, siamo certi che i nodi esplorati partendo da v facciano tutti parte della stessa componente fortemente connessa. Infatti gli eventuali vicini di v che NON fanno parte della SSC di v non vengono esplorati, e questo perché il loro colore nella DFS_VISIT() non sarà bianco (ciò ce lo assicura l'ordinamento topologico).

es. dopo il punto 2 avremo la seguente situazione:



Notare che:

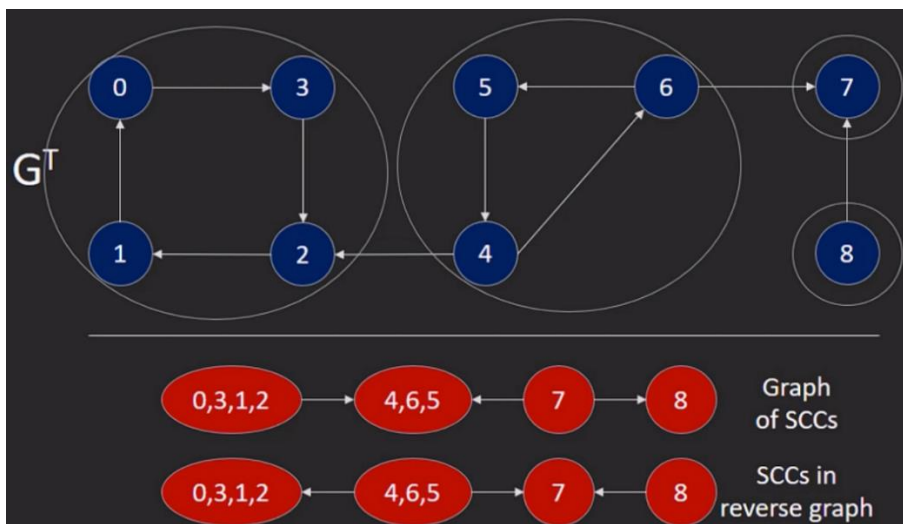
- G è il grafo originale;
- G^T è il grafo trasposto, ovvero che ha i vertici invertiti;
- S è lo stack risultato dall'ordinamento topologico.

Cominciamo il **while**:

- **estraiamo 7**: è bianco, dunque lo visitiamo, tuttavia 7 non esplora nessun vicino, perciò si tratta di un sottografo della SCC composto da un solo elemento;
- **estraiamo 8**: è bianco, ed ha come vicino 7, tuttavia 7 lo abbiamo processato prima (non è più bianco), quindi anche 8 è un sottografo della SCC composto da un solo elemento; però a differenza di prima questo sottografo (8) punterà al sottografo (7).
- **estraiamo 0**: è bianco. Visitiamo in ordine 3, 2 e 1. Il sottografo (0,3,2,1) è un sottografo della SCC;
- **estraiamo 1**: già processato, passiamo avanti;
- **estraiamo 2**: già processato, passiamo avanti;
- **estraiamo 4**: è bianco. Visitiamo in ordine 6 e 5. Notare che non è possibile visitare 7 e 2 in quanto li abbiamo già processati (non sono più bianchi). Il sottografo (4,6,5) è un sottografo della SCC, che punta ai sottografi di cui fa parte 7 e di cui fa parte 2.
- **estraiamo 5**: già processato, passiamo avanti;
- **estraiamo 6**: già processato, passiamo avanti;
- **estraiamo 3**: già processato, passiamo avanti;

Abbiamo estratto tutti i nodi da S , dunque la SCC sarà composta dai seguenti sottografi: (7), (8), (0,3,2,1), (4,6,5).

Per quanto riguarda gli archi, possiamo notare che si forma un arco fra due sottografi ogni volta che un nodo cerca di esplorare un nodo che però non è più bianco (cioè, appartiene ad un altro sottografo).



In C++

```
void SCC(DirectedGraph G, DirectedGraph& GSCC){
    DirectedGraph GT;
    std::string sccLabel = "";
    std::queue<std::string> Q;

    std::stack<Node*> S = tsDFS(&G);           //top sort
    G.getTransposed(GT);                       //create G transposed
    while(!S.empty()){
        Node* v = S.top(); S.pop(); //extract
        Node* gtv = GT.V[v->key]; //v by GT
        if( gtv->color == WHITE ){
            sccLabel += "(" + gtv->label + ","; //first node + aesthetic
            sccDFSvisit(gtv,sccLabel,Q); //find SSC, add edges with other SCC to Q
            sccLabel += "\b"; //aesthetic
            GSCC.addNode(sccLabel); //subgraph represented ad node
            sccLabel = ""; //reset string

            while(!Q.empty()){
                for(int i=0; i<GSCC.V.size()-1; i++){
                    if( (GSCC.V[i]->label).find(Q.front()) != std::string::npos){
                        GSCC.addEdge(i, GSCC.V.back()->key);
                        break;
                    }
                }
                Q.pop();
            }
        }
    }
}

void sccDFSvisit(Node* u, std::string& sccLabel, std::queue<std::string>& Q){
    u->color = GRAY;
    u->d = ++t;
    for(auto uedge:u->adj){
        Node* v = uedge->v;
        if(v->color == WHITE){
            sccLabel += v->label + ",";
            v->p = u;
            sccDFSvisit(v,sccLabel,Q);
        }
        else if(v->color == BLACK){ //node is reaching another subgraph
            Q.push(v->label);
        }
    }
    u->color = BLACK;
    u->f = ++t;
}
```

Semplicemente, tutti i nodi raggiungibili da u che sono bianchi (nel grafo trasposto) fanno tutti parte dello stesso sottografo. Se un nodo raggiungibile da u è invece nero, significa che è un nodo che fa parte di un altro sottografo (ce ne ricorderemo inserendo l'etichetta di quel nodo in una coda, ci servirà per creare un edge fra i due sottografi).

nb: in "getTransposed", per GT inizializziamo anche tutti i nodi a WHITE.

Il nostro scopo è creare un nuovo grafo (e non un insieme di grafi), dove ogni nodo è un sottografo. Il sottografo sarà rappresentato come una concatenazione delle etichette di ogni suo nodo.

Creato il nodo-sottografo, verifichiamo se ci sono collegamenti con altri sottografi (esploriamo la coda finché non è vuota, e se c'è un elemento cerchiamo di capire a quale sottografo appartiene analizzando l'etichetta). Il primo sottografo non potrà mai avere archi uscenti verso altri sottografi (ma può avere archi entranti).

nb implementazione: a volta, può capitare che un nodo abbia due archi uguali verso un altro nodo. Questo problema può essere risolto eliminando eventuali duplicati o migliorando il ragionamento per trovare gli archi.

(nel risultato sottostante questa situazione non si verifica, ma col grafo esempio di Ferone si).

es. grafo precedente:

```
Strongly Connected Components

Original Graph:
vertex{0} a -> b
vertex{1} b -> c
vertex{2} c -> d,e
vertex{3} d -> a
vertex{4} e -> f
vertex{5} f -> g
vertex{6} g -> e
vertex{7} h -> g,i
vertex{8} i ->

SCC:
vertex{0} (h) -> (i),(e,g,f)
vertex{1} (i) ->
vertex{2} (a,d,c,b) -> (e,g,f)
vertex{3} (e,g,f) ->
```



LEZ 19 – GRAFI (INSIEMI DISGIUNTI e MST(KRUSKAL E PRISM))

Ripetizione lezione 16 (da Insiemi Disgiunti) e lezione 17 di Camastra.

Da notare solo che se vogliamo creare una struttura di insiemi disgiunti:

- **attraverso una lista di adiacenze:** **Make-set(x)** crea una lista con un solo elemento, **Find-set(x)** segue il puntatore all'oggetto dell'insieme (partendo da x), e **Union(x,y)** concatena la lista di cui fa parte x con la lista di cui fa parte y (se seguiamo l'euristica dell'unione pesata, concateniamo quella più corta a quella più lunga).
- **attraverso alberi:** **Make-set(x)** crea un albero con un solo nodo, **Find-set(x)** segue i puntatori ai padri fino alla radice (rappresentante), e **Union(x,y)** fa puntare la radice di un albero alla radice di un altro (o se seguiamo le euristiche, tutti i nodi di un albero diventano figli dell'altro albero che ha rappresentante con rango maggiore).

Operazioni su insiemi disgiunti per alberi

Pseudo-codice

MAKE-SET(x)

1. *create new tree with root "x"* //basta anche solo mettere x.p=x

UNION(x,y) //version 1

1. LINK(FIND_SET(X), FIND-SET(Y))

LINK(x,y)

```
1. if x.rank > y.rank
2.     y.p = x
3. else //x.rank <= y.rank
4.     x.p = y
5.     if x.rank == y.rank
6.         y.rank = y.rank + 1
```

UNION(x,y) //version 2

```
1. rx = FIND_SET(x)
2. ry = FIND_SET(y)
3. if rx.rank > ry.rank
4.     ry.p = rx
5. else //rx.rank <= ry.rank
6.     rx.p = ry
7.     if rx.rank == ry.rank
8.         ry.rank = ry.rank + 1
```

FIND_SET(X)

```
1. if x ≠ x.p
2.     x.p = FIND_SET(x.p)
3. return x.p
```

nb: le due funzioni UNION non utilizzano la compressione dei cammini. Inoltre, se i due rappresentanti (x,y) hanno rango uguale, considera come rappresentante y (e in tal caso ne aumenta anche il rango).

Per la compressione dei cammini, un'idea è che ogni nodo dovrebbe avere una lista di puntatori a figli. Visitiamo tutti i figli del rappresentante (è l'unico che ha figli), e ne impostiamo il padre al rappresentante dell'altro albero.

Rappresentazione di insiemi disgiunti mediante liste

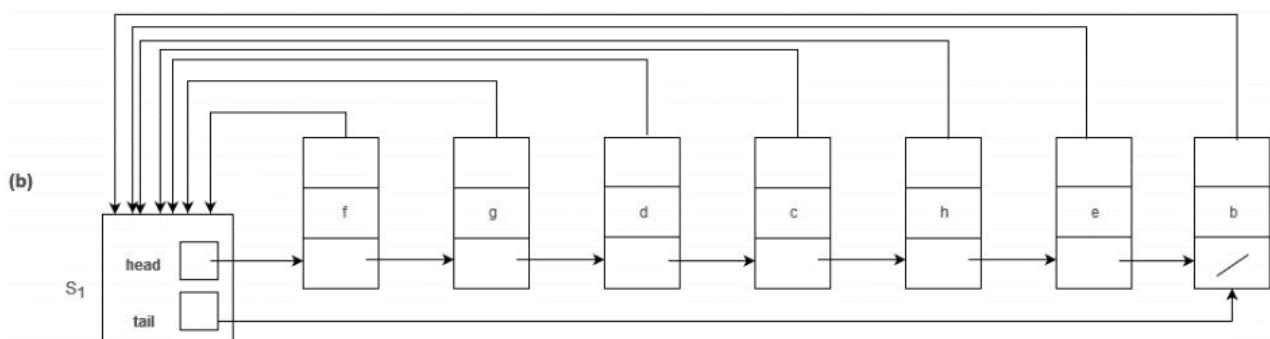
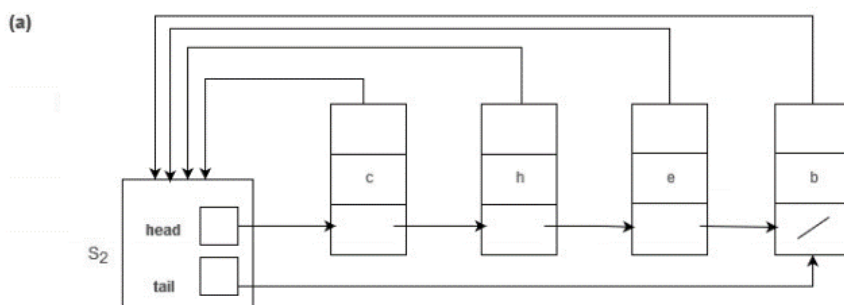
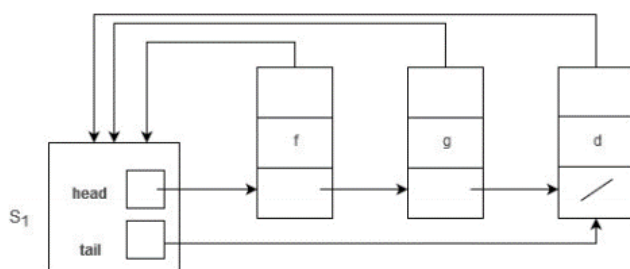
Ciascun insieme è rappresentato da una **lista concatenata**.

Ogni oggetto della lista contiene un elemento dell'insieme, un **puntatore** all'elemento **successivo** ed un **puntatore** all'oggetto dell'insieme.

Ogni oggetto dell'insieme ha inoltre un puntatore **head** ed uno **tail**.

In questo caso:

1. **Make-Set(x)** : crea una lista con un solo elemento;
2. **Find-Set(x)** : parte da x, e si segue il puntatore all'oggetto dell'insieme;
3. **Union(x,y)** : concatena una lista all'altra.



Per l'**unione** è possibile utilizzare l'**euristica dell'unione pesata** per migliorare le prestazioni, nella quale si conserva, per ogni lista, l'informazione relativa alla sua **lunghezza**, e si accoda la lista più corta a quella più lunga.

Un nodo è generalmente così composto:

```
Node{
    int rank;
    int data;
    Node parent;
};
```

In C++ (rappresentazione mediante alberi)

```
void makeset(Node* x){
    x->p = x;
}

Node* findset(Node* x){
    if(x != x->p)
        x->p = findset(x->p);
    return x->p;
}

void unionsets(Node* x, Node* y){
    Node* rx = findset(x);
    Node* ry = findset(y);
    if(rx->rank > ry->rank)
        ry->p = rx;
    else
        rx->p = ry;
    if(rx->rank == ry->rank)
        ry->rank++;
}
```

ALBERO DI COPERTURA DI MINIMO

Un **albero ricoprente T** (spanning tree) di G, ove G è un grafo connesso pesato e non orientato, è un subgrafo che è un albero, e connette tutti i vertici.

Un grafo può avere differenti **spanning trees**.

L'albero ricoprente che ha il peso minimo prende il nome di **albero ricoprente minimo** (minimum spanning tree, MST). Più precisamente, considerati i pesi di ogni arco (u,v) , $w(u,v)$, di tutti gli spanning tree Ts, avremo che:

$$MST = \min_{T \in Ts} \left(\sum_{(u,v) \in T} w(u,v) \right)$$

Generic-MST(G, w)

1. $A = \emptyset$
2. **while** *A non forma un albero di copertura*
3. trova un arco sicuro (u, v)
4. $A = A \cup \{(u, v)\}$
5. **return** A

Possiamo usare l'algoritmo di Kruskal o di Prim per calcolare un MST.

KRUSKAL

Pseudo-codice

MST_KRUSKAL(G, w)

1. $A = \emptyset$
2. **for each** u **in** $G.V$
3. MAKE_SET(u)
4. SORT($G.E$) //order edges by non-desc weight
5. **for each** $\{(u,v)\}$ **in** $G.E$
6. **if** FIND_SET(u) $\neq$ FIND_SET(v)
7. $A = A \cup \{(u,v)\}$
8. UNION(u,v)
9. **return** A

In C++

```
bool edgeWeightComparison(Edge* uv, Edge* xy){
    return (uv->weight < xy->weight);
}

void kruskalMST(UndirectedGraph G, UndirectedGraph &A){
    for(auto& u:G.V)
        makeset(u);
    std::vector<Edge*> edges = G.getEdges();
    std::sort(edges.begin(), edges.end(), edgeWeightComparison); //order by weight
    for(auto& uv:edges)
        if(findset(uv->u) != findset(uv->v)){
            A.graphUnion(uv);
            unionsets(uv->u, uv->v);
        }
}
```

nb: A è un grafo non orientato che viene creato fuori dallo scope della funzione. Dovrà essere vuoto e verrà passato per riferimento.

Per l'unione di due set, aggiungiamo un nuovo metodo alla classe Graph "graphUnion()", la quale accetta un edge (u,v) e da questo aggiunge i vertici u e v e l'arco uv nel grafo da cui il metodo è stato chiamato.

Siccome stiamo facendo un'unione, inseriremo un vertice all'interno del grafo solo se questo non è già presente (verifichiamo solo se le label sono uguali). Fatto questo controllo preliminare, e avendo ora i vertici u e v (appena creati o che già avevamo), possiamo aggiungere un nuovo arco bidirezionale fra u e v che ha lo stesso peso dell'arco passato prima.

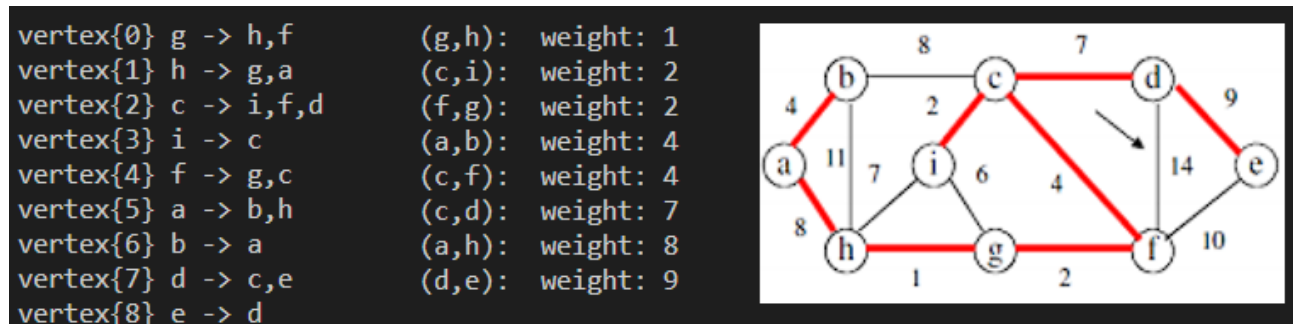
```
void Graph::graphUnion(Edge* uv){
    Node *u, *v;

    bool found = false;
    for(auto& x:V)
        if(x->label == uv->u->label){
            found = true;
            u = x;
            break;
        }
    if(!found)
        u = addNode(uv->u->label);

    found = false;
    for(auto& x:V)
        if(x->label == uv->v->label){
            found = true;
            v = x;
            break;
        }
    if(!found)
        v = addNode(uv->v->label);

    addEdge(u->key, v->key, uv->weight);
}
```

es.



PRIM

Pseudo-codice

MST_PRIM(G,w,r)

```
1. for each u in G.V
2.     u.key =  $\infty$ 
3.     u.p = NIL
4. r.key = 0
5. Q = G.V
6. while Q  $\neq \emptyset$ 
7.     u = EXTRACT_MIN(Q)
8.     for each v in u.adj
9.         if v  $\in$  Q and w(u,v) < v.key
10.            v.p = u
11.            v.key = w(u,v) //decrease key
```

In C++

nb: per poter implementare questo algoritmo in modo efficiente dovremmo utilizzare un min-heap nel quale, ogni volta abbiamo trovato un arco con un peso minore per il nodo v, chiamiamo una “decrease-key” così da aggiornare la chiave di key e far risalire v nella giusta posizione all’interno del min-heap.

Il problema in C++ è che la classe `std::priority_queue` non fornisce un’operazione “decrease-key”, tantomeno un’operazione per verificare se un elemento appartiene alla coda, dunque se vogliamo essere “fedeli” allo pseudo-codice dovremmo implementarla a mano.

LEZ 20 – IL PROBLEMA DEI CAMMINI MINIMI (DIJKSTRA e BM-FORD)

IL PROBLEMA DEI CAMMINI MINIMI

Il **cammino minimo** tra due vertici u e v di un grafo è quel percorso che permette di collegare tali vertici minimizzando il costo degli archi da attraversare.

Sia dato un grafo $G = (V, E)$ orientato e pesato, con una funzione peso $w : E \rightarrow \mathbb{R}$, che associa ad ogni arco un peso, rappresentato da un numero reale.

Dato che il **peso di un cammino** $p = (v_0, \dots, v_k)$ è dato dalla somma dei pesi degli archi che lo costituiscono, ovvero formalmente:

$$w(p) = \sum_{i=1}^k (w(v_{i-1}, v_i))$$

Allora il **peso di un cammino minimo**, da u a v , è definito come:

$$\delta(u, v) = \begin{cases} \min (w(p) : u \xrightarrow{p} v) & \text{se esiste } p(u, v) \\ \infty & \text{altrimenti} \end{cases}$$

Archivi con pesi negativi

I pesi possono anche essere **negativi**, ma nel caso nel grafo fossero presenti **cicli di peso negativo** non può essere definito un cammino minimo.

Un **ciclo di peso negativo** è un ciclo nel quale la somma di tutti gli archi dà come risultato un peso negativo.

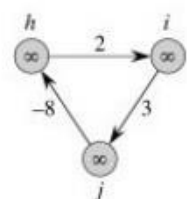
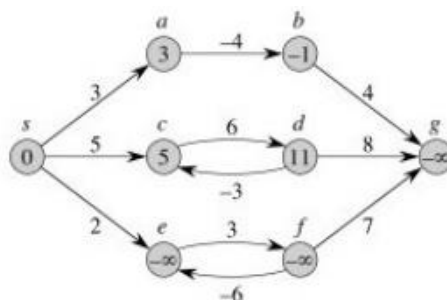
Infatti nessun peso di cammino minimo può essere definito dalla sorgente s ad un vertice del ciclo, poiché ci si bloccherebbe in un ciclo infinito dove si andrebbe a rivisitare perennemente gli archi del ciclo.

Se vi è un **ciclo di peso negativo** lungo un cammino tra due vertici s e v , si definisce $\delta(s, v) = -\infty$.

Alcuni algoritmi suppongono che tutti i pesi degli archi sia non negativi (Dijkstra), per altri è indifferente (Bellman-Ford).

es. tra c e d si forma un ciclo. La somma degli archi che fanno parte del ciclo (ovvero (c, d) e (d, c)) è positiva ($6 - 3 = 3$), dunque non si tratta di un ciclo negativo.

tra h , i e j si forma un ciclo.
La somma degli archi che fanno parte del ciclo (ovvero (h, i) , (i, j) , e (j, h)) è negativa ($2 + 3 - 8 = -3$), dunque si tratta di un ciclo negativo.



Rappresentazione dei cammini minimi

Dato un grafo $G = (V, E)$, manteniamo per ogni vertice $v \in V$ un **predecessore** $\pi(v)$.

Dato un vertice v , **seguendo a ritroso** la catena dei predecessori è possibile ripercorrere il cammino minimo fino alla sorgente s .

Anche in questo caso siamo interessati al **sottografo dei predecessori** $G_\pi = (V_\pi, E_\pi)$, indotto dai valori dei predecessori.

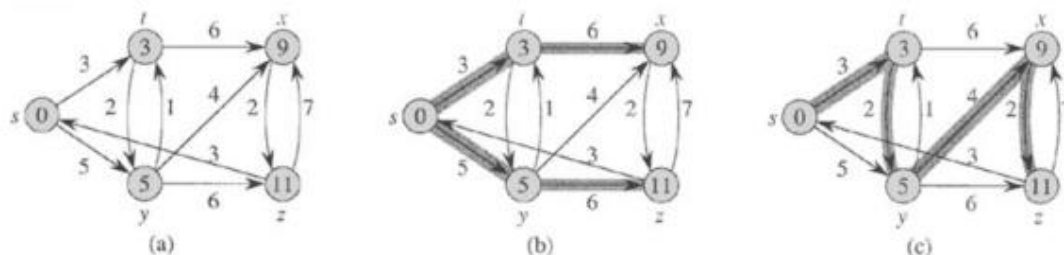
$$V_\pi = \{v \in V : v.\pi \neq NIL\} \cup \{s\}$$

$$E_\pi = \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\}$$

Albero dei cammini minimi

Un albero dei cammini minimi, con radice s , è un sottografo orientato $G_1 = (V_1, E_1)$, dove V_1 e E_1 sono rispettivamente sottoinsiemi di V ed E tali che:

- V_1 è l'insieme dei vertici raggiungibili da s in G .
- G_1 forma un albero con radice s .
- Per ogni $v \in V_1$, l'**unico cammino semplice** da s a v in G_1 è un **cammino minimo** da s a v in G .



Sottostruttura ottima dei cammini minimi

Sapendo che un **sottocammino** di un cammino minimo è a sua volta un cammino minimo, diremo che:

dato un grafo orientato e pesato $G = (V, E)$ ed una funzione peso $w : E \rightarrow \mathbb{R}$,

sia $p = (v_1, \dots, v_k)$ un cammino minimo dal vertice v_1 al vertice v_k ,

e sia inoltre $p_{i,j} = (v_i, \dots, v_j)$ il sottocammino di p tra v_i e $v_j \quad \forall i, j : 1 \leq i \leq j \leq k$.

Allora $p_{i,j}$ è un **cammino minimo** tra v_i e v_j .

Corollario

Sia un grafo orientato e pesato $G = (V, E)$ con una funzione peso $w : E \rightarrow \mathbb{R}$,

si supponga che un **cammino minimo** p da una sorgente s ad un vertice v possa essere **decomposto** in un cammino p' dalla sorgente s ad un vertice u e da un cammino dal vertice u al vertice v .

Allora avremo che il peso del cammino minimo da s a v è $\delta(s, v) = \delta(s, u) + \delta(u, v)$.

Tecnica del Rilassamento

Il processo di **rilassare** un arco consiste nel **migliorare il cammino minimo** di un nodo v dal nodo sorgente s , abbassando la stima superiore del cammino, ovvero $d[v]$.

In particolar modo, rilassare l'arco (u, v) significa che la distanza del cammino che ci permette di raggiungere v passando per u è MINORE di quella fra v e il suo attuale predecessore. In tal caso, u diventa il nuovo predecessore di v ed aggiorniamo la distanza di cammino minimo di v .

Tale processo di rilassamento è effettuato dalla procedura **RELAX**(u, v, w).

RELAX(u, v, w)

1. **if** $v.d > u.d + w(u, v)$
2. $v.d = u.d + w(u, v)$ //se usiamo una coda, è una decrease-key()
3. $v.p = u$

LEMMA

Sia $G = (V, E)$ un grafo orientato e pesato con funzione peso $w: E \rightarrow R$, e $(u, v) \in E$.

Allora dopo il rilassamento di (u, v) avremo che:

$$d[v] \leq d[u] + w(u, v)$$

Infatti se dobbiamo aggiornare $d[v]$, allora $d[v] = d[u] + w(u, v)$.

Altrimenti, significa che la stima di $d[v]$ è migliore, allora $d[v] < d[u] + w(u, v)$.

INIZIALIZZAZIONE DELLE STIME

Le stime di cammino minimo ed i predecessori vengono **inizializzati** dalla procedura:

INITIALIZE_SINGLE_SOURCE(G, s)

1. **for each** v **in** $(G.V - \{s\})$
2. $v.d = \infty$
3. $v.p = \text{NIL}$
4. $s.d = 0$
5. $s.p = \text{NIL}$

All'inizio tutti i nodi avranno distanza "infinita" dal nodo sorgente, perché non è stato ancora stimato alcun cammino minimo. Il nodo sorgente avrà distanza 0 da se stesso.

PROPRIETÀ DEI CAMMINI MINIMI E DEL RILASSAMENTO

Possiamo distinguere 6 proprietà:

- **Disuguaglianza triangolare:**
per qualsiasi arco $(u, v) \in E$, si ha: $\delta(s, v) \leq \delta(s, u) + w(u, v)$. ¹
- **Proprietà del limite superiore:**
per tutti i vertici $v \in V$, si ha sempre $d[v] \geq \delta(s, v)$ e, una volta che il limite superiore $d[v]$ assume il valore $\delta(s, v)$, questo non cambia più. ²
- **Proprietà dell'assenza di un cammino:**
se non c'è un cammino da s a v , allora si ha sempre: $d[v] = \delta(s, v) = \infty$.
- **Proprietà della convergenza:**
se $s \rightsquigarrow u \rightarrow v$ è un cammino minimo in G per qualche $u, v \in V$,
e se $d[u] = \delta(s, u)$ in un istante qualsiasi prima del rilassamento dell'arco (u, v) ,
allora $d[u] = \delta(s, v)$ in tutti gli istanti successivi. ⁴
- **Proprietà del rilassamento del cammino:**
Se $p = (v_0, v_1, \dots, v_k)$ è un cammino minimo da $s = v_0$ a v_k e gli archi di p vengono rilassati nell'ordine $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, allora $d[v_k] = \delta(s, v_k)$.
Questa proprietà è soddisfatta indipendentemente da altri passi di rilassamento che vengono effettuati, anche se sono interposti fra i rilassamenti degli archi di p .
- **Proprietà del sottografo dei predecessori:**
una volta che $d[v] = \delta(s, v) \quad \forall v \in V$, il sottografo dei predecessori è un albero di cammini minimi radicato in s .

¹ Infatti, se u fa parte del cammino minimo avremo che: $\delta(s, v) = \delta(s, u) + w(u, v)$;
altrimenti, avremo che: $\delta(s, v) < \delta(s, u) + w(u, v)$.

² In altre parole, una volta che abbiamo finito di esplorare tutti i vicini di un nodo v , significa anche che avremo trovato $\delta(s, v)$. Anche se in seguito incontriamo di nuovo v (perché è il vicino di qualche altro nodo), già sappiamo che il costo del percorso dalla sorgente è il minore (Dijkstra sfrutta questa proprietà).

⁴ " $s \rightsquigarrow u \rightarrow v$ " sarebbe il "il sottocammino che va da s a u , del cammino che va da s a v ".

ALGORITMO DI DIJKSTRA

L'algoritmo di Dijkstra è un algoritmo utilizzato per cercare i cammini minimi in un grafo partendo da una sorgente s . Presuppone che venga usato un grafo orientato e pesato $G = (V, E)$ con funzione peso non negativa $w: E \rightarrow R^+$ e sorgente s .

L'algoritmo di Dijkstra mantiene un **insieme S** che contiene i **vertici il cui peso di cammino minimo dalla sorgente è già stato determinato**, cioè per tutti vertici $v \in S$, vale $d[v] = \delta(s, v)$.

Esso seleziona ripetutamente il **vertice $u \in V - S$** con la minima stima di cammino minimo, inserisce u in S e **rilassa tutti gli archi uscenti da u** .

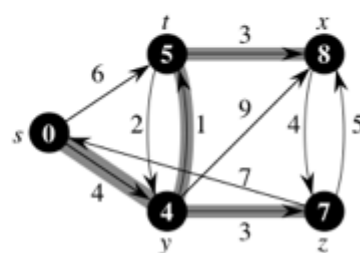
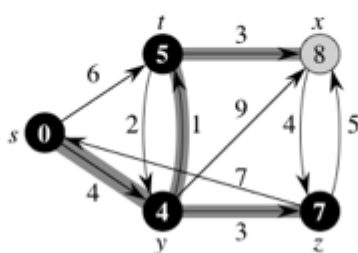
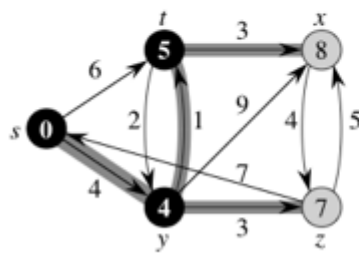
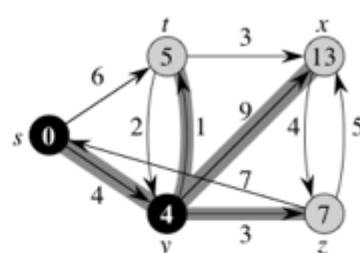
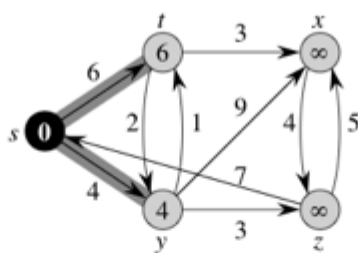
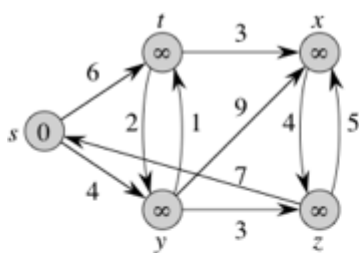
Utilizza una **coda di minima priorità Q** che contiene tutti i vertici $V - S$, usando come **chiave la stima di cammino minimo**.

Utilizza una strategia **greedy** scegliendo ad ogni passo il vertice u in $V - S$, con stima di cammino minimo inferiore.

È possibile utilizzare una greedy choice in quanto vale il teorema della correttezza dell'algoritmo di Dijkstra.

DIJKSTRA(G, w, s)

1. INITIALIZE_SINGLE_SOURCE(G, s)
2. $S = \emptyset$
3. $Q = G.V$
4. **while** $Q \neq \emptyset$
5. $u = \text{EXTRACT_MIN}(Q)$
6. $S = S \cup \{u\}$
7. **for each** v **in** $u.\text{adj}$
8. RELAX(u, v, w)



analisi dello pseudo-codice

Il valore d rappresenta la stima del cammino minimo dalla sorgente. All'inizio questa stima sarà infinito per tutti i nodi tranne la sorgente, che invece sarà 0.

Ad ogni iterazione estrarremo dalla coda il nodo con la stima minore e, per ogni suo vicino, andremo ad applicare la tecnica del rilassamento. Il primo nodo ad essere estratto sarà sempre la sorgente.

Attraverso la tecnica del rilassamento andiamo a vedere se il nuovo percorso darà un valore di d inferiore a quello già posseduto dal nodo v . In tal caso, aggiorniamo la stima per v .

Quando aggiorniamo il valore di d per un nodo v nella tecnica del rilassamento, in realtà stiamo facendo una **decrease-key**($v, u.d + w(u,v)$), questo perché vogliamo che la coda si aggiorni ad ogni passo di rilassamento. A livello pratico, effettuare solo un'assegnazione non comporta alcuna modifica nell'ordine dei vertici della coda.

Dal momento che un nodo esce dalla coda, il peso del suo cammino NON CAMBIA più. Questo ha come conseguenza anche che non cambia nemmeno più il predecessore di quel nodo (*proprietà del limite superiore*).

(ad esempio, nell'esempio quando abbiamo esplorato i vicini di s , estrarrò poi x dalla coda. Questo significa che ho già trovato il cammino minimo (s,x) e questo non cambierà più).

nb: quando estraiamo un vertice dalla coda, se esploriamo vertici già estratti (perché sono adiacenti) sapremo che il controllo nella tecnica di rilassamento non verrà mai superato, per questo motivo non abbiamo bisogno di controllare se il nodo NON è presente nella coda.

Il ciclo terminerà quando nella coda non sono più presenti elementi.

La **complessità** dell'algoritmo di Dijkstra varia in base a come viene implementato, ma con una coda di minima priorità basata su un min-heap è **$O((V+E) \log V)$** , o se il numero di archi è molto superiore a quello di vertici, allora sarà **$O(E \log V)$** .

Corollario

Se si esegue l'algoritmo su un grafo orientato e pesato $G = (V,E)$ con funzione peso non negativa $w: E \rightarrow R^+$ e sorgente s , allora al termine il **sottografo dei predecessori G_π** è un **albero di cammini minimi con radice s** .

Teorema della correttezza dell'algoritmo di Dijkstra

Se si esegue l'algoritmo di Dijkstra su un grafo orientato e pesato $G = (V, E)$ con funzione peso non negativo $w: E \rightarrow \mathbb{R}^+$ e sorgente s , allora al termine vale $d[u] = \delta(s, u) \forall u \in V$.

All'inizio del ciclo while $d[v] = \delta(s, v) \forall v \in S$.

È sufficiente **dimostrare** che $d[u] = d[v] = \delta(s, v) \forall v \in V$ nel momento in cui u viene aggiunto ad S , infatti per la *proprietà del limite superiore* l'eguaglianza resterà valida in tutti gli istanti successivi.

Possiamo dimostrare tale teorema **per assurdo**. Infatti supponiamo per assurdo che u sia il primo vertice per cui $d[u] \neq \delta(s, u)$.

Deve essere vero che:

- $u \neq s$, perché $d[s] = \delta(s, s) = 0$;
- $S \neq \emptyset$;
- $\exists p_{s,u}$, altrimenti $d[u] = \delta(s, u) = \infty$;
- deve esistere un **cammino minimo**, poiché esiste almeno un cammino.

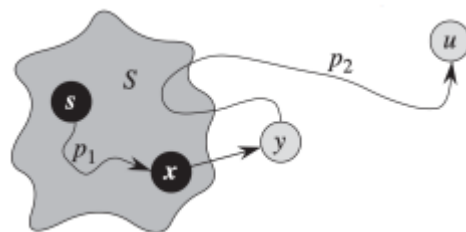
Prima di aggiungere u ad S , il cammino p collega un vertice in S (ovvero s) con un vertice in $V - S$ (ovvero u).

Consideriamo il primo vertice y in p tale che $y \in V - S$, e sia $x \in S$ il suo predecessore.

Il cammino p può essere scomposto in $s \rightsquigarrow x$ e $y \rightsquigarrow u$ (che potrebbero anche non contenere archi).

Quando u viene aggiunto ad S , abbiamo che $d[y] = \delta(s, y)$ perché:

- $x \in S$;
- u è il primo vertice per cui $d[u] \neq \delta(s, u)$;
- quindi $d[x] = \delta(s, x)$, e rilassiamo l'arco (x, y)
- dunque $d[y] = \delta(s, y)$.



Poiché y precede u si ha:

- $\delta(s, y) \leq \delta(s, u)$;
- $d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]$;
- ma poiché sia y che u sono in $V - S$ quando u entra in S (esce dalla coda), allora avremo che $d[u] \leq d[y]$. Dunque la considerazione di prima diventa:
 $d[y] = \delta(s, y) = \delta(s, u) = d[u]$.

Tuttavia stiamo affermando che $d[u] = \delta(s, u)$, che **contraddice la nostra ipotesi iniziale** $d[u] \neq \delta(s, u)$, e dunque l'algoritmo di Dijkstra è dimostrato per assurdo.

ALGORITMO DI BELLMAN-FORD

L'algoritmo di Bellman-Ford risolve il problema dei cammini minimi con sorgente singola nel caso più generale in cui i **pesi** possono essere anche **negativi**.

Si tratta di un algoritmo che sfrutta la **programmazione dinamica**.

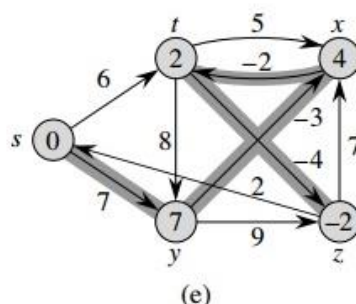
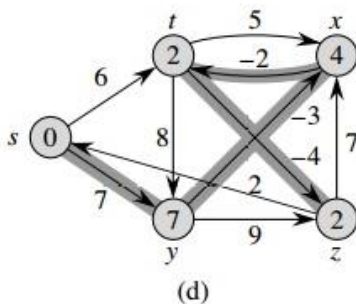
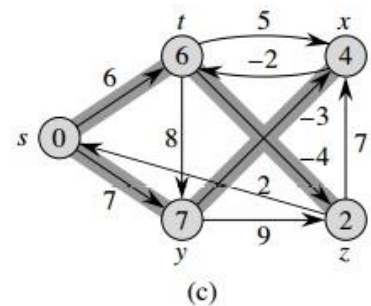
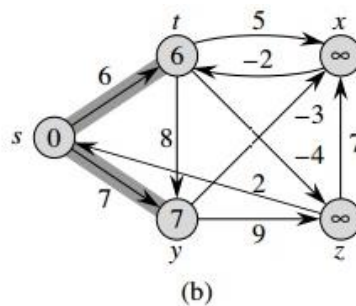
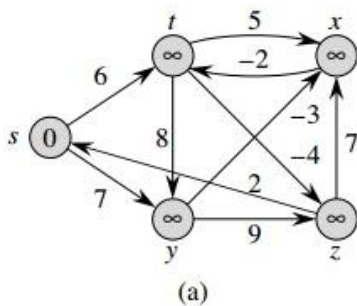
Come nel caso di Dijkstra, vogliamo trovare il percorso più breve da un nodo sorgente ad uno qualsiasi degli altri nodi del grafo.

Dato un grafo orientato $G = (V;E)$ e pesato con funzione peso $w:E \rightarrow R$ e sorgente $s \in V$, l'algoritmo di Bellman-Ford restituisce un **valore booleano** che indica se **esiste** o meno un **ciclo di peso negativo raggiungibile dalla sorgente**.

- In caso affermativo, **non esiste alcuna soluzione**.
Infatti in un ciclo negativo continuiamo a girare in tondo all'infinito.
- Se un tale ciclo non esiste, allora l'algoritmo produce i cammini minimi ed i loro pesi.

BELLMAN_FORD(G, w, s)

```
1. INITIALIZE_SINGLE_SOURCE( $G, s$ )
2. for  $i=1$  to  $|V|-1$ 
3.   for each  $(u,v)$  in  $G.E$ 
4.     RELAX( $u,v,w$ )
5. for each  $(u,v)$  in  $G.E$ 
6.   if  $v.d > u.d + w(u,v)$ 
7.     return false
8. return true
```



ESEMPIO di esecuzione dell'algoritmo di Bellman-Ford:

Consideriamo il grafo dell'esempio precedente. Avremo $|V| = 5$ e i seguenti archi:

- (s,t,6)
- (s,y,7)
- (t,y,8)
- (t,x,5)
- (t,z,-4)
- (x,t,-2)
- (y,x,-3)
- (y,z,9)
- (z,s,2)
- (z,x,7)

Dobbiamo applicare per $|V|-1$ (ovvero 4) volte il rilassamento a tutti gli archi:

it/vertex	s	t	x	y	z
0(start)	0	∞	∞	∞	∞
1	0	6	∞	7	∞
2	0	6	4	7	2
3	0	2	4	7	2
4	0	2	4	7	-2

Se non sono presenti cicli di peso negativo, allora dai passi precedenti abbiamo trovato i cammini minimi per tutti i nodi partendo dal nodo sorgente.

Per verificare se esiste un ciclo di peso negativo, basta verificare per ogni arco (u,v) se la distanza di v è la stima di cammino minimo minore, ovvero che $d[v] \leq d[u] + w(u,v)$, altrimenti se $d[v] > d[u] + w(u,v)$ allora $d[v]$ non è la stima di cammino minimo, e dunque ci troviamo di fronte ad un ciclo negativo.

Verifica dei cicli negativi:

- $d[t] > d[s] + w(s,t)$? cioè, $2 > 0 + 6$? no
- $d[y] > d[s] + w(s,y)$? cioè, $7 > 0 + 7$? no
- $d[y] > d[t] + w(t,y)$? cioè, $7 > 2 + 8$? no
- $d[x] > d[t] + w(t,x)$? cioè, $4 > 2 + 5$? no
- $d[z] > d[t] + w(t,z)$? cioè, $-2 > 2 + (-4)$? no
- $d[t] > d[x] + w(x,t)$? cioè, $2 > 4 + (-2)$? no
- $d[x] > d[y] + w(y,x)$? cioè, $4 > 7 + (-3)$? no
- $d[z] > d[y] + w(y,z)$? cioè, $-2 > 7 + 9$? no
- $d[s] > d[z] + w(z,s)$? cioè, $0 > -2 + 2$? no
- $d[x] > d[z] + w(z,x)$? cioè, $4 > -2 + 7$? no

Tutti gli archi hanno passato la verifica, dunque non sono presenti cicli di peso negativo.

Allora al termine il **sottografo dei predecessori G_π** è un **albero di cammini minimi con radice s**.

La **complessità** dell'algoritmo di Bellman-Ford è **$O(V \cdot E)$** .

Lemma

Dato un grafo orientato $G = (V, E)$ e pesato con funzione peso $w: E \rightarrow R$, e supponendo che G non contenga cicli di peso negativo raggiungibili da s , dopo $|V| - 1$ iterazioni del ciclo di rilassamento, avremo che $d[v] = \delta(s, v)$ per tutti i vertici v raggiungibili da s .

Infatti, considerando un qualunque vertice v raggiungibile da s con il cammino $p = (v_0, v_1, \dots, v_k)$ dove $s = v_0$ e $v_k = v$, dopo $|V|-1$ iterazioni del ciclo di rilassamento, per la *proprietà del rilassamento* abbiamo che: $d[v_k] = \delta(s, v_k) = \delta(s, v)$.

Corollario

Sia $G = (V, E)$ un grafo orientato e pesato con funzione peso $w: E \rightarrow R$ e sorgente s . Allora $\forall v \in V$ esiste un cammino da s a v se e solo se, completato l'algoritmo di Bellman-Ford su G , questo termina con $d[v] < \infty$.

Correttezza dell'algoritmo di Bellman-Ford

Si esegue Bellman-Ford su di un grafo orientato e pesato $G = (V, E)$ con funzione peso $w: E \rightarrow R$ e sorgente $s \in V$. È possibile avere **due risultati**:

1. **Se G non contiene cicli di peso negativo** raggiungibili da s , allora:
 - l'algoritmo restituisce **TRUE**;
 - per tutti i vertici $v \in V$, $d[v] = \delta(s, v)$;
 - il sottografo dei predecessori G_π è un albero di cammini minimi con radice s .
2. **Se invece G contiene un ciclo di peso negativo** raggiungibile da s , allora l'algoritmo restituisce **FALSE**.

Dobbiamo **dimostrare** che, al termine dell'algoritmo, $d[v] = \delta(s, v)$ per tutti i vertici $v \in V$:

- Se v è **raggiungibile** da s , allora l'asserzione è dimostrata dal **lemma precedente**.
- Se v **non è raggiungibile** da s , allora l'asserzione è vera per la **proprietà dell'assenza del cammino**.

La **proprietà del sottografo dei predecessori** implica che G_π è un **albero di cammini minimi**.

Sappiamo che al termine dell'algoritmo, per tutti i vertici $v \in V$, $d[v] = \delta(s, v)$; se inoltre ci ricordiamo cosa dice la *disuguaglianza triangolare* (ovvero che $\delta(s, v) \leq \delta(s, u) + w(u, v)$), allora vale che $d[v] \leq \delta(s, u) + w(u, v)$, e quindi $d[v] \leq d[u] + w(u, v)$.

Quindi nessuno dei controlli che effettuiamo nell'ultimo ciclo for è soddisfatto, pertanto Bellman-Ford restituisce TRUE.

Supponiamo **per assurdo** che il grafo contenga un **ciclo di peso negativo** raggiungibile da s , e che Bellman-Ford restituisca **TRUE**.

Definiamo tale ciclo c come un percorso $(v_0, v_1, \dots, v_k)$ con $v_0 = v_k$, ed essendo di peso negativo significa che la somma del peso di tutti i suoi archi è negativa.

$$\sum_{i=1}^k w(v_{i-1}, v_i) < 0$$

Se Bellman-Ford ha restituito TRUE, significa che per ogni iterazione dell'ultimo ciclo for, effettuata sul ciclo di peso negativo c , vale:

$$d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i) \quad \text{per } i = 1, \dots, k$$

pertanto se consideriamo tale disuguaglianza per tutte le iterazioni, avremo:

$$\sum_{i=1}^k d[v_i] \leq \sum_{i=1}^k (d[v_{i-1}] + w(v_{i-1}, v_i))$$

che per le proprietà delle sommatorie può essere riscritto come:

$$\sum_{i=1}^k d[v_i] \leq \sum_{i=1}^k d[v_{i-1}] + \sum_{i=1}^k w(v_{i-1}, v_i)$$

Ma $\sum_{i=1}^k d[v_i] = \sum_{i=1}^k d[v_{i-1}]$ poiché ogni vertice nel ciclo c compare una sola volta quindi avremo:

$$\begin{aligned} \sum_{i=1}^k \cancel{d[v_i]} - \sum_{i=1}^k \cancel{d[v_{i-1}]} &\leq + \sum_{i=1}^k w(v_{i-1}, v_i) \\ 0 &\leq \sum_{i=1}^k w(v_{i-1}, v_i) \\ \sum_{i=1}^k w(v_{i-1}, v_i) &\geq 0 \end{aligned}$$

che tuttavia **contraddice l'assunzione iniziale** (la somma di tutti i pesi del ciclo negativo risulta essere positiva, che non è possibile proprio per la definizione di ciclo negativo). Quindi se il grafo ha cicli di peso negativo che sono raggiungibili dalla sorgente s , l'algoritmo di Bellman-Ford restituisce **FALSE**.

PRECISAZIONI SUI KRUSKAL, PRISM, DIJKSTRA E BELLMAN-FORD

- **L'algoritmo di Dijkstra può essere usato su grafi non orientati?**

SI, infatti un grafo non orientato è fondamentalmente lo stesso di un grafo orientato ma con archi bidirezionali (dunque per ogni coppia di nodi avremo un ciclo), tuttavia a Dijkstra non interessa se sono presenti cicli perché non ci sono archi negativi.

- **Perché Dijkstra non può essere usato con archi negativi?**

L'algoritmo di Dijkstra afferma che, una volta che un vertice è marchiato come "chiuso" (cioè, è stato estratto dalla coda), l'algoritmo ha trovato il cammino minimo per quel nodo, e quel valore non verrà mai più aggiornato. Ma con i pesi negativi, ciò potrebbe non essere vero.

- **L'algoritmo di Bellman-Ford può essere usato su grafi non orientati?**

NON ESATTAMENTE, perché sebbene un *grafo non orientato* non sia altro che un *grafo orientato* ma con connessioni **bidirezionali** fra i nodi, se tale grafo presenta dei pesi negativi, anche se non ha cicli negativi, l'algoritmo di Bellman-Ford potrebbe comunque rilevarne uno.

Per questo motivo, l'algoritmo di Bellman-Ford può essere per i grafi non orientati solo nel caso non siano presenti pesi negativi.

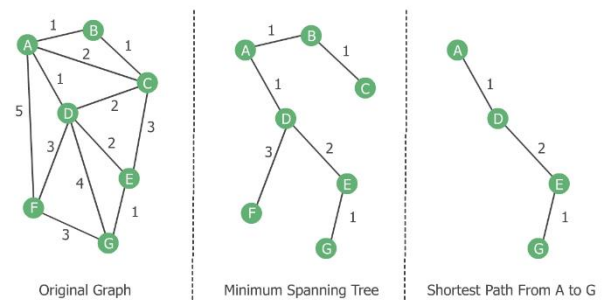
- **Che differenza computazionale c'è fra Dijkstra e Bellman-Ford?**

Dijkstra ha una complessità $O((V+E) \log E)$, mentre per Bellman-Ford è $O(V \cdot E)$, dunque approssimativamente una complessità dell'ordine $n \log n$ contro una quadratica, perciò Dijkstra risulta prestazionalmente migliore.

- **Che differenza c'è tra l'algoritmo di Dijkstra e quello di Kruskal e di Prism?**

L'algoritmo di **Dijkstra** viene utilizzato per individuare la distanza minima da un nodo a tutti gli altri nodi in un grafo pesato.

L'algoritmo di **Kruskal** e quello di **Prism** sono invece utilizzati per individuare il MST.



- **Quali ulteriori differenze ci sono tra Dijkstra e Prism?**

Qualche ulteriore differenza tra Dijkstra e Prism è che l'algoritmo di Dijkstra funziona per grafi sia orientati che non orientati, mentre quello di Prism solo su quelli orientati. Inoltre, l'algoritmo di Prism può gestire anche archi di peso negativo, mentre l'algoritmo di Dijkstra non è in grado di calcolare precisamente le distanze se è presente almeno un arco negativo.

LEZ 21 – BACKTRACKING

IL PROBLEMA DELLE 8 REGINE

Il problema consiste nel posizionare 8 regine su una scacchiera in modo tale che nessuna regina possa essere attaccata dalle altre.

Una regina può spostarsi liberamente in orizzontale, verticale e diagonale.

Il **problema generale** è quello di posizionare **N regine** su una scacchiera **NxN**.

Due regine sono sotto attacco se sono sulla stessa:

- riga;
- colonna;
- diagonale.

La **strategia principale** consiste nell'assegnare ad ogni regina una riga diversa, e poi si prova a verificare le diverse assegnazioni della colonna.

Date N regine, avremo N! permutazioni. Se $N = 8$, allora avremo $8! = 40320$ permutazioni.

Notare che non è possibile inserire più di N regine su una scacchiera NxN, perché ciò farebbe presumere che ci sono più regine sulla stessa riga (dunque non c'è soluzione).

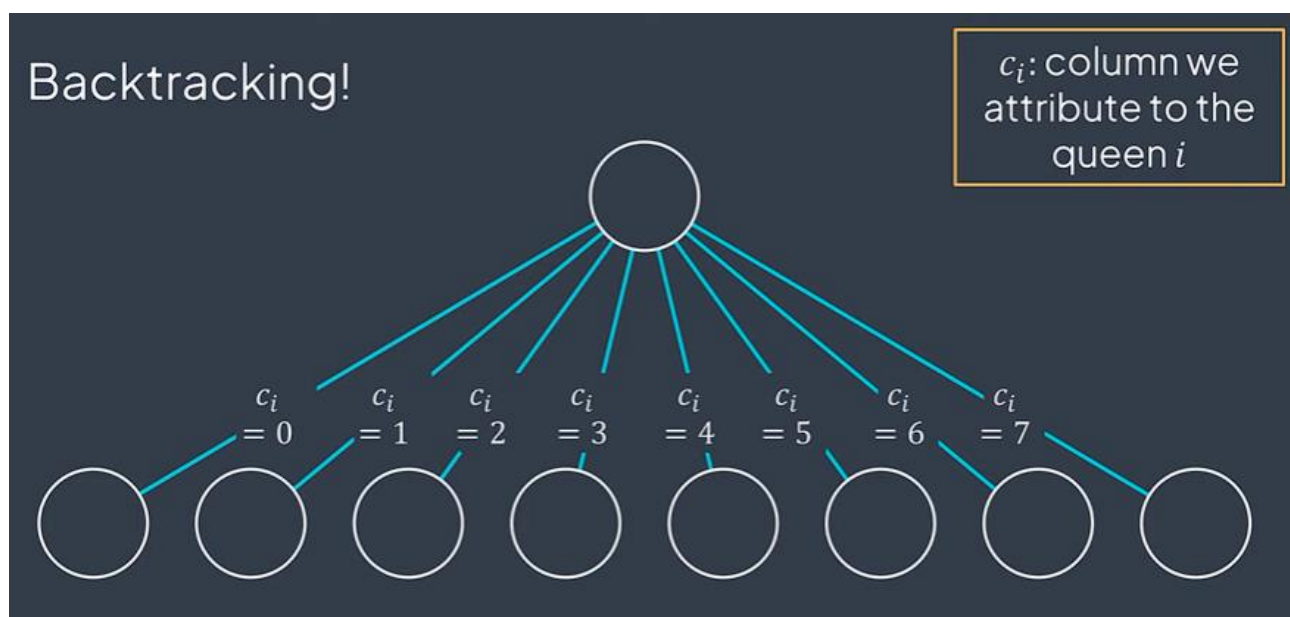
Ovviamente, come può esserci una sola regina per ogni riga, può esserci anche una sola regina per ogni colonna.

Si parte posizionando la prima regina nella prima riga/colonna. Poi si passa alla riga successiva cercando di posizionare la seconda regina. Se non è possibile inserire la regina ad una certa colonna, si prova con la colonna successiva. Se non ci sono colonne successive, si risale e si riprende il procedimento dalla regina precedente.

Possiamo implementare una soluzione a questo problema attraverso il backtracking.

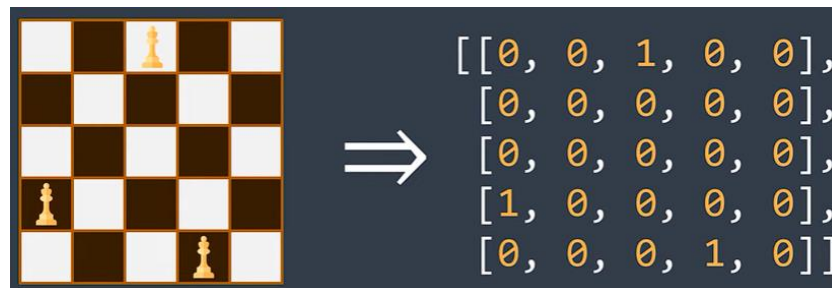
Proviamo tutte le possibili colonne per ogni regina e risaliamo (backtracking) appena ci accorgiamo che la cella non è sicura (è minacciata da un'altra regina).

Per una questione di efficienza, ci conviene effettuare questo controllo prima di posizionare la regina sulla scacchiera.



In C++

La scacchiera sarà rappresentata da una matrice NxN di valori booleani: 0 se la cella è vuota, 1 se contiene una regina.

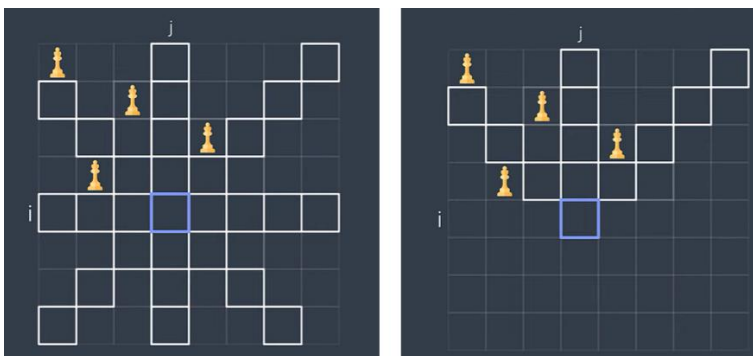


Definiamo una funzione `isSafe(board, i, j)` per verificare se la regina in una certa cella (i,j) è in salvo.

Un approccio naive consisterebbe nel controllare tutte le celle in verticale/orizzontale/diagonale partendo dalla cella (i,j) , tuttavia stiamo inserendo le regine partendo dall'alto verso il basso, dunque non ci interessa controllare le celle sotto (i,j) perché siamo già sicuri che non sarà presente alcuna regina.

Inoltre siccome possiamo inserire una sola regina per riga, è anche inutile controllare la riga i -esima.

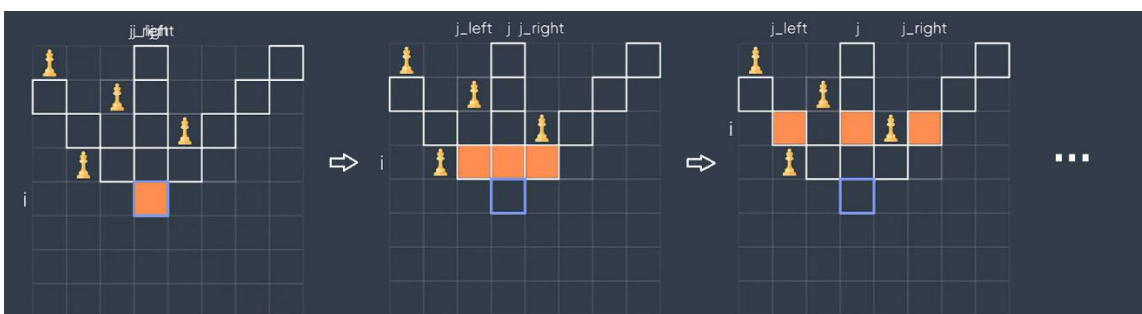
Dunque passeremo da: a: celle da controllare.



Partiamo da i . Possiamo salire di riga decrementandolo.

Per ogni riga, dobbiamo considerare 3 valori: j , j_left , e j_right , che rappresentano le 3 possibili celle che la regina può occupare ad una certa riga i .

j rimane sempre la stessa, mentre j_left e j_right variano man mano che si sale: per ogni riga percorsa, j_left si sposta di una posizione a sinistra, j_right si sposta di una posizione a destra.



verificare se una cella è sicura

```
bool isSafe(bool** board, int n, int i, int j){
    int jLeft = j;
    int jRight = j;
    while(i>=0){
        if(board[i][j] == 1) return false;
        else if(jLeft >=0 && board[i][jLeft] == 1) return false;
        else if(jRight < n && board[i][jRight] == 1) return false;
        i--;
        jLeft--;
        jRight++;
    }
    return true;
}
```

All'inizio sia jLeft che jRight saranno uguali a j. Finché i>=0 (cioè esamineremo fino alla prima riga della scacchiera), la cella corrente sarà sicura se, ad ogni iterazione, nessuna di quelle 3 condizioni si verifica (ovvero se la regina non si trova in nessuna di quelle 3 possibili celle).

Il motivo per cui controlliamo se jLeft e jRight sono entro certi limiti è perché dobbiamo controllare che la cella sia all'interno della scacchiera.

ricerca delle possibili soluzioni

Prendiamo in considerazione la funzione nQueensRec(board, i), che cerca tutte le possibili soluzioni partendo dalla riga i. Nello specifico, considera solo se è possibile inserire la regina nell'i-sima riga, e in caso affermativo richiama se stessa ma sulla riga successiva, ovvero chiama: nQueens(board, i+1).

```
int nQueensRec(bool** board, int n, int i){
    if(i == n)
        return 1;
    else{
        int nbSolutions = 0;
        for(int j=0; j<n; j++){
            if(isSafe(board,n,i,j)){
                board[i][j] = 1;
                nbSolutions += nQueensRec(board, n, i+1);
                board[i][j] = 0;
            }
        }
        return nbSolutions;
    }
}
```

Nel backtracking, abbiamo spesso **due casi base**:

- uno per quando abbiamo trovato la soluzione;
- uno per quando la soluzione che stiamo costruendo non è valida.

In questo caso:

- **successo**: se abbiamo inserito tutte e N le regine.
Abbiamo trovato **1** soluzione da aggiungere al totale.
- **fallimento**: non siamo riusciti a trovare una cella sicura per inserire la regina successiva.

Nel caso ricorsivo consideriamo una variabile `nb_solutions`, che contiene tutte le soluzioni trovate fino a quel momento. All'inizio sarà 0.

(inizio for) Per ognuna delle N colonne, verificheremo se la combinazione (i,j) è sicura: se così fosse, inseriamo l' i -sima regina nella cella (i,j) ed aggiungiamo a `nb_solutions` il valore ritornato dalla chiamata ricorsiva (che è rappresentato dal numero di soluzioni trovate a partire dalla riga $i+1$).

Nel backtracking è anche tipico cancellare le modifiche prima di effettuare un nuovo backtracking. Infatti anche in questo caso dobbiamo eliminare la regina appena inserita (`board[i][j]`), altrimenti avremmo come effetto che tutte le regine verrebbero posizionate sulla stessa riga.

(fine for) Terminato il ciclo `for`, ritorneremo il numero di soluzioni individuate per la riga i .

Notare che il caso base del **fallimento** è implicito all'interno del controllo `isSafe()`: infatti se la cella non è sicura non verrà mai richiamata ricorsivamente la soluzione, e dunque il valore di `nb_solutions` sarà necessariamente 0.

Funzione principale

Utilizziamo una funzione `nQueens(n)` che crea una matrice di booleani $N \times N$ ed effettua la prima chiamata ricorsiva a `nQueensRec()`.

```
int nQueens(int n){
    bool** board = new bool*[n];           //alloca memoria per la matrice
    for(int i=0; i<n; i++)
        board[i] = new bool[n]{0};

    int res = nQueensRec(board, n, 0);      //calcolo la soluzione

    for(int i=0; i<n; i++)                  //dealloca la matrice
        delete[] board[i];
    delete[] board;

    return res;                             //restituisco il risultato
}
```

nb: se si volesse visualizzare anche l'aspetto della scacchiera quando si è trovata una soluzione, basta modificare il caso base ($i == n$) in `nQueensRec`, chiedendo inoltre di stampare `board`.

Extra (percorso hamiltoniano)

Un percorso Hamiltoniano in un grafo non orientato è un percorso che visita ogni vertice esattamente una volta.

Un ciclo Hamiltoniano è un percorso Hamiltoniano tale che esiste un arco tra il primo e l'ultimo vertice del percorso.

LEZ 22 – TABELLE HASH

Ricordando che:

- una hash table è una struttura dati efficiente per l'implementazione di **dizionari**;
- può essere considerata come una **generalizzazione** della nozione di **array** (indirizzamento diretto: una posizione per ogni chiave);
- una hash table è conveniente se il numero di chiavi memorizzate è **piccolo** rispetto al numero totale di chiavi possibili (dimensione proporzionale al numero di chiavi memorizzate);
- la **chiave** viene utilizzata per calcolare la **posizione**.

Usiamo una funzione hash invece di un indirizzamento diretto in quanto quest'ultimo comporterebbe uno spreco di memoria fin troppo importante. Il risparmio di spazio però è ottenuto a costo di un altro problema: le collisioni.

FUNZIONI DI HASHING

Una buona funzione di hash soddisfa l'ipotesi dell'**hashing uniforme semplice**: ogni chiave ha la stessa probabilità di essere mappata in una qualsiasi delle m celle (indipendentemente dalla cella in cui viene mappata ogni altra chiave).

Distinguiamo 3 metodi: *hashing per divisione*, *hashing per moltiplicazione*, e *hashing universale*.

Hashing per divisione

La funzione hash è del tipo $h(k) = k \bmod m$ cioè il valore hash è il resto della divisione di k per m .

Hashing per moltiplicazione

Il metodo della moltiplicazione opera in due passi:

1. si moltiplica la chiave per una costante A , tale che $0 < A < 1$ e si estrae la parte frazionaria del risultato;
2. si moltiplica questo valore per m e si prende la parte intera inferiore.

La funzione di hash è $h(k) = \lfloor m (kA \bmod 1) \rfloor$ dove $kA \bmod 1$ è la parte frazionaria di $kA - \lfloor kA \rfloor$.

Un vantaggio del metodo della moltiplicazione è che il valore di m non è critico. Pertanto si sceglie una potenza di 2, ossia $m = 2^p$.

Sebbene il metodo funziona con qualsiasi valore di A , si propone come valore ottimale il *valore decimale della sezione aurea* (0,6180339887...).

nb: Ferone utilizza un metodo di calcolo prendendo in considerazione direttamente i bit. Oltre i valori considerati prima, quali m , k , A e p dove p rappresenta il numero di bit che determina il numero di celle della hash table, consideriamo due ulteriori valori: w e s .

Calcoliamo A come: $A = \frac{s}{2^w}$

Da cui ricaviamo per formula inversa: $s = A \cdot 2^w$, e dove w è il numero di bit del registro.

es.

$k = 123456$ $p = 14$

$m = 2^p = 2^{14} = 16384$

$w = 32$ $A = 0,618033...$ (sezione aurea),

da cui: $s = A \cdot 2^w = 0,618033... \cdot 4.294.967.296 \approx 2654435769$

calcoliamo: $r_0 = ks \bmod 2^{32} =$

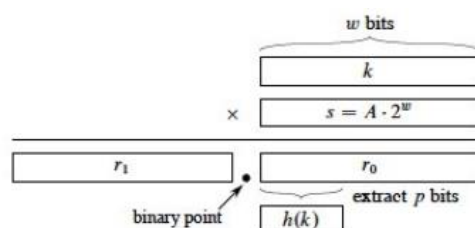
$(123456 \cdot 2.654.435.769) \bmod 2^{32} = 327.706.022.297.664 \bmod 2^{32} = \mathbf{17612864}$

che in binario è: $r_0 = \mathbf{0000\ 0001\ 0000\ 1100\ 1100\ 0000\ 0100\ 0000}$

Dal quale estraiamo i primi 14 bit più significativi:

$0000\ 0001\ 0000\ 11 \rightarrow 100\ 0011$, ovvero 67 in decimale.

Dunque $h(123456) = 67$



Hashing universale

L' hashing universale consiste nel selezionare nel modo casuale la funzione hash da una classe di funzioni accuratamente progettate, all'inizio dell' esecuzione dell'algoritmo.

Si inizia scegliendo un numero primo p sufficientemente grande tale che ogni possibile chiave sia compresa tra 0 e $p - 1$.

Siano $Z_p = \{0, \dots, p - 1\}$ e $Z_p^* = \{1, \dots, p - 1\}$.

Poiché supponiamo che la dimensione dell'universo delle chiavi sia maggiore del numero di celle della tabella hash, abbiamo $p > m$.

Definiamo la funzione hash h_{ab} per ogni $a \in Z_p^*$ e per ogni $b \in Z_p$, avremo che: $h_{ab}(k) = ((ak + b) \bmod p) \bmod m$.

ad esempio: se $p=17$, $m=6$, $a=3$, $b=4$ e $k=8$, avremo che:

$$h_{34}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5 .$$

La *famiglia* di tutte queste funzioni di hash è:

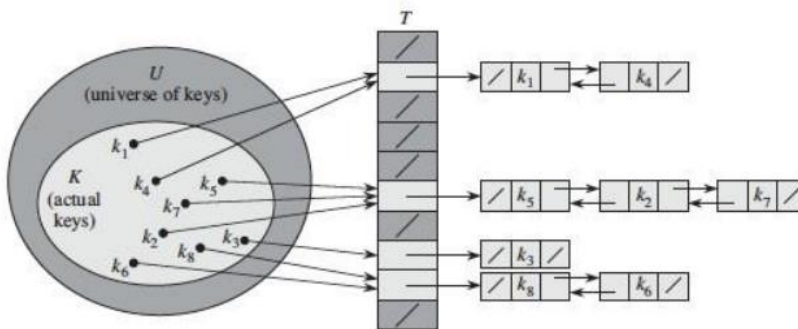
$$\mathcal{H}_{p,m} = \{h_{ab} : a \in Z_p^* \text{ e } b \in Z_p\}$$

METODI DI RISOLUZIONE DELLE COLLISIONI

Distinguiamo due tipi di risoluzione delle collisioni: con concatenamento o con indirizzamento aperto.

Risoluzione delle collisioni per concatenazione

- CHAINED-HASH-INSERT(T, x)
 - **inserisce x in testa alla lista** $T[h(key[x])]$ (si suppone che l'elemento non sia già presente)
- CHAINED-HASH-SEARCH(T, k)
 - ricerca un elemento con chiave k nella lista $T[h(k)]$
- CHAINED-HASH-DELETE(T, k)
 - cancella x dalla lista $T[h(key[x])]$. (più efficiente se le liste sono doppiamente linkate)



Risoluzione delle collisioni con indirizzamento aperto

HASH-INSERT(T, k)

```
1  $i \leftarrow 0$ 
2 repeat  $j \leftarrow h(k, i)$ 
3   if  $T[j] = \text{NIL}$ 
4     then  $T[j] \leftarrow k$ 
5     return  $j$ 
6   else  $i \leftarrow i + 1$ 
7 until  $i = m$ 
8 error "overflow della tabella hash"
```

HASH-SEARCH(T, k)

```
1  $i \leftarrow 0$ 
2 repeat  $j \leftarrow h(k, i)$ 
3   if  $T[j] = k$ 
4     then return  $j$ 
5    $i \leftarrow i + 1$ 
6 until  $T[j] = \text{NIL}$  o  $i = m$ 
7 return NIL
```

Cancellazione

La cancellazione risulta essere più difficile. Se si prevede che le chiavi debbano essere cancellate, si preferisce gestire le collisioni attraverso il concatenamento.

• indirizzamento aperto, tipi di scansione

In genere si usano tra approssimazioni:

- scansione lineare;
- scansione quadratica;
- hashing doppio.

•• Scansione linerare

Sia $h' : U \rightarrow \{0, 1, \dots, m-1\}$ una funzione hash ordinaria, la *scansione lineare* usa la funzione hash: $h(k, i) = (h'(k) + i) \bmod m$ per $i = 1, \dots, m$.

In poche parole, io genero la funzione hash tramite h' , se è occupata avanzo di una posizione (incrementando la i).

•• Scansione quadratica

Sia $h' : U \rightarrow \{0, 1, \dots, m-1\}$ una funzione hash ordinaria, la *scansione quadratica* usa la funzione hash: $h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$.

In poche parole, la posizione iniziale esaminata è $T[h'(k)]$, poi le posizioni successivamente esaminate sono distanziate da quantità che dipendono in modo quadratico dal numero di scansione i .

•• Doppio Hashing

Siano $h_1 : U \rightarrow \{0, 1, \dots, m-1\}$ e $h_2 : U \rightarrow \{0, 1, \dots, m-1\}$ due funzioni hash, l'*hashing doppio* usa la funzione hash: $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$.

Questo è il metodo che più si avvicina alla proprietà di uniformità. Inoltre risolve i problemi di agglomerazione che si verificano nella scansione lineare e in quella quadratica.

L'idea è quella di partire da un valore hash e fare in modo che i valori delle posizioni successive dipendano proprio da un'altra funzione. Grazie a questa caratteristica **non vi è agglomerazione** proprio perché la distanza cambia ogni volta.

Tuttavia, dobbiamo assicurarci che h_2 non sia multiplo di m (numero di slot della tabella). Infatti, se m e $h_2(k)$ avessero un massimo comun divisore, allora la sequenza cercata per k non esaminerebbe tutti gli elementi.

Un modo pratico è di scegliere m potenza di 2 e scegliere h_2 in modo che produca un numero dispari.

Per esempio: $h_1(k) = k \bmod m$ e $h_2(k) = 1 + (k \bmod m)$